

Advanced System Design and SRE

Concepts, Case Studies and Labs

Prof. Vipul Kushwaha

Table of contents

1	System Design Fundamentals and Core Concepts	5
1.1	Key Components	5
1.2	HLD vs LLD Distinction	6
1.2.1	High-Level Design (HLD)	6
1.2.2	Low-Level Design (LLD)	6
1.3	The Scaling Paradigm	7
1.4	Back-of-the-envelope Estimation	7
1.5	Availability and Scalability Metrics	9
1.6	Availability Numerical Examples	9
1.6.1	Solved Numerical Examples	9
1.7	Summary of Availability Targets	10
1.8	Scalability Metrics: Throughput and Bandwidth	11
1.8.1	Throughput (Capacity)	11
1.8.2	Bandwidth	11
1.9	Throughput vs. Bandwidth	12
1.9.1	Little's Law	12
1.10	Solved Numericals for Latency	12
1.10.1	Throughput vs. Latency: The SRE Perspective	13
1.11	Distributed Data, CAP, and PACELC	13
1.11.1	Introduction to NoSQL	13
1.11.2	CAP Theorem vs. PACELC Theorem	13
1.11.3	The CAP Theorem	14
1.11.4	The PACELC Theorem	15
2	Data Partitioning, Sharding & Consistent Hashing	17
2.1	Partitioning vs. Sharding	17
2.1.1	The Routing Problem & Consistent Hashing	17
2.2	The Four Families of NoSQL	17
2.2.1	1. Key-Value Stores (The “Giant Hash Map”)	17
2.2.2	2. Document Stores (The “JSON Warehouse”)	18
2.2.3	3. Column-Family Stores (“Wide Column Spreadsheet”)	18
2.2.4	4. Graph Databases (The “Relationship Map”)	18
2.3	Partitioning vs. Sharding	19
2.3.1	Partitioning (The Logical Split)	19
2.3.2	Sharding (The Physical Split)	19
2.4	Consistent Hashing	19
2.4.1	The Modulo Operator (The Naive Approach)	19
2.4.2	The Anecdote: The Round Table Dinner	19
2.4.3	The Solution: Consistent Hashing	20
2.5	Replication Mechanics	20
2.5.1	Sharding Key Design	20

2.6	Replication & Consistency	21
2.6.1	Replication Architectures	21
2.6.2	Quorum Consensus	21
3	Storage Technologies & Filesystems	22
3.1	NAS (Network Attached Storage)	22
3.2	SAN (Storage Area Network)	22
3.3	Technical Comparison	22
3.3.1	The Hardware Reality & Latency	24
3.3.2	Storage Architectures (Block vs. File vs. Object)	24
3.4	RAID	24
3.4.1	RAID 0 (Striping)	24
3.4.2	RAID 1 (Mirroring)	24
3.4.3	RAID 5 (Striping with Distributed Parity)	24
3.4.4	RAID 6 (Striping with Dual Distributed Parity)	24
3.4.5	RAID 10 (Striping of Mirrors)	28
3.5	RAID Estimation Numerical Example	28
4	Parallel Databases and Query Evaluation	32
4.1	The “Why” of Parallelism – Speedup vs. Scaleup	32
4.2	Hardware Architectures for Parallel Databases	32
4.3	Parallel Query Evaluation	33
4.3.1	The Parallel Join – A Deep Dive	34
5	Performance Caching & Load Balancing	35
5.1	The “Why” of Caching – Physics and Locality	35
5.1.1	Cache Eviction	35
5.1.1.1	LRU (Least Recently Used) – The Industry Workhorse	36
5.1.1.2	LFU (Least Frequently Used) – The Heavy Hitter	36
5.1.1.3	Real-World Architecture & Summary	37
5.1.2	Caching Strategies	37
5.1.2.1	The Read Strategies (How data gets INTO the cache)	37
5.1.2.2	Cache-Aside (Lazy Loading)	37
5.1.2.3	Read-Through	37
5.1.3	The Write Strategies (How data stays SYNCED)	39
5.1.3.1	Write-Through	39
5.1.3.2	Write-Behind (Write-Back)	40
5.1.3.3	Write-Around	40
5.1.3.4	Distributed Caching in Redis Clusters	41
5.1.3.5	The Nightmare of Cache Invalidation	43
5.1.3.6	Failsafes and The Thundering Herd	43
5.2	Load Balancing	44
5.2.1	Layer 4 Load Balancing	44
5.2.2	Layer 7 Load Balancing	45
5.2.3	Static Load Balancing Algorithms	46
5.2.3.1	Round Robin	46
5.2.3.2	Weighted Round Robin	46
5.2.4	Dynamic Load Balancing Algorithms	47
5.2.4.1	Least Connections	47
5.2.4.2	Weighted Least Connections	47
5.2.5	Load Balancing Strategy Comparison	47
5.2.5.1	Pros, Cons, and Use Cases	50
5.2.6	The Hybrid Reality	50
5.2.6.1	The Intersection (Mixing the Dials)	51

5.2.7	The Enterprise Hierarchy (The “Stack”)	51
5.2.7.1	Tier 1: Global DNS Load Balancing (Static)	51
5.2.7.2	Tier 2: The Edge Layer 4 Load Balancer (Dynamic)	51
5.2.7.3	Tier 3: The Internal Layer 7 Load Balancer (Dynamic)	52
5.3	The Proxy – Forward vs. Reverse	52
5.3.1	The Forward Proxy (Protecting the Client)	52
5.3.2	The Reverse Proxy (Protecting the Server)	52
5.3.3	Proxy Comparison: Forward vs. Reverse	53
5.3.3.1	Forward Proxy Tools	53
5.3.3.2	Reverse Proxy Tools	53
5.4	Throttling and Rate Limiting	54
5.4.1	Rate Limiting	54
5.4.2	Throttling	54
5.4.3	The Core Algorithms	54
5.4.3.1	The Token Bucket	54
5.4.3.2	2. The Leaky Bucket	54
5.4.3.3	Fixed Window Counter	56
5.4.3.4	Sliding Window	56
5.4.4	Comparison of Rate Limiting Algorithms	56
5.4.4.1	Pros, Cons, and Applications	57
5.4.5	The User Experience & HTTP Headers	57
5.4.6	Industry grade use case	60
5.5	Performance Optimization	61
5.5.1	Connection Pooling	61
5.5.2	Content Compression (Gzip/Brotli)	61
5.5.3	Reducing I/O (Disk and Network)	62
5.5.4	Database Query Optimization	62
6	Asynchronous Processing and Communication	63
6.1	Message Queues, Brokers, and Routing Models	63
6.1.1	Point-to-Point (Queue) Model	63
6.1.2	Publish-Subscribe (Pub/Sub) Model	63
6.1.3	RabbitMQ	64
6.1.3.1	Why Introduce RabbitMQ into a System?	64
6.1.3.2	Exchange Types	65
6.1.4	Apache Kafka	66
6.1.4.1	Why Introduce Kafka into a System?	70
6.1.5	Kafka vs. RabbitMQ: Choosing the Right Broker	70
6.2	Data Ingestion and Stream Processing	71
6.2.1	The Evolution of Data Processing: Batch vs. Stream	71
6.2.2	Data Ingestion: The Gateway to the Stream	72
6.3	Core Concepts of Stream Processing	72
6.3.1	Kafka Streams: The Lightweight Native Engine	73
6.3.2	Apache Flink: The Heavyweight Distributed Engine	73
6.3.3	REST (Representational State Transfer)	77
6.3.4	gRPC & Protocol Buffers	77
6.3.5	gRPC vs. REST	77
6.4	Real-Time Communication Patterns	78
6.4.1	Comparison of Real-Time Communication Patterns	78
7	Security, Monitoring, and Reliability	85
7.1	Observability Architecture	85
7.2	Telemetry	87
7.3	The Pillars of Observability and MELT	87

7.3.1	Metrics	89
7.3.2	Events	89
7.3.3	Logs	89
7.3.3.1	Structured vs. Unstructured Logging	90
7.3.3.2	Log Levels	90
7.3.3.3	Centralized Logging (The ELK Stack)	90
7.3.4	Traces	90
7.4	Distributed Tracing and Jaeger	91
7.4.1	The Anatomy of a Trace	91
7.4.2	Visualizing the Trace Context	92
7.4.3	Introduction to Jaeger	93
7.4.3.1	Jaeger Backend Architecture	94
8	Case Studies	95
8.1	The Twitter “Fail Whale”	95
8.2	Netflix: Global State Persistence with Apache Cassandra	96
8.3	Amazon’s \$100 Million Shopping Cart (Dynamo)	96
8.4	Facebook: Haystack (Object Storage Efficiency)	97
8.5	Discord: Scaling the Message Store	97
8.6	Dropbox’s “Magic Pocket”	97
9	Labs	99
9.1	Experiment 1	99
9.2	Experiment 2	99
9.3	Experiment 3	99
9.4	Experiment 4	99
	References	100

Chapter 1

System Design Fundamentals and Core Concepts

System design is the process of defining the architecture, components, modules, interfaces, and data flows of a software/hardware system to meet functional and non-functional requirements like scalability and performance.

1.1 Key Components

When designing large-scale distributed systems, engineers rely on a standard set of building blocks. Think of these components as the architectural vocabulary used to construct everything from a simple web app to massive platforms like Netflix or Google.

Here is a breakdown of the key components of system design, moving from the user's screen down to the data center.

1. The Entry Point

- **DNS (Domain Name System):** The “phonebook” of the internet. It translates human-readable domain names (like `google.com`) into the IP addresses (like `192.0.2.1`) where the servers actually live.
- **CDN (Content Delivery Network):** A globally distributed network of proxy servers. It caches static assets (images, videos, HTML, CSS, JavaScript) geographically closer to the user to reduce latency and save backend bandwidth.

2. Traffic Management

- **Load Balancer (LB):** The traffic cop. It sits in front of your servers and distributes incoming network traffic across multiple backend servers to ensure no single machine gets overwhelmed. It also detects server failures and routes traffic only to healthy nodes.
- **API Gateway:** The single entry point for all client requests in a microservices architecture. It handles routing, authentication, rate limiting, and SSL termination before the request ever hits your application code.

3. The Compute Layer (Application)

- **Web Servers:** Handle HTTP requests and serve basic web content.
- **Application Servers:** The “brains” of the operation. This is where your business logic, algorithms, and data processing live. In modern architectures, these are often broken down into independent **Microservices** rather than one giant monolith.

4. The Data Layer

- **Relational Databases (SQL):** Structured data with strict schemas and ACID compliance (e.g., PostgreSQL, MySQL). Best for financial transactions, user accounts, and data with clear relationships.
- **Non-Relational Databases (NoSQL):** Flexible schemas optimized for specific scale or access patterns (e.g., MongoDB, Cassandra, DynamoDB). Best for massive scale, rapid iteration, and unstructured data.
- **Cache:** A temporary, in-memory data store (e.g., Redis, Memcached) that serves frequently requested data at lightning speed (nanoseconds) to reduce the load on the primary database.

5. The Storage Layer

- **Object Storage:** Stores unstructured data as “blobs” (Binary Large Objects) via APIs. Ideal for user uploads, profile pictures, videos, and backups (e.g., AWS S3).
- **Block/File Storage:** Raw disk storage attached directly to compute instances, used primarily for database storage and OS files (e.g., AWS EBS, EFS).

6. Asynchronous Communication

- **Message Queues / Event Streams:** Buffers that hold messages between different parts of the system (e.g., Kafka, RabbitMQ, SQS). They decouple microservices, allowing them to communicate asynchronously. If a user uploads a video, the web server puts a “Process Video” message on the queue and immediately returns a success message to the user, while a background worker picks up the task later.

7. Observability & Reliability

- **Logging, Monitoring, and Alerting:** Systems (like Elasticsearch, Logstash, Kibana (ELK), Prometheus, or Datadog) that collect metrics (CPU usage, latency, error rates) and logs from all servers. They provide dashboards to visualize system health and trigger alerts if something breaks.

1.2 HLD vs LLD Distinction

1.2.1 High-Level Design (HLD)

Scale Estimation: Back-of-the-envelope calculations for Traffic (QPS) and Storage (TB).

System Architecture: Deployment topology and the selection of core architectural patterns.

Database Selection: Choosing between SQL vs. NoSQL based on data partitioning and sharding needs.

High-Level Components: Identification of Load Balancers, Proxies, and Message Queues (e.g., Kafka or RabbitMQ).

Non-functional Requirement Justification: Strategic trade-offs using CAP and PACELC theorems to meet availability and scalability targets.

1.2.2 Low-Level Design (LLD)

Schema Design: Detailed ER diagrams, normalization/denormalization strategies, and sharding key selection.

Algorithm Logic: Implementation of specific patterns like Consistent Hashing, LRU/LFU caching, or Rate Limiting (Token Bucket).

Module Logic: Method signatures, class diagrams, and specific data structures for microservices.

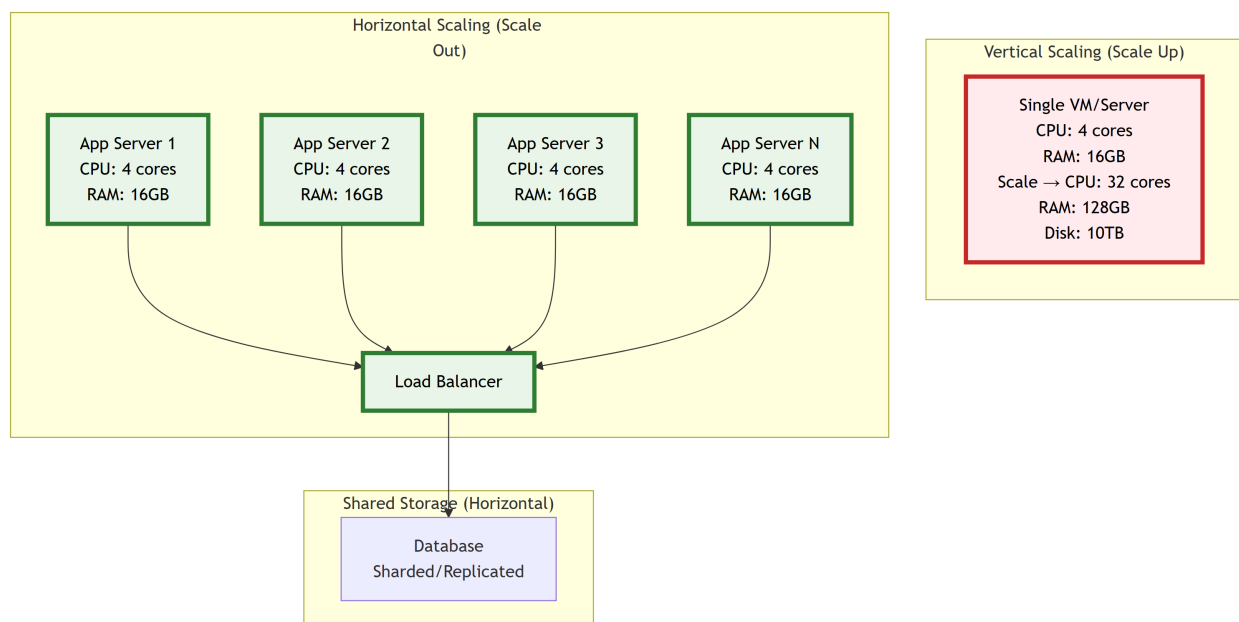
Resilience Patterns: Implementation of Circuit Breakers (Closed/Open/Half-Open) and retry logic.

Security & Auth: Detailed design for JWT, OAuth 2.0, or session-based authentication.

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Focus	System architecture & big picture	Detailed implementation specs
Audience	Architects, stakeholders, devs	Developers, testers
Key Artifacts	System diagrams, component overview, APIs, tech stack	Class diagrams, ER diagrams, sequence diagrams, algorithms
Level of Detail	What to build, why, how parts connect	How exactly to code each module
Inputs	Business requirements, NFRs	HLD document
Outputs	Architecture blueprint	Developer-ready specs
Examples	Deployment topology, data flow, scalability plan	Method signatures, DB schemas, error handling
Timing	Early design phase	Post-HLD, pre-coding

1.3 The Scaling Paradigm

- **Vertical Scaling (Scaling Up):** Adding more CPU/RAM to a single machine. It is limited by hardware ceilings (e.g., upgrading a single traditional RDBMS).
- **Horizontal Scaling (Scaling Out):** Adding more standard machines to a network to share the load. This is the foundation of distributed computing and NoSQL.



1.4 Back-of-the-envelope Estimation

Back-of-the-envelope estimations are a critical component of System Design. This section serves as the “Proof of Concept” before any code is written. It allows an SRE to anticipate hardware saturation and a Designer to choose the right database.

In the context of the **Twitter Fail Whale** Section 8.1 case study given at the end, these numbers explain *why* the original monolithic MySQL approach was mathematically destined to fail.

To estimate the requirements, we first establish a set of realistic assumptions based on Twitter’s massive scale.

1. Core Assumptions

- **DAU (Daily Active Users):** 300 million.
- **Average Tweets per User/Day:** 2 tweets.
- **Read-to-Write Ratio:** 100:1 (Twitter is highly read-heavy).
- **Follower Count:** On average, 200 followers per user.
- **Media Content:** 10% of tweets contain a 5MB video; 20% contain a 200KB image.

2. Traffic (QPS - Queries Per Second)

We need to calculate both the **Write QPS** (posting tweets) and the **Read QPS** (viewing timelines).

Write QPS (Posting Tweets)

- **Daily Tweets:** $300 \text{ million users} \times 2 \text{ tweets/day} = 600 \text{ million tweets/day}$.
- **Average QPS:** $600 \text{ million} / 86,400 \text{ seconds (1 day)} \approx 7,000 \text{ QPS}$.
- **Peak QPS:** $2 \times \text{Average QPS} \approx 14,000 \text{ QPS}$.

Read QPS (Timeline Viewing)

- **Daily Reads:** $600 \text{ million tweets} \times 100 \text{ (Read-to-Write Ratio)} = 60 \text{ billion reads/day}$.
- **Average QPS:** $60 \text{ billion} / 86,400 \approx 700,000 \text{ QPS}$.

SRE Insight: The massive gap between 7,000 Write QPS and 700,000 Read QPS is why Twitter moved to the **Fan-out/Push model**. You cannot query a relational database 700,000 times a second for complex joins without significant latency.

3. Storage Estimation

How much data must we store daily and over a 5-year period?

Text Content

- Assume each tweet (including metadata like IDs and timestamps) is **280 bytes**.
- **Daily Storage:** $600 \text{ million tweets} \times 280 \text{ bytes} \approx 168 \text{ GB/day}$.

Media Content (Images & Video)

- **Images:** $(600 \text{ million} \times 20\%) \times 200 \text{ KB} = 120 \text{ million} \times 200 \text{ KB} = 24 \text{ TB/day}$.
- **Video:** $(600 \text{ million} \times 10\%) \times 5 \text{ MB} = 60 \text{ million} \times 5 \text{ MB} = 300 \text{ TB/day}$.

Total Daily Storage: $\approx 324 \text{ TB/day}$. **5-Year Storage:** $324 \text{ TB} \times 365 \times 5 \approx 591 \text{ PB (Petabytes)}$.

4. Bandwidth Estimation

Bandwidth is split into **Ingress** (incoming data) and **Egress** (outgoing data).

- **Ingress:** $324 \text{ TB} / 86,400 \text{ seconds} \approx 3.75 \text{ GB/s}$.
- **Egress:** $3.75 \text{ GB/s (Ingress)} \times 100 \text{ (Read-to-Write Ratio)} \approx 375 \text{ GB/s}$.

5. Memory Estimation (Caching)

For SREs, caching is about the **80/20 Rule**: 80% of traffic comes from 20% of the data. We want to cache the top 20% of tweets requested daily to minimize database hits.

- **Daily Tweets (Text only):** 168 GB.
- **Cache Size (20%):** $168 \text{ GB} \times 0.2 \approx 33.6 \text{ GB}$.

While 33.6 GB fits on a single modern server, in practice, we replicate this across multiple regions for high availability (HA).

6. Fan-out Estimation (The SRE Challenge)

The “Fan-out” is the most computationally expensive part of Twitter.

- **Total Deliveries/Second:** $\text{Write QPS} \times \text{Average Followers} = 7,000 \times 200 = 1.4 \text{ million updates/second}$.
- **Challenge:** If a celebrity with 10 million followers tweets, that single write triggers 10 million cache updates. This is the **Hot Key** problem.

Summary Table

Aspect	Estimated Value
Daily Active Users (DAU)	300 Million
Write QPS	7,000 (Peak: 14,000)
Read QPS	700,000
Daily Text Storage	168 GB
Daily Media Storage	324 TB
5-Year Total Storage	~600 PB
Cache Requirement	~34 GB (Top 20% Text)

1.5 Availability and Scalability Metrics

This section is the cornerstone of “**Architecting Scalability**”. In both academic theory and industrial practice (SRE), we must move beyond qualitative terms like “fast” or “reliable” and embrace quantitative metrics. These metrics allow us to define Service Level Objectives (SLOs) and justify architectural changes to stakeholders.

Availability: The “Nines” Framework

Availability is the percentage of time a system remains operational and functional to process requests. In SRE, this is often measured by the “Number of Nines.”

The Mathematical Definition

The traditional formula for availability is based on the relationship between failure and recovery:

$$Availability = \frac{MTBF}{MTBF + MTTR}$$

MTBF (Mean Time Between Failures): The average time the system runs before crashing. **MTTR (Mean Time To Repair):** The average time taken to bring the system back online after a failure.

1.6 Availability Numerical Examples

In the **Advanced System Design** curriculum, availability is a key metric used to define Service Level Objectives (SLOs). It is mathematically derived from the relationship between how long a system stays up and how quickly it can be recovered.

1.6.1 Solved Numerical Examples

Example 1: Basic Availability Calculation

Scenario: A distributed database server at DJSCE runs for an average of 500 hours before crashing. On average, it takes the SRE team 2 hours to reboot the server and restore the database state. Calculate the availability percentage.

- **Given:** MTBF = 500 hours, MTTR = 2 hours.
- **Calculation:**

$$Availability = \frac{500}{500 + 2} = \frac{500}{502} \approx 0.9960$$

- **Result: 99.60%**

- **Design Insight:** This system falls between “Two Nines” and “Three Nines.” According to our course metrics, this is suitable for internal tools but may not be resilient enough for core infrastructure.

Example 2: Determining MTTR for a “Three Nines” SLO

Scenario: You are designing a News Feed system and have promised a **99.9% (Three Nines)** availability SLO. If your hardware history shows that a node fails once every 30 days (720 hours), what is the maximum allowable MTTR?

- **Given:** Availability = 0.999, MTBF = 720 hours.
- **Formula Rearrangement:** $MTTR = \frac{MTBF \times (1 - \text{Availability})}{\text{Availability}}$
- **Calculation:**

$$MTTR = \frac{720 \times (1 - 0.999)}{0.999} = \frac{720 \times 0.001}{0.999} \approx 0.7207 \text{ hours}$$

Result: ~43 minutes.

SRE Action: To maintain this SLO, your monitoring and alerting stack (Prometheus/Alertmanager) must detect the failure and trigger an automated recovery script within 43 minutes.

Example 3: Impact of Redundancy on MTBF

Scenario: A single disk has an MTBF of 100,000 hours. You decide to use **RAID 1 (Mirroring)** with two disks. For the entire array to fail, both disks must fail. (Note: In a simplified academic model, the MTBF of a redundant system is significantly higher).

Concept: While RAID 1 doesn’t change the MTBF of an *individual* disk, it drastically reduces the **MTTR** of the *service* because the system stays online while the failed disk is replaced.

System Design Choice: This redundancy is a primary method for achieving “Five Nines” (99.999%) in critical data modeling and storage systems.

1.7 Summary of Availability Targets

Availability	Annual Downtime	Justification/Service Type
99%	3.65 days	Internal experimental tools
99.9%	8.77 hours	Standard SaaS, Content sites
99.99%	52.6 minutes	Banking, E-commerce Checkout
99.999%	5.26 minutes	Core Cloud Infrastructure

2. Scalability: Measuring the Load

Scalability is the ability of a system to handle increased load by adding resources. We measure this through **Throughput** and **Latency**.

2.1 Throughput (Capacity)

Throughput measures the number of units of work a system can process in a given time.

- **Web Servers:** Measured in **QPS** (Queries Per Second) or **RPS** (Requests Per Second).
- **Databases:** Measured in **TPS** (Transactions Per Second).
- **Data Pipelines:** Measured in **Events/Sec** or **GB/s**.

2.2 Latency vs. Response Time

- **Latency:** The time a request spends waiting to be handled (the “delay”).

- **Response Time:** The total time from the client sending the request to receiving the response (includes processing + network round trip).

2.3 The Percentile Metric (P_{95} , P_{99} , $P_{99.9}$)

Averages (Means) are misleading in System Design because they hide **outliers** (the “Tail Latency”).

- **P50 (Median):** 50% of requests are faster than this value.
- **P99:** 99% of requests are faster than this. Only 1% of users experience a slower response.
- **Why it matters:** If your P_{99} is 2 seconds, 1 out of every 100 users is having a frustrating experience. For a system like Twitter with 700k Read QPS, that’s 7,000 unhappy users every second.

3. Reliability Metrics (The SRE Perspective)

Reliability is the probability that a system will perform its intended function without failure for a specific period.

3.1 The Golden Signals

As defined in the Google SRE Handbook, these four signals must be monitored:

1. **Latency:** Time taken to service a request.
2. **Traffic:** Demand placed on the system (QPS).
3. **Errors:** The rate of requests that fail (explicitly, implicitly, or by policy).
4. **Saturation:** How “full” your service is (e.g., CPU/Memory usage).

To visualize the relationship between Load and Performance, we use a **Load-Performance Curve**.

5. Summary

- **Availability** is about “Uptime.”
- **Scalability** is about “Growth.”
- **Reliability** is about “Trust.”

1.8 Scalability Metrics: Throughput and Bandwidth

In the **Advanced System Design** curriculum, scalability is defined as the ability of a system to handle increased load by adding resources. To quantify this “growth” capability, we move beyond qualitative descriptions and rely on precise measurements of data movement and processing power.

1.8.1 Throughput (Capacity)

Throughput measures the total number of units of work a system can process successfully in a given time interval. It is the primary metric for determining the **capacity** of your high-level design.

- **Web Servers & APIs:** Measured in **QPS** (Queries Per Second) or **RPS** (Requests Per Second).
- **Databases:** Measured in **TPS** (Transactions Per Second).
- **Data Pipelines & Streaming:** Measured in **Events Per Second** or **Messages Per Second**.

1.8.2 Bandwidth

Bandwidth represents the maximum rate of data transfer across a network or between system components. In a distributed architecture, we must account for data moving in two directions:

- **Ingress:** Data flowing *into* your system (e.g., users uploading photos or posting tweets).
- **Egress:** Data flowing *out* of your system (e.g., users downloading videos or viewing their feeds).

In the context of **Advanced System Design**, understanding the distinction between data capacity and actual data movement is vital for accurate scale estimation and high-level design justification.

1.9 Throughput vs. Bandwidth

While these terms are often used interchangeably in casual conversation, they represent two different constraints in a distributed system.

- **Bandwidth:** This is the **theoretical maximum** capacity of a communication channel. It represents the “width of the pipe” or the maximum number of bits that can be transmitted per second under ideal conditions.
- **Throughput:** This is the **actual** amount of data that is successfully processed or transmitted over a period of time. It is the “real-world flow” and is always less than or equal to the bandwidth due to overhead, network congestion, and hardware limitations.

Feature	Bandwidth	Throughput
Nature	Theoretical Potential	Actual Performance
Measured In	bits per second (bps)	QPS, TPS, or bps
Primary Goal	Define Capacity	Measure Success Rate

1.9.1 Little’s Law

Little’s Law is a fundamental theorem in queuing theory used by SREs to understand the relationship between concurrency, throughput, and latency. It is expressed as:

$$L = \lambda \times W$$

L (Concurrency): The average number of requests currently being processed in the system.

λ (Throughput): The average arrival rate of requests (e.g., Requests per Second).

W (Latency): The average time a request spends in the system (Response Time).

SRE Application: If you know your server can handle 1,000 concurrent connections (L) and your average response time is 100ms (W), you can calculate your maximum throughput (λ) as $1,000/0.1 = 10,000$ RPS.

1.10 Solved Numericals for Latency

Example 1: Calculating Network Latency (Round Trip Time)

Problem: A client in Mumbai sends a request to a server in Virginia (approx. 13,000 km away). If the speed of light in fiber is 2×10^8 m/s, calculate the theoretical minimum network latency.

Distance (One Way): 13,000 km = 1.3×10^7 meters. **One-way Latency:** $1.3 \times 10^7 / 2 \times 10^8 = 0.065$ seconds (65 ms). **Round Trip Time (RTT):** $65 \text{ ms} \times 2 = 130$ ms.

Design Insight: This explains why we use **Distributed Caching (Redis)** and **Edge Locations** to bring data closer to users.

Example 2: Impact of Tail Latency (P99)

Problem: A system handles 100,000 QPS. If the P99 latency is 2 seconds, how many users per second are experiencing high latency?

- **Percentage of outliers:** $100\% - 99\% = 1\%$.
- **Impacted Requests:** $100,000 \times 0.01 = 1,000$ requests/sec.
- **Design Insight:** Monitoring **P99 latency spikes** is a core **SRE objective** to ensure system reliability.

Example 3: Little’s Law in Action

Problem: An API has an average throughput of 500 Requests Per Second. On average, there are 25 requests being processed at any given time. What is the average latency (W)?

- **Given:** $\lambda = 500$, $L = 25$.
- **Formula:** $W = L/\lambda$.
- **Calculation:** $25/500 = 0.05$ seconds = **50 ms**.

1.10.1 Throughput vs. Latency: The SRE Perspective

A critical **SRE principle** is understanding that throughput and latency are related but distinct. As throughput increases and a system reaches its **Saturation Point**, latency begins to spike exponentially.

- **P99 Latency:** For high-throughput systems, we focus on the 99th percentile to ensure that 99% of requests are processed within a specific time, even as bandwidth utilization peaks.
- **Traffic Signals:** Monitoring traffic (QPS) is one of the **Four Golden Signals** used to debug performance bottlenecks.

Summary Table: Scalability Metrics

Metric	Measured Unit	Design Justification
Throughput	QPS, TPS, RPS	Justifies the number of horizontal nodes and load balancer algorithms.
Bandwidth	Mbps, GB/s	Justifies the selection of communication protocols (REST vs. gRPC) and compression (Gzip).
Saturation	% CPU/RAM	Indicates the “Knee of the Curve” where throughput plateaus and failures begin.

1.11 Distributed Data, CAP, and PACELC

1.11.1 Introduction to NoSQL

NoSQL databases reject rigid RDBMS schemas in favor of data models optimized for specific access patterns.

Key-Value Stores (e.g., Redis, DynamoDB): $O(1)$ lookups. Opaque values accessed via a unique key. Used for caching and session management.

Document Stores (e.g., MongoDB): Stores structured, schema-flexible data (JSON/BSON). Fields can be indexed. Used for content management and user profiles.

Wide-Column / Column-Family (e.g., Cassandra): Optimized for massive write throughput and compression. Used for time-series data and logs.

Graph Databases (e.g., Neo4j): Nodes and edges represent data and relationships natively. $O(1)$ relationship traversal. Used for social networks and fraud detection rings.

1.11.2 CAP Theorem vs. PACELC Theorem

The CAP theorem is foundational but incomplete because network partitions are rare. We must also define database behavior during *normal* operations.

The CAP Theorem (During a Partition): In a distributed data store, you can only guarantee two of the following three traits simultaneously:

- **Consistency (C):** Every read receives the most recent write or an error.
- **Availability (A):** Every request receives a non-error response, but without the guarantee of containing the most recent write.
- **Partition Tolerance (P):** The system operates despite network failures between nodes. (Note: Because networks *will* fail, distributed systems must always be P. The real choice is between C and A).

1.11.3 The CAP Theorem

In distributed systems, the CAP Theorem (Brewer's Theorem) states that it is impossible for a web service to provide more than two out of three of the following guarantees:

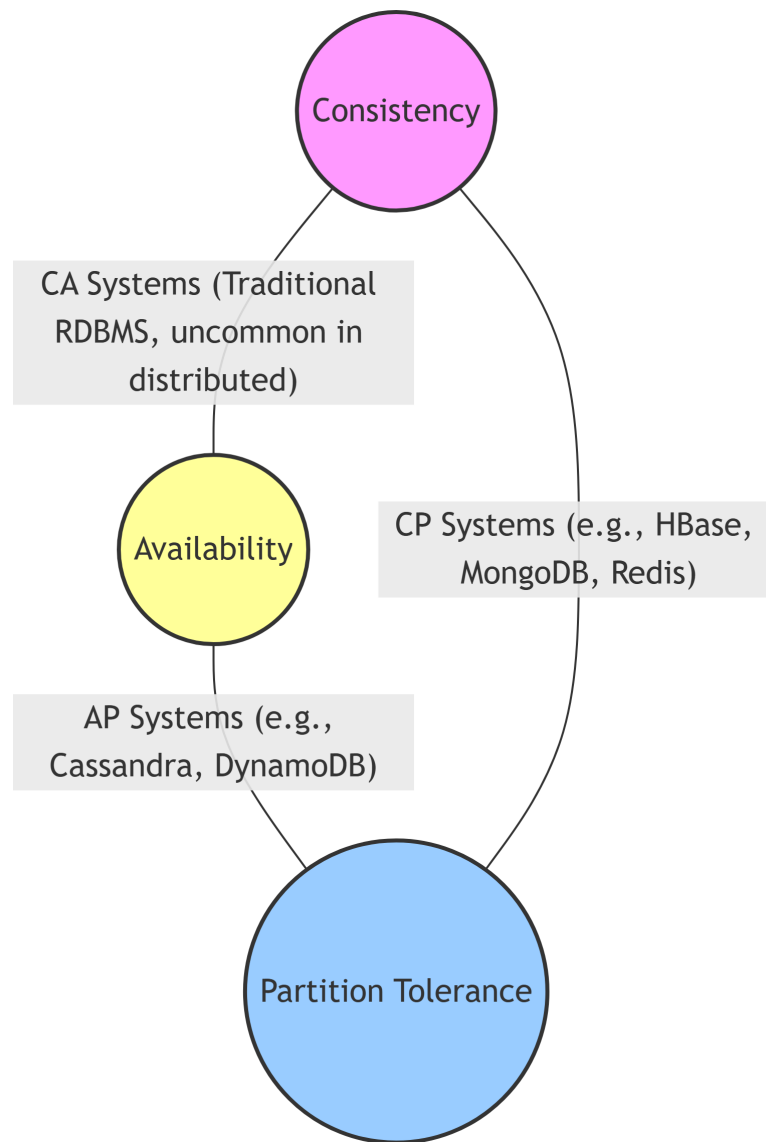


Figure 1.1: The CAP Theorem: Pick Any Two

Type	Properties	Typical Examples	Notes
CA (Consistency + Availability, no Partition Tolerance)	Systems that guarantee consistency and availability as long as no partition occurs.	Traditional RDBMS (like PostgreSQL, MySQL when used on a single node or tightly coupled cluster)	Rare in distributed systems; these systems sacrifice partition tolerance.
CP (Consistency + Partition Tolerance)	Systems that remain consistent under partitions but may sacrifice availability.	HBase, MongoDB (in default configuration), Redis (with replication but not cluster failover), Zookeeper, etcd	Suitable when strong consistency is required (e.g., configuration or metadata services).
AP (Availability + Partition Tolerance)	Systems that stay available even under network partitions but may return stale data.	Cassandra, DynamoDB, Riak, Couchbase	Designed for high availability, eventual consistency, and geographic distribution.

1.11.4 The PACELC Theorem

Formulated by Daniel Abadi, it extends CAP:

- If there is a **P**artition → Trade-off between **A**vailability and **C**onsistency.
- **E**lse (Normal Operation) → Trade-off between **L**atency and **C**onsistency.
- *Examples:*
- **DynamoDB / Cassandra (PA/EL):** Chooses Availability during a partition, and Low Latency over Consistency during normal operations (Eventual Consistency).
- **MongoDB (CP/EC):** Chooses Consistency over Availability during a partition (stops accepting writes if the primary goes down), and Consistency over Latency during normal operations.

PACELC extends CAP by stating: If there is a Partition (**P**), one must choose between Availability (**A**) and Consistency (**C**); Else (**E**), even when the system is running normally, one must choose between Latency (**L**) and Consistency (**C**).

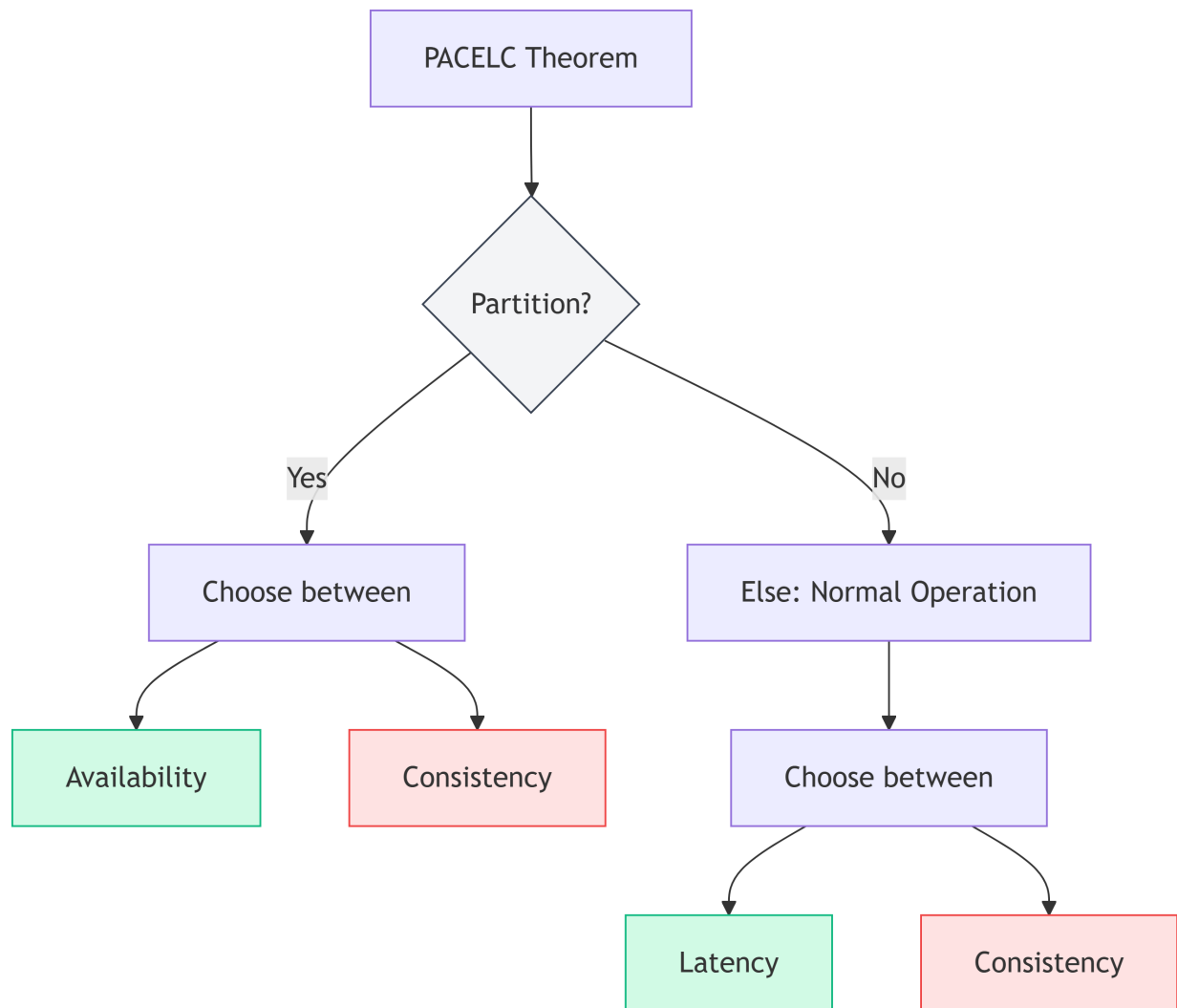


Figure 1.2: PACELC Theorem: Distributed System Trade-offs

Chapter 2

Data Partitioning, Sharding & Consistent Hashing

2.1 Partitioning vs. Sharding

- **Partitioning:** Dividing a dataset into smaller, logical chunks (can reside on the same server).
- **Sharding:** Horizontal partitioning spread across multiple distinct physical servers.

2.1.1 The Routing Problem & Consistent Hashing

Using a naive modulo operator for routing ($ServerIndex = hash(key) \% N$) causes catastrophic data migration (nearly 100% of data moves) when N (number of servers) changes.

Consistent Hashing Mechanics:

1. Servers and data keys are hashed to the same circular key space (a Ring).
2. Data is assigned to the next server found by moving clockwise on the ring.
3. When a node is added or removed, only K/N keys move (where K is total keys), isolating the disruption.
4. **Virtual Nodes:** Assigning multiple points on the ring to a single physical server to prevent data skew and balance the load.

2.2 The Four Families of NoSQL

Concept: “NoSQL” isn’t one thing; it’s a rejection of the “One Size Fits All” philosophy of Relational Database Management Systems (RDBMS). We choose the database based on the *shape* of the data and the *query patterns*.

2.2.1 1. Key-Value Stores (The “Giant Hash Map”)

- **How it works:** Data is stored as a simple key pointing to a blob of value. The database doesn’t care what the value is (it’s opaque).
- **Intricacies:**
- **Performance:** extremely high ($O(1)$ lookups).
- **limitation:** You cannot query by the *value*. You can’t say “Give me all users who live in Mumbai” unless you store “Mumbai” in the key.
- **Industry Standard:** Redis, Amazon DynamoDB, Memcached.

- **Use Case:** Caching user sessions, shopping carts, real-time leaderboards.
- **Analogy:** A **Coat Check** at a theater. You give a ticket (Key), you get your coat (Value). You don't ask the attendant, "Give me all the red coats." You only ask for *your* specific coat.

2.2.2 2. Document Stores (The “JSON Warehouse”)

- **How it works:** Similar to Key-Value, but the value is a structured document (XML, JSON, BSON) that the database *understands*.
- **Intricacies:**
- **Schema Flexibility:** You can add a new field (e.g., “middle_name”) to one document without downtime or updating all other documents.
- **Indexing:** Unlike Key-Value, you *can* index fields inside the document to run complex queries.
- **Industry Standard:** MongoDB, Couchbase.
- **Use Case:** Content management systems (blogs), catalogs (e-commerce products with different attributes), user profiles.
- **Analogy:** A **Manila Folder** in a filing cabinet. One folder might contain a resume, another might contain a photo and a tax form. They are distinct, self-contained units, but you can still search the cabinet for “Everyone with a tax form.”

2.2.3 3. Column-Family Stores (“Wide Column Spreadsheet”)

- **How it works:** Instead of storing data row-by-row (like SQL), it stores data column-by-column.
- **Intricacies:**
- **Write Speed:** Optimized for massive write throughput. It appends data to the end of a log rather than seeking a specific spot on the disk.
- **Compression:** Since a column contains uniform data (e.g., 1,000,000 integers), it compresses extremely well.
- **Industry Standard:** Apache Cassandra, HBase, ScyllaDB.
- **Use Case:** Storing time-series data (IoT sensors), server logs, message history (Discord/Facebook Messenger).
- **Analogy:** A **Phone Book** organized by “First Name” only. If you want to count how many people are named “Rahul,” it's instant because all “Rahuls” are stored physically next to each other on the disk.

2.2.4 4. Graph Databases (The “Relationship Map”)

- **How it works:** Stores “Nodes” (entities) and “Edges” (relationships). The *relationship* is as important as the data itself.
- **Intricacies:**
- **No Joins:** In SQL, joining 5 tables is expensive. In Graph, “hopping” from one node to another is constant time $O(1)$ because the pointer to the neighbor is stored directly on the node.
- **Industry Standard:** Neo4j, Amazon Neptune.
- **Use Case:** Social networks (friends of friends), recommendation engines (“People who bought X also bought Y”), fraud detection rings.

- **Analogy: A Spiderweb.** To find out who knows whom, you just follow the thread. You don't need to check a central registry.

2.3 Partitioning vs. Sharding

Concept: How to handle data that doesn't fit on one machine.

2.3.1 Partitioning (The Logical Split)

Partitioning is the general concept of dividing a dataset into smaller, distinct chunks. This can happen *on a single server* (to improve query speed) or across multiple servers.

- **Vertical Partitioning:** Splitting columns. (e.g., Storing “User Bios” in a slow table and “User Passwords” in a fast table).
- **Horizontal Partitioning:** Splitting rows. (e.g., Rows 1-1000 in Table A, 1001-2000 in Table B).

2.3.2 Sharding (The Physical Split)

Sharding is **Horizontal Partitioning** applied across **multiple distinct servers (nodes)**.

- Every shard holds a unique subset of the data.
- Collectively, all shards make up the complete dataset.
- **The Challenge:** If I need “User A,” which server do I ask? This leads us to the problem of **Routing**.

2.4 Consistent Hashing

The Problem: We need to distribute 1,000,000 keys across N servers.

2.4.1 The Modulo Operator (The Naive Approach)

The classic way to route data is using simple modulo arithmetic:

$$ServerIndex = hash(key) \% N$$

- If you have $N = 3$ servers.
- Key “Apple” hashes to 10. $10 \% 3 = 1$. Save to Server 1.
- Key “Banana” hashes to 14. $14 \% 3 = 2$. Save to Server 2.

The Catastrophe of Resizing: What happens if Server 1 crashes? Now $N = 2$.

- “Apple” (Hash 10): $10 \% 2 = 0$.
- **Result:** The data for “Apple” moved from Server 1 to Server 0.
- **The Disaster:** When N changes, the result of the modulo operation changes for *almost every single key*. This forces a **massive data migration** where nearly 100% of the database has to move to a new server simultaneously. The system crashes under the load.

2.4.2 The Anecdote: The Round Table Dinner

Imagine a wedding dinner with a massive circular table.

- **The Guests (Data Keys):** Guests are seated randomly around the table.
- **The Waiters (Servers):** Waiters stand at specific points around the table.
- **The Rule:** Each waiter serves all the guests seated to their *right*, up until the next waiter.

Scenario A (Modulo Failure): The manager says, “Count off 1, 2, 3. All ‘1s’ go to Waiter A, ‘2s’ to Waiter B...” If Waiter A quits, everyone has to stand up and switch seats to get a new number. Chaos.

Scenario B (Consistent Hashing Success): If Waiter B leaves for a smoke break, Waiter C (who is to the right of B) simply takes over Waiter B’s section *in addition* to their own. The guests don’t move. The other waiters (A, D, E) don’t change anything. The disruption is local, not global.

2.4.3 The Solution: Consistent Hashing

Consistent hashing solves the “reshuffling” problem.

The Ring Topology Imagine a circle (a ring) representing the entire range of hash values (e.g., 0 to $2^{32} - 1$).

- **Place the Servers:** We hash the *IP addresses* of the servers and place them at points on the ring.
- **Place the Data:** We hash the *Keys* and map them to the ring.
- **The Rule:** To find which server stores a Key, go **clockwise** on the ring from the Key’s location until you hit a Server.

[Consistent Hashing Blog Post](#)

2.5 Replication Mechanics

Concept: Never store data in just one place.

1. Leader-Follower Replication (Master-Slave)

- **Leader:** Accepts all Writes. Replicates them to followers.
- **Follower:** Read-only.
- **Sync vs. Async:**
- *Synchronous:* Leader waits for Follower to say “OK” before telling the client “Success.” (Safe, but slow).
- *Asynchronous:* Leader says “Success” immediately, then sends data to Follower in the background. (Fast, but risk of data loss if Leader crashes).

2. Peer-to-Peer Replication (Master-Master)

- Common in Cassandra and DynamoDB.
- Any node can accept a write.
- **Benefit:** Zero downtime. If one node is down, write to another.
- **Complexity: Write Conflicts.** If two users update the same record at the exact same second on different nodes, which one wins?
- *Resolution:* Vector Clocks or Last-Write-Wins (LWW).

3. Replication Factor (RF)

- Standard industry RF is 3. (Three copies of every piece of data).
- Even if one data center burns down, and a server in the second data center crashes, you still have the data.

2.5.1 Sharding Key Design

Choosing a sharding key is a trade-off between write distribution and read efficiency.

- **Date Key:** Causes massive write hotspots (“today’s” server melts down).
- **Geolocation Key:** Causes data skew based on population density.
- **User_ID Key:** Even writes, but causes “Scatter-Gather” inefficiencies during temporal reads, and vulnerabilities to “Whale” users.

The Industry Solution (Composite Keys): Large-scale logging services combine hashing with time buckets: $\text{Hash}(\text{User_ID}) + \text{Time_Bucket}(\text{YYYY-MM})$. This distributes today's writes evenly across the cluster while keeping temporal queries for a specific user localized to a single shard.

2.6 Replication & Consistency

2.6.1 Replication Architectures

Replication ensures High Availability (HA) and read scalability.

- **Leader-Follower (Master-Slave):** Writes strictly hit the Leader; Reads can hit Followers. Introduces replication lag.
- **Peer-to-Peer (Master-Master):** Any node accepts writes. Great for global availability but requires conflict resolution.

2.6.2 Quorum Consensus

In distributed systems, strict consistency is maintained via voting quorums, defined by the formula:

$$R + W > N$$

Where N is total replicas, W is the write quorum, and R is the read quorum. If the sum of read and write nodes exceeds the total nodes, the sets overlap, guaranteeing the reader sees the latest write.

Chapter 3

Storage Technologies & Filesystems

Here is a breakdown of **SAN (Storage Area Network)** vs. **NAS (Network Attached Storage)**, tailored for a System Design context.

In short:

- **NAS** is a **Shared Folder** over the network.
- **SAN** is a **Shared Hard Drive** over the network.

3.1 NAS (Network Attached Storage)

- **Concept:** Think of NAS as a specialized computer whose only job is to serve files to other computers on the network. It handles the filesystem (NTFS, EXT4) itself.
- **Data Type: File-Level Storage.** You ask for “Report.pdf” or “Image.png”.
- **Analogy:** Ordering food at a restaurant. You say “I want Pizza” (File), and the kitchen (NAS) handles the ingredients and cooking. You just get the finished product.

3.2 SAN (Storage Area Network)

- **Concept:** Think of SAN as a dedicated, high-speed network that connects servers to storage devices. To the server’s Operating System, the storage looks like a **local hard drive**, not a network drive. The server manages the filesystem, not the storage device.
- **Data Type: Block-Level Storage.** You ask for “Block 1000 to Block 2000”.
- **Analogy:** Renting a commercial kitchen. You get raw counter space (Blocks). You bring your own ingredients and do your own cooking (Filesystem management).

3.3 Technical Comparison

Feature	NAS (Network Attached Storage)	SAN (Storage Area Network)
Data Unit	Files (Documents, Images)	Blocks (Raw disk sectors)
Primary Protocol	TCP/IP (Ethernet)	Fibre Channel (Optical) or iSCSI
File Protocol	NFS (Linux), SMB/CIFS (Windows)	SCSI Command Set
How OS sees it	A Mapped Network Drive (Z:\ or /mnt/shared)	A Local Disk (/dev/sda or D:\)

Feature	NAS (Network Attached Storage)	SAN (Storage Area Network)
Latency	Higher. The NAS CPU has to process the file request.	Lower. It is close to “metal” speed.
Complexity	Low. Plug into LAN, configure, done.	High. Requires specialized switches/cables.
Cost	Lower. Uses existing Ethernet network.	Higher. Dedicated hardware/fiber optics.

3. When to Use Which (System Design Scenarios)

Scenario A: The “Shared Workspace” (Use NAS)

Use Case: You are designing a system like **Dropbox**, a corporate file server, or a Content Management System (CMS) where multiple web servers need to serve the same user images. **Why NAS? Concurrency:** NAS is designed for multiple users/servers accessing the same *files* simultaneously. **Ease of Management:** You don’t want to manage raw disk blocks; you just want a folder to dump PDFs into.

Cloud Equivalent: AWS EFS (Elastic File System), Google Cloud Filestore.

Scenario B: The “High-Performance Database” (Use SAN)

- **Use Case:** You are designing the backend for a bank or a high-frequency trading platform using **Oracle, PostgreSQL, or Microsoft SQL Server**.
- **Why SAN?**
- **Speed:** Databases need to read/write tiny specific blocks of data instantly. The overhead of a filesystem protocol (NFS) is too slow.
- **Raw Access:** Databases prefer to manage the disk directly for optimization.

Cloud Equivalent: AWS EBS (Elastic Block Store) - *Note: While EBS isn’t strictly a “network” in the traditional fiber sense, it acts as a SAN (Block Storage attached over network).*

Scenario C: Virtualization Cluster (Use SAN)

- **Use Case:** You have 50 physical servers running 1,000 Virtual Machines (VMware/Hyper-V).
- **Why SAN?**
- **Mobility:** If Server A dies, Server B can instantly “mount” the same raw disk blocks and boot up the VM without moving any data.

4. System Design Interview Cheat Sheet

If the interviewer asks:

1. “We need a central place for all our web servers to store user profile pictures.”
 - **Answer: NAS (or Object Storage like S3).** You need file-level sharing.
2. “We are deploying a massive PostgreSQL cluster and need the lowest possible write latency.”
 - **Answer: SAN.** You need block-level access without filesystem overhead.
3. “How do we backup our Exchange Email Server?”
 - **Answer: SAN** is preferred for the live database, but you might backup the *logs* to a **NAS**.

Summary Diagram

- **NAS:** [Clients] –(Ethernet Switch)– [NAS Filer]
- **SAN:** [Servers] –(Fibre Channel Switch)– [Storage Array]

3.3.1 The Hardware Reality & Latency

- **HDD (Spinning Rust):** High seek times. Fast sequential I/O, terrible random I/O.
- **SSD (Flash):** Near-instant access. Random I/O is comparable to sequential I/O.
- **Latency context:** A RAM reference is ~100 ns; an HDD random read is ~10,000,000 ns (10 ms).

3.3.2 Storage Architectures (Block vs. File vs. Object)

- **Block Storage (SAN):** Raw disk blocks mapped to an OS. Lowest latency, used for databases.
- **File Storage (NAS):** Hierarchical directories shared over a network (NFS/SMB). Used for shared workspaces.
- **Object Storage:** Flat structure accessed via REST API. Keys map to Values (Blobs). Immutable data.

3.4 RAID

RAID (Redundant Array of Independent Disks) is a technology that combines multiple physical hard drives into a single logical unit. The goal is to improve performance, provide data redundancy, or both.

In the context of Architecting Scalability, RAID is a fundamental concept for ensuring Data Durability and High Availability at the physical storage layer.

RAID (Redundant Array of Independent Disks) is a technology that combines multiple physical hard drives into a single logical unit. The goal is to improve performance, provide data redundancy, or both.

The Three Core Mechanisms of RAID

To understand how RAID works, you need to understand these three techniques:

Striping: Splitting data across multiple disks. This increases Performance because you can read/write from multiple disks simultaneously (Parallelism).

Mirroring: Duplicating the exact same data on two or more disks. This increases Fault Tolerance; if one disk dies, the data is safe on the other.

Parity: A mathematical calculation (usually XOR) used to rebuild lost data if a drive fails. It provides a middle ground between the speed of striping and the safety of mirroring.

3.4.1 RAID 0 (Striping)

This diagram shows how data blocks (A1, A2, A3...) are split perfectly across two disks for maximum speed, but with no redundancy.

3.4.2 RAID 1 (Mirroring)

This diagram shows identical blocks being written to both disks simultaneously for 1-to-1 redundancy.

3.4.3 RAID 5 (Striping with Distributed Parity)

This diagram requires a minimum of three disks. It illustrates how the parity blocks (P) are staggered across all drives to avoid a single-disk bottleneck during the “Read-Modify-Write” cycle.

3.4.4 RAID 6 (Striping with Dual Distributed Parity)

This requires a minimum of four disks. It is similar to RAID 5, but it generates *two* parity blocks (P and Q) for every stripe, allowing the array to survive two simultaneous drive failures.

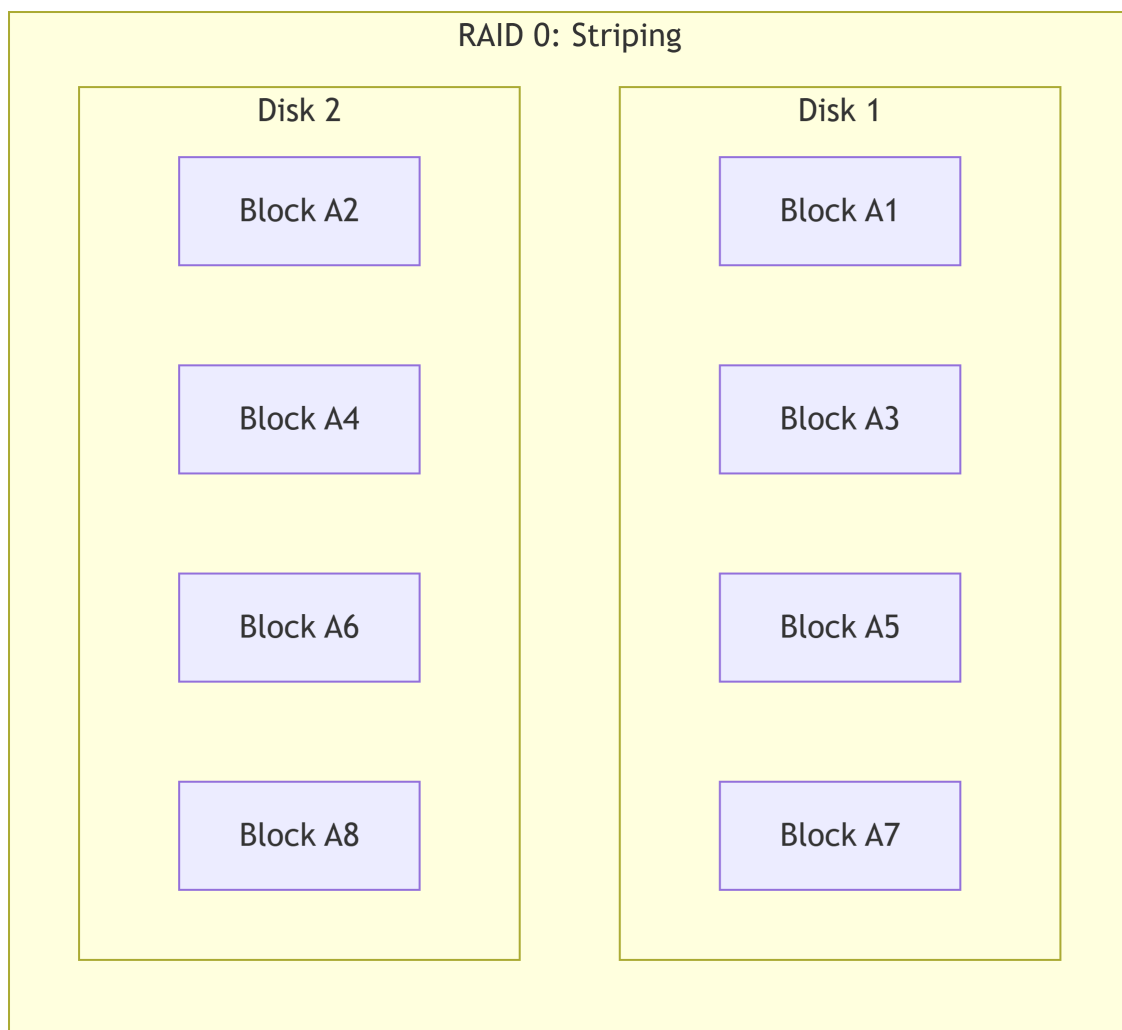


Figure 3.1: RAID 0

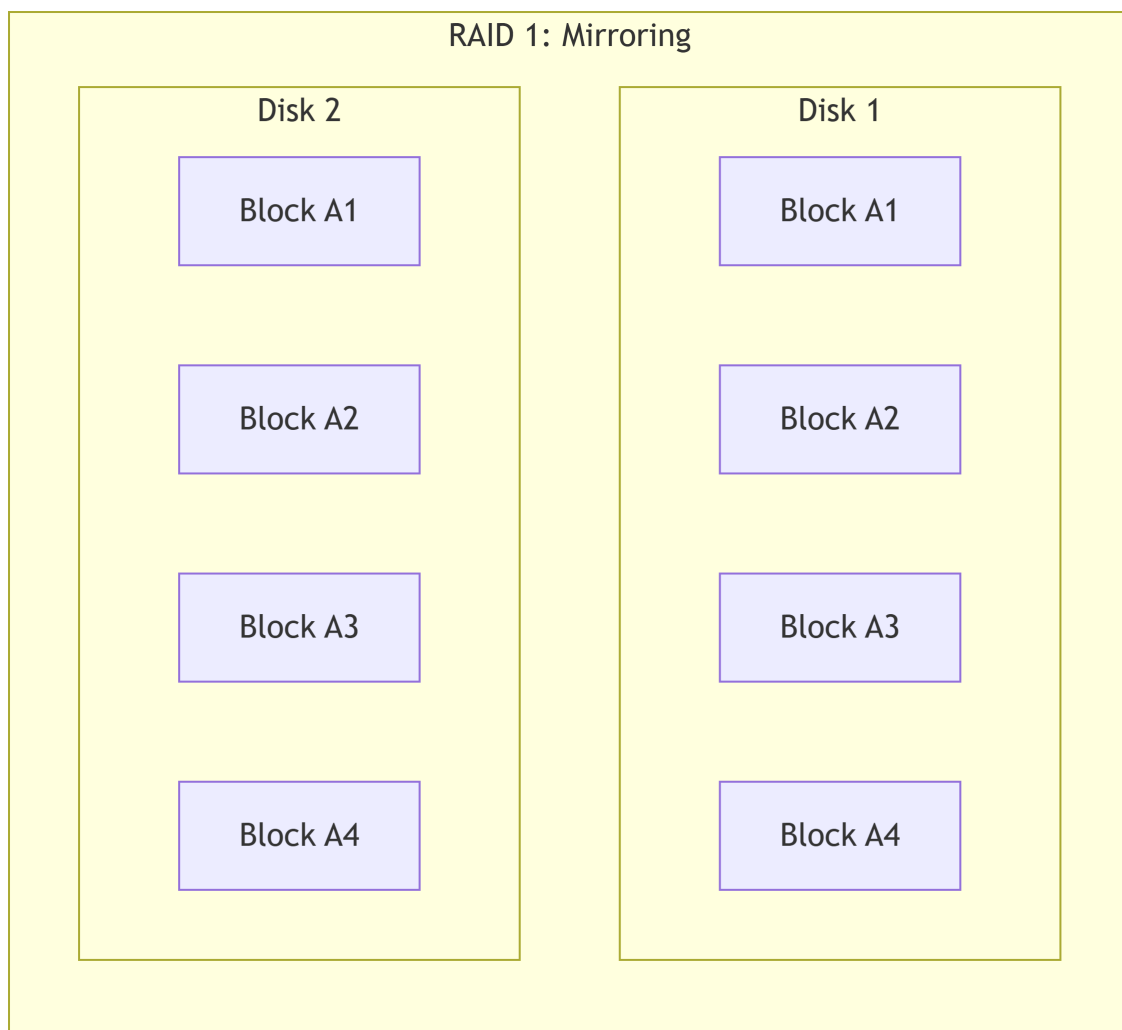


Figure 3.2: RAID 1

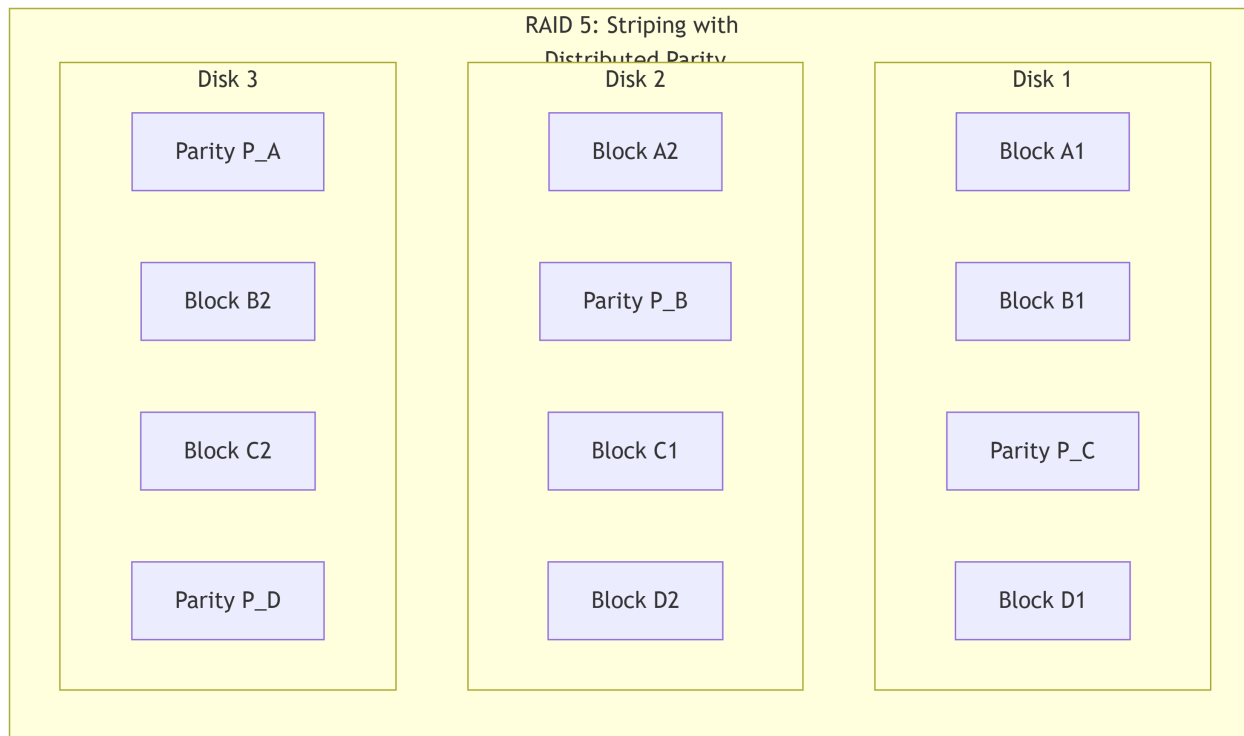


Figure 3.3: RAID 5

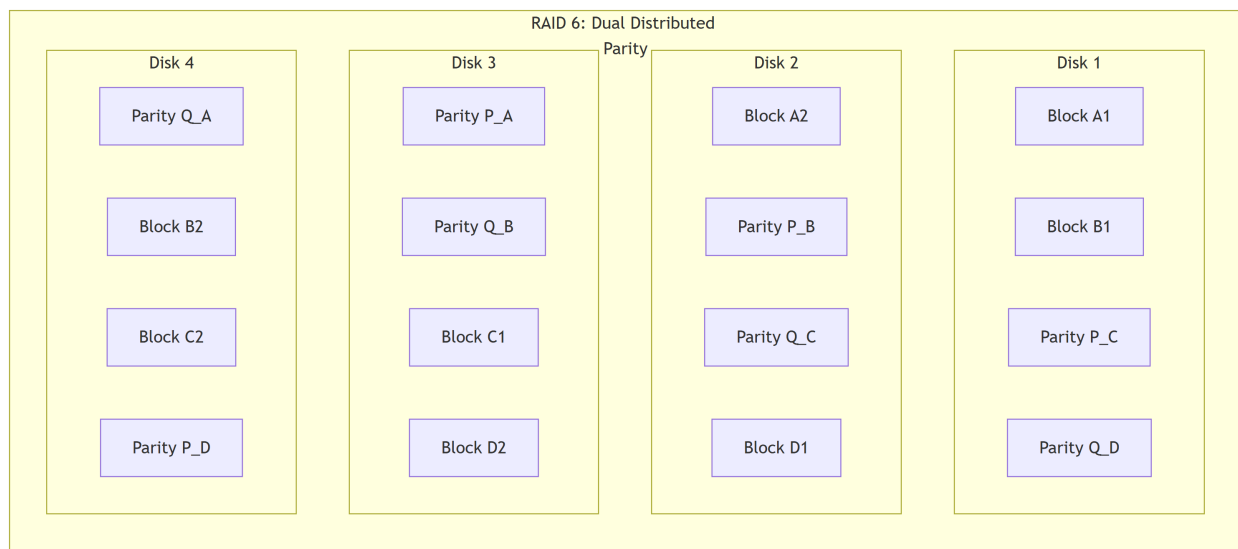


Figure 3.4: RAID 6

3.4.5 RAID 10 (Striping of Mirrors)

RAID 10 is a nested RAID level (RAID 1 + RAID 0) requiring a minimum of four disks. The diagram shows data being striped (A1, A2, A3, A4) across two sub-arrays, while each sub-array internally mirrors the data.

3.5 RAID Estimation Numerical Example

This is a very common interview question style for Site Reliability Engineering (SRE) or Database Architect roles.

Problem Statement: Designing a Database Storage Subsystem

Scenario: You are designing the physical storage layer for a high-traffic transactional database (like PostgreSQL).

- **Target Usable Capacity:** You need to store **10 TB** of data.
- **Workload Profile:** The application generates **3,000 Write IOPS** and **7,000 Read IOPS** (Total 10,000 IOPS).
- **Access Pattern:** Random I/O (4KB block size).

The Hardware Options: You have a shelf of **15k RPM SAS Hard Drives**.

- **Capacity per Disk:** 600 GB
- **Performance per Disk:** 180 IOPS

The Question: Calculate the number of Hard Drives required to meet **BOTH** the Capacity and Performance requirements for:

1. **RAID 10** (Mirroring + Striping)
2. **RAID 5** (Striping with Parity)

Solution Step 1: Understand the RAID Penalties

Before calculating, we must recall the “Write Penalty” of RAID levels.

- **Reads:** No penalty. Reading from one disk is just one operation.
- **Writes (RAID 10):** Penalty = **2**. (Every write must go to Disk A and its Mirror A’).
- **Writes (RAID 5):** Penalty = **4**. (To update one block, the controller must: Read Data, Read Parity, Write New Data, Write New Parity). This is known as the “Read-Modify-Write” cycle.

Solution Step 2: Calculate Back-End IOPS

The “Front-End” IOPS is what the app asks for (10,000). The “Back-End” IOPS is what the disks actually feel.

$$EffectiveIOPS = (ReadIOPS) + (WriteIOPS \times RAIDPenalty)$$

Case A: RAID 10

$$IOPS_{R10} = 7,000 + (3,000 \times 2)$$

$$IOPS_{R10} = 7,000 + 6,000 = \mathbf{13,000 \text{ IOPS}}$$

Case B: RAID 5

$$IOPS_{R5} = 7,000 + (3,000 \times 4)$$

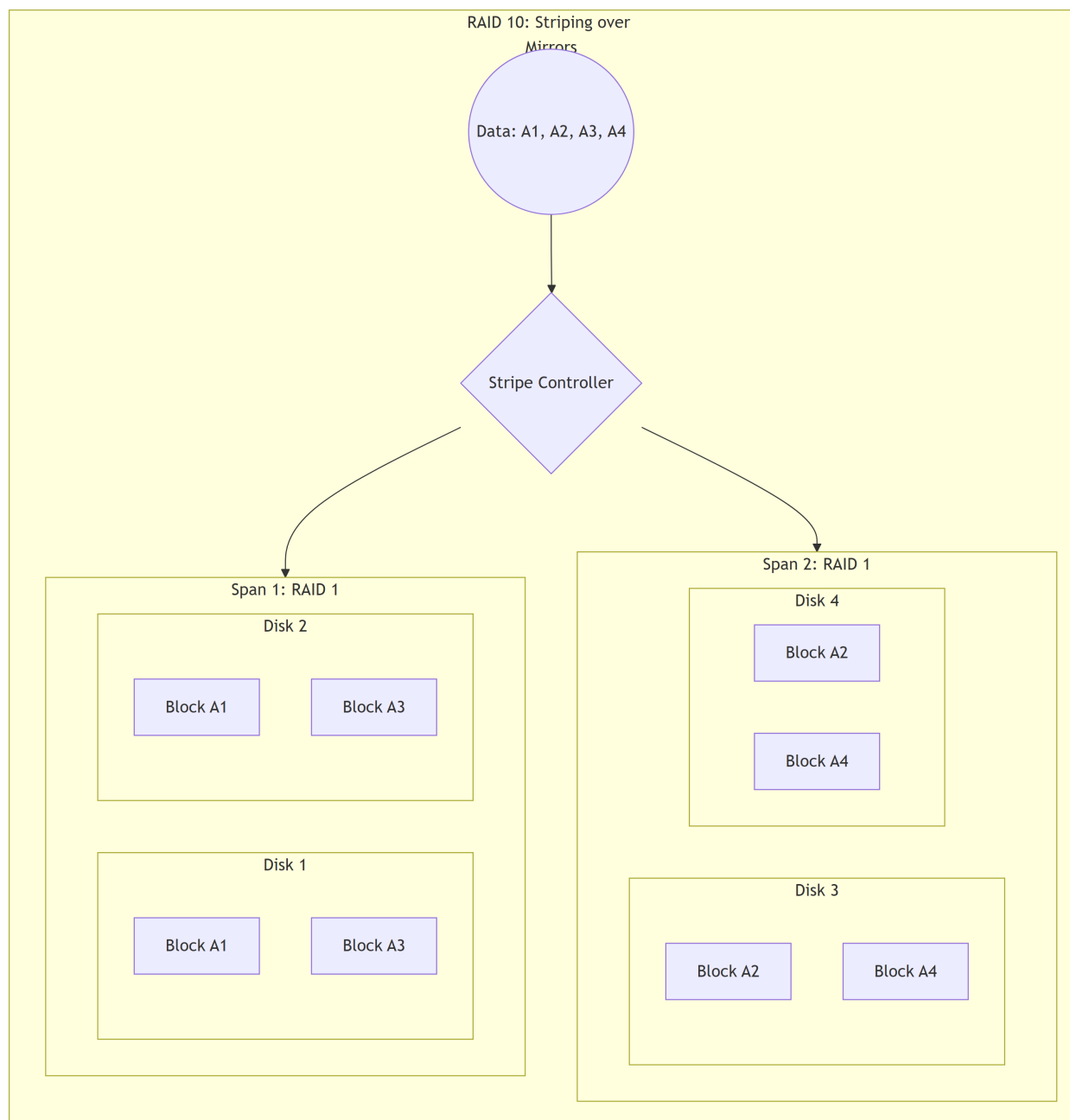


Figure 3.5: RAID 10

$$IOPS_{R5} = 7,000 + 12,000 = \mathbf{19,000} \text{ IOPS}$$

Observation: Notice how RAID 5 requires the disks to work much harder (19k vs 13k) for the exact same application workload.

Solution Step 3: Calculate Disks for Performance

We need enough disks to handle the Back-End IOPS. Each disk provides 180 IOPS.

$$Disks_{perf} = \frac{\text{Total Effective IOPS}}{\text{IOPS per Disk}}$$

Case A: RAID 10

$$Disks_{perf} = \frac{13,000}{180} = 72.22 \rightarrow \mathbf{73} \text{ Disks}$$

Case B: RAID 5

$$Disks_{perf} = \frac{19,000}{180} = 105.5 \rightarrow \mathbf{106} \text{ Disks}$$

Solution Step 4: Calculate Disks for Capacity

Now we check if those disks are enough to hold the **10 TB** of data.

- Target Usable: 10 TB = 10,000 GB.
- Disk Size: 600 GB.

Case A: RAID 10 (50% Overhead)

- You need 2× raw space for mirroring.
- Raw Required: 20,000 GB.
-

$$Disks_{cap} = \frac{20,000}{600} = 33.3 \rightarrow \mathbf{34} \text{ Disks}$$

Case B: RAID 5 (1 Disk Overhead)

- Formula: $N = \frac{Usable}{DiskSize} + 1$
-

$$Disks_{cap} = \frac{10,000}{600} = 16.6 \rightarrow \mathbf{17} \text{ Disks} + 1(\text{Parity}) = \mathbf{18} \text{ Disks}$$

Final Answer & Analysis

In storage design, the answer is always the **Maximum** of the Performance count or the Capacity count.

Feature	RAID 10	RAID 5
Disks needed for Capacity	34	18
Disks needed for Performance	73	106
FINAL DISK COUNT	73 Disks	106 Disks

We intuitively think “RAID 5 saves space, so it’s cheaper.”

- **Reality:** RAID 5 saves *space* (you only need 18 disks for capacity), but it is terrible for *performance* (you need 106 disks to handle the write penalty).
- **Result:** For this write-heavy database, **RAID 10 is actually cheaper** (73 drives vs 106 drives), even though it “wastes” 50% of the space!

Takeaway: This calculation proves why we rarely use RAID 5/6 for write-intensive databases. The “Write Penalty” forces us to buy way more disks than we need just to satisfy the IOPS requirement.

Chapter 4

Parallel Databases and Query Evaluation

Duration: 60 Minutes **Goal:** To understand how to physically wire databases for parallel execution and how the SQL engine splits a single massive query into concurrent sub-tasks.

4.1 The “Why” of Parallelism – Speedup vs. Scaleup

Concept: We don’t parallelize just for fun; we do it for two very specific mathematical metrics.

1. Speedup (Doing the *Same* Job Faster)

- **Definition:** Keeping the data size constant, but increasing the hardware to reduce the execution time.
- **Goal:** Linear Speedup (2x hardware = half the time).
- **Analogy:** You have 100 exam papers to grade. If you grade them alone, it takes 10 hours. If you and a co-professor grade them simultaneously, it takes 5 hours. The job didn’t change; the speed did.

2. Scaleup (Doing a *Bigger* Job in the *Same* Time)

- **Definition:** Increasing the data size *and* the hardware proportionally so that the execution time remains constant.
- **Goal:** Linear Scaleup (2x data + 2x hardware = same time).
- **Analogy:** Next year, your department intake doubles, so you have 200 exam papers. You bring in two more faculty members. It still takes exactly 5 hours to grade everything.

The Bottlenecks (Why we rarely get “Linear” results):

- **Startup Cost:** The time it takes to wake up the parallel threads and assign them work.
- **Interference:** Processes fighting over the same resources (locks, disk arm).
- **Skew:** The workload isn’t divided evenly (one professor gets all the long-essay papers).

4.2 Hardware Architectures for Parallel Databases

Concept: How do the CPUs, Memory (RAM), and Disks actually connect to each other?

1. Shared Memory (The “Giant Server” approach)

- **How it works:** Multiple CPUs all share one massive pool of RAM and one massive Disk array.
- **Pros:** Communication is lightning fast. A CPU just writes to RAM, and another CPU instantly sees it.

- **Cons: The Bus Bottleneck.** If 64 CPUs all try to read the same memory chip at the exact same microsecond, the internal bus gets jammed. It doesn't scale well beyond 32-64 processors.
- **Analogy:** 10 Chefs working on one giant prep table. They can easily hand ingredients to each other, but they keep bumping elbows.

2. Shared Disk (The “Cluster” approach)

- **How it works:** Each CPU has its *own* private RAM, but they all connect to the same central Storage Area Network (SAN).
- **Pros:** Great for fault tolerance. If CPU 1 dies, CPU 2 can just read CPU 1's data right off the shared disk. (Oracle RAC is famous for this).
- **Cons:** The network connection to the disk becomes the bottleneck.
- **Analogy:** 10 Chefs, each with their own prep table (RAM), but they all have to walk to the exact same pantry (Disk) to get ingredients.

3. Shared Nothing (The “Modern Distributed” approach)

- **How it works:** Every node is an independent computer. CPU 1 has its own RAM and its own Disk. CPU 2 has its own RAM and Disk. They only communicate over a high-speed network (like Ethernet).
- **Pros:** Infinite scalability. You want more power? Just plug another server into the network switch.
- **Cons:** Hard to program. Moving data between nodes is expensive.
- **Analogy:** 10 different Food Trucks. Each chef has their own table and their own pantry. To share an ingredient, they have to physically walk outside and hand it to the next truck.

Key Takeaway for the Class: Almost all modern massive parallel databases (Teradata, Snowflake, Redshift) use **Shared Nothing** architectures.

4.3 Parallel Query Evaluation

Concept: How does the Database Management System (DBMS) actually execute SQL queries in parallel over that Shared Nothing architecture?

1. Inter-Query Parallelism (The Easy Way)

- **What it is:** Running *different* queries at the same time.
- **Example:** Student A checks their attendance, while Student B checks their grades. The DBMS routes Student A to Node 1 and Student B to Node 2.
- **Benefit:** Increases the overall *Throughput* (Transactions Per Second) of the system. It scales up concurrent user access.

2. Intra-Query Parallelism (The Hard Way)

- **What it is:** Taking a *single*, massive, complex query and splitting it across multiple nodes to execute simultaneously.
- **Benefit:** Decreases the *Response Time* of a massive data warehousing query.

How Intra-Query Works (The Two Flavors):

- **A. Intra-Operation Parallelism (Splitting the Data):**
 - You want to run a SORT on a 100-million row STUDENTS table.
 - Instead of one node sorting 100 million rows, you split the table. Node A sorts A-M. Node B sorts N-Z.
 - Because the data is partitioned (Range Partitioning), both nodes sort their chunks simultaneously.
- **B. Inter-Operation Parallelism (Pipelining the Execution Plan):**
 - Imagine a query:

```
SELECT * FROM Students JOIN Departments ON Students.dept_ID = Departments.dept_ID WHERE GPA > 9.0 ORDER BY Students.name;
```

- This is a tree of operations: **Scan** -> **Filter** -> **Join** -> **Sort**.
- Instead of doing them sequentially, we use **Pipelining**.
- Node 1 scans the table and filters the **GPA > 9.0**. As soon as it finds a matching row, it *streams* it over the network to Node 2.
- Node 2 is doing the **Join** on the fly as the data arrives. It then streams its output to Node 3.
- Node 3 is buffering the data to run the **Sort**.
- *Analogy*: An assembly line in a car factory. The chassis worker doesn't wait for 100 chassis to be built before passing them to the engine worker. They pass them one by one.

4.3.1 The Parallel Join – A Deep Dive

Concept: Joins are the most expensive operation in relational databases. How do we do them in parallel on a Shared Nothing architecture?

The Partitioned Hash Join:

1. We have Table A (10 million rows) and Table B (50 million rows). We want to join them on **department_id**.
2. **Phase 1 (Partitioning):** Both tables are fed through the *same* hash function on **department_id**. The output routes the rows to different nodes.
 - *Magic result:* All AIML students and the AIML department record are guaranteed to end up on the exact same physical node (e.g., Node 3).
3. **Phase 2 (Local Join):** Now, Node 3 just runs a standard, single-node Hash Join on its local memory.
4. **Phase 3 (Merge):** The results from all nodes are concatenated and sent to the user.

Recap:

- **Speedup vs Scaleup:** Faster vs. Bigger.
- **Architectures:** Shared Memory (Bottlenecks), Shared Disk (Fault Tolerant), Shared Nothing (Scalable).
- **Query Evaluation:** Inter-Query (Many users) vs Intra-Query (One massive query split into pieces).
- **Pipelining:** Passing intermediate results instantly between nodes like an assembly line.

Chapter 5

Performance Caching & Load Balancing

5.1 The “Why” of Caching – Physics and Locality

Kleppmann reminds us that databases are fundamentally bounded by hardware physics. Caching exists because disk I/O is the enemy of scale.

1. The Memory Hierarchy (Recap)

- Going to the network or a hard drive takes milliseconds. Reading from RAM takes nanoseconds. **The Golden Rule of Scaling** : To scale read-heavy systems, move the data closer to the CPU and put it in a faster storage medium.

2. Locality of Reference (The Core Assumption of Caching) Why does caching even work? Because user behavior is predictable.

Temporal Locality: If a piece of data is requested now, it is highly likely to be requested again very soon (e.g., a viral tweet).

Spatial Locality: If a piece of data is requested, data physically stored near it will likely be requested next (e.g., reading a sequential log file).

Analogy: The Library vs. Your Desk Imagine you are writing a research paper.

- **The Database (Disk):** The main college library. It holds every book, but walking there and finding a book takes 15 minutes.
- **The Cache (RAM):** Your physical desk. It is instantly accessible ($O(1)$ reach), but it can only hold 5 books at a time.
- *The Problem:* What do you do when you need a 6th book? You have to put one back. This is **Cache Eviction**.

5.1.1 Cache Eviction

A cache is not a database. It is a transient, finite snapshot.

1. Cache Capacity When you deploy Redis or Memcached, you allocate a hard limit (e.g., 16GB of RAM). When that 16GB fills up, the cache must purge old data to make room for new data.

2. Naive Policies (And why we don’t use them for complex systems)

- **FIFO (First In, First Out):** Kick out the oldest item.

- *Flaw:* What if the “oldest” item is the site’s main CSS file that every user requests? You just evicted your most important asset.
- **Random Replacement:** Roll a dice, kick out a random key.
- *Flaw:* Terrible hit rate. It ignores human behavior entirely.

5.1.1.1 LRU (Least Recently Used) – The Industry Workhorse

Concept: Time-based eviction. The assumption is that if data hasn’t been accessed recently, it has gone “cold.”

1. The Logic Every time a key is accessed (read or written), it is moved to the “front” of the line. When the cache is full, the item at the absolute “back” of the line is evicted.

2. The Classic Interview Question: Implementing LRU in $O(1)$ Time

Alex Xu highlights this as one of the most common system design coding rounds. How do you build a cache where both GET and PUT operations take $O(1)$ time?

You need a hybrid data structure: **A Hash Map + A Doubly Linked List.**

The Hash Map: Gives us $O(1)$ lookup to find the data instantly. The *value* in the map points directly to the node in the Linked List. **The Doubly Linked List:** Gives us $O(1)$ updates. When an item is accessed, we detach its node and snap it to the “Head” (Most Recently Used). The “Tail” is always the Least Recently Used.

Trace Example (Capacity = 3):

1. Insert A → [Head] A [Tail]
2. Insert B → [Head] B - A [Tail]
3. Insert C → [Head] C - B - A [Tail]
4. Read A (A moves to front) → [Head] A - C - B [Tail]
5. Insert D (Capacity full! Evict Tail ‘B’) → [Head] D - A - C [Tail]

The Flaw with LRU:

It is susceptible to “**Cache Pollution.**” Imagine a daily cron job that scans the entire database once a night to generate a report. It will pull thousands of useless records into the cache, pushing out all the actual, useful user data.

5.1.1.2 LFU (Least Frequently Used) – The Heavy Hitter

Concept: Frequency-based eviction. It tracks *how many times* an item is requested. It assumes that data accessed 10,000 times historically is more valuable than data accessed once 5 seconds ago.

1. The Logic Every item has a counter. When the cache is full, the item with the lowest counter is evicted. (If there is a tie, we usually fall back to LRU among the tied items).

2. Implementation Complexity LFU is much harder to implement in $O(1)$ time than LRU. It typically requires two Hash Maps (one for Key->Value, one for Frequency->DoublyLinkedList) to maintain $O(1)$ time complexity.

3. The Flaws with LFU (Why it’s rarely used in its pure form)

- **The “Viral Baggage” Problem:** Imagine a YouTube video goes viral in 2024 and gets 50 million views. It builds up a massive frequency counter. By 2026, nobody watches it anymore. However, because its historical counter is so high, LFU will *never* evict it, wasting precious RAM forever.
- **Solution:** Modern implementations use “Dynamic Aging” or decay mechanisms (e.g., dividing all counters by 2 every few hours) to let old viral content naturally fade out.

5.1.1.3 Real-World Architecture & Summary

1. What does the industry actually use? Very few modern systems use pure LRU or pure LFU. They use hybrids.

W-TinyLFU (Used by Caffeine Cache in Java): It maintains a tiny “admissions” window using LFU to prevent one-off database scans from polluting the cache, but uses an LRU-like structure for the main storage to let old data age out.

2. Caching Pitfalls When you put a cache in front of a database, you solve one problem but create three new ones.

- **Cache Penetration:** Hackers querying random, non-existent keys to bypass the cache and hammer your database.
- **Cache Stampede (Thundering Herd):** A highly popular cached item (like a live cricket score) expires. Suddenly, 50,000 concurrent users miss the cache and hit your database at the exact same millisecond.

Summary:

- **LRU:** Evicts based on *Time*. Easy to build, fast, but vulnerable to sequential scans.
- **LFU:** Evicts based on *Popularity*. Harder to build, tracks historical value, but vulnerable to “old baggage.”

5.1.2 Caching Strategies

5.1.2.1 The Read Strategies (How data gets INTO the cache)

When an application needs data, it has two ways to interact with the cache and the database.

5.1.2.2 Cache-Aside (Lazy Loading)

This is the most common pattern. The cache sits *adjacent* to the database. The application is the manager and communicates with both.

- **The Flow:**
 1. Application asks the Cache for data.
 2. **Hit:** Data is returned.
 3. **Miss:** Application asks the Database for data.
 4. Application writes the data to the Cache for next time, then returns it.
- **Pros:** Resilient. If the cache goes down, the application can still talk directly to the database. It is also memory-efficient because data is only cached if it is actually requested (“lazily” loaded).
- **Cons:** The application code is more complex because it has to handle the logic of checking the cache, querying the DB, and populating the cache. The first user to request data pays a latency penalty.
- **Best Used For:** Read-heavy, general-purpose web applications (e.g., user profiles, news articles).

5.1.2.3 Read-Through

The cache sits *in-line* between the application and the database. The application treats the cache as the main data store and never talks to the database directly.

- **The Flow:**
 1. Application asks the Cache for data.
 2. **Hit:** Data is returned.
 3. **Miss:** The *Cache Provider/Library* automatically fetches the data from the Database, updates itself, and returns the data to the application.

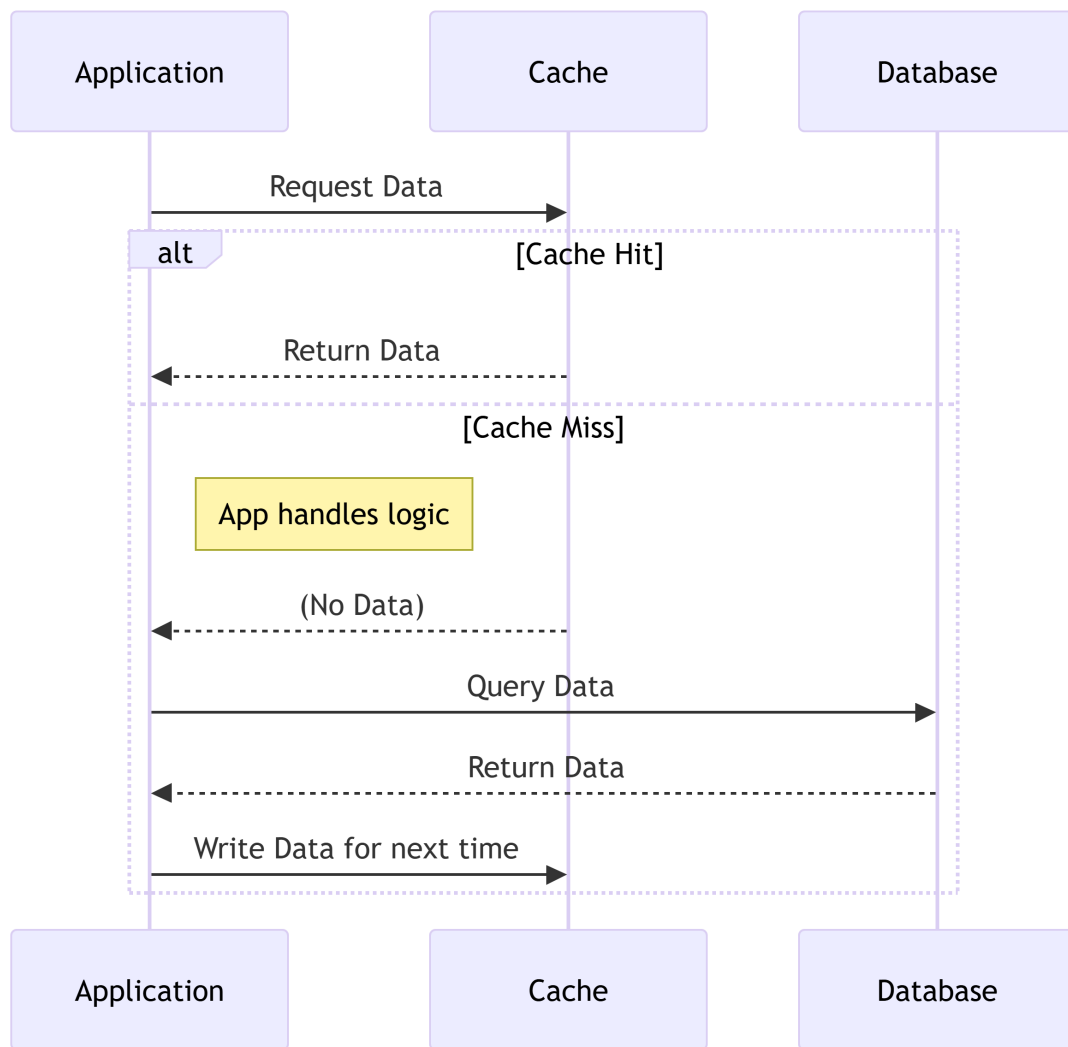


Figure 5.1: Cache-Aside Strategy

- **Pros:** Application code is incredibly clean and simple. The caching logic is completely abstracted away into the infrastructure layer.
- **Cons:** You are tightly coupled to a cache provider that supports this feature. If the cache layer goes down, the application might not know how to talk to the DB directly.
- **Best Used For:** Read-heavy workloads where you want strict separation of concerns between application logic and data fetching.

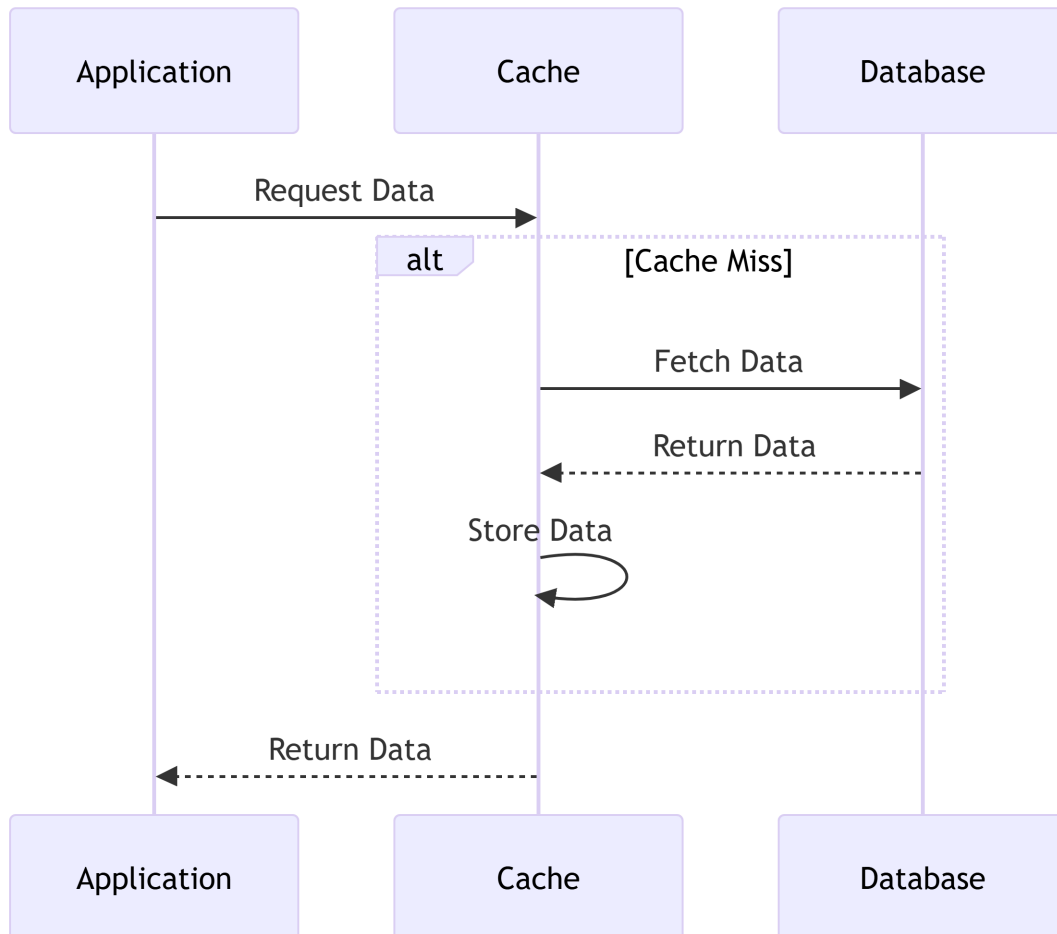


Figure 5.2: Read-Through Strategy

5.1.3 The Write Strategies (How data stays SYNCED)

When data is created or updated, how do we ensure the cache doesn't serve stale, outdated information?

5.1.3.1 Write-Through

Data is written to the cache and the database at the exact same time.

The Flow:

1. Application writes data to the Cache.
2. The Cache (or application) synchronously writes the data to the Database.
3. Return "Success" only when both writes are confirmed.

- **Pros: Absolute Consistency.** You are guaranteed that the cache and the DB are perfectly matched. Excellent for pairing with Read-Through.
- **Cons: High Write Latency.** Every write operation pays a double time penalty (Cache Write + DB Write). It also wastes memory by caching data that might never be read.
- **Best Used For:** Systems where reading stale data is unacceptable (e.g., banking dashboards, inventory control, authorization roles).

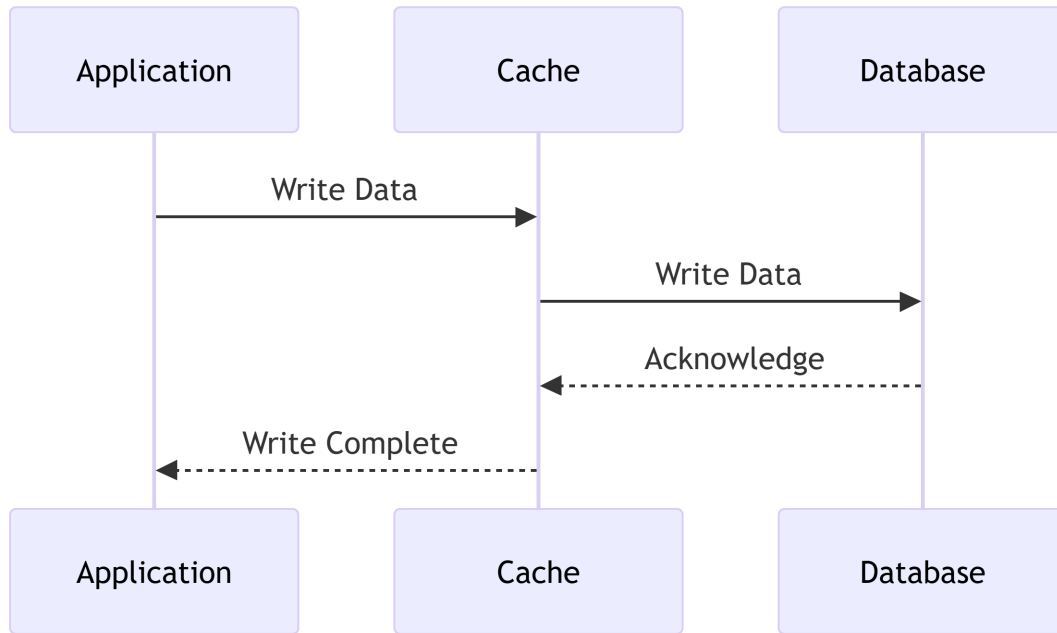


Figure 5.3: Write-Through Strategy

5.1.3.2 Write-Behind (Write-Back)

We prioritize pure write speed over data safety. The cache acts as a fast buffer.

- **The Flow:**

1. Application writes data to the Cache.
2. Cache immediately returns “Success” to the user.
3. An asynchronous background process periodically (e.g., every 5 seconds) takes all the new writes and flushes them to the Database in a bulk batch.

- **Pros:** Blazing fast user experience. It shields the database from massive traffic spikes because 1,000 rapid updates to the same row can be merged into a single DB write.
- **Cons: High Risk of Data Loss.** If the cache server loses power before the background sync happens, the data is gone forever.
- **Best Used For:** Write-heavy workloads where slight data loss is acceptable, and speed is paramount (e.g., YouTube view counters, real-time gaming leaderboards, IoT sensor telemetry).

5.1.3.3 Write-Around

Data is written directly to the database, completely bypassing the cache.

- **The Flow:**

1. Application writes data directly to the Database.
2. The cache is left alone.

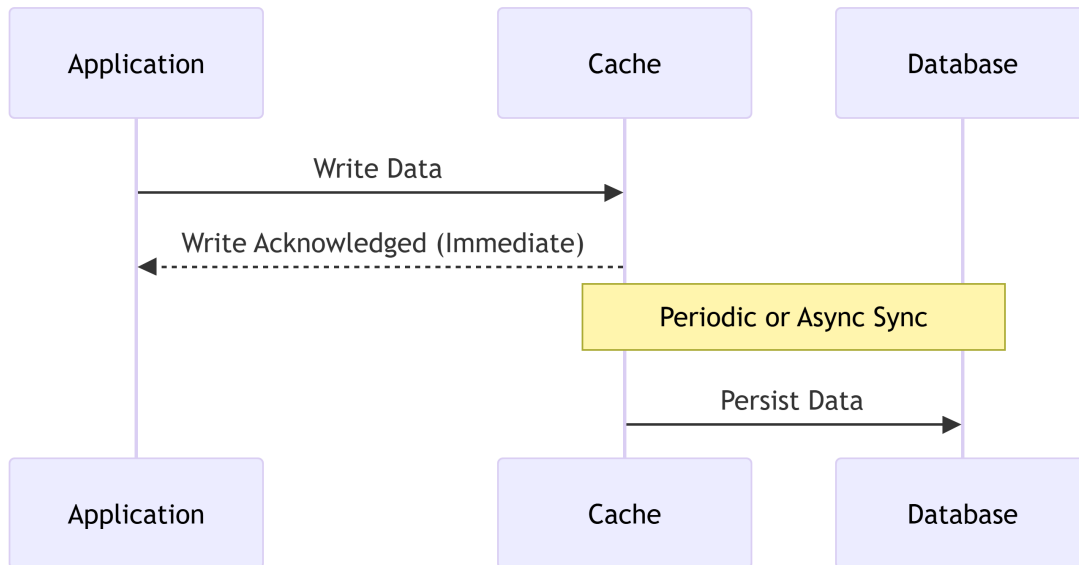


Figure 5.4: Write-Behind Strategy

3. When the data is eventually read, a Read Strategy (like Cache-Aside) will pull it into the cache.

- **Pros:** Prevents “Cache Pollution.” If you are writing a massive amount of data that will likely never be read (or won’t be read for a long time), you avoid flushing out useful data from your expensive RAM.
- **Cons:** A “Read Penalty.” If a user writes a post and then immediately refreshes the page to view it, they will experience a Cache Miss because the data wasn’t proactively cached.
- **Best Used For:** Data written once and read rarely, or massive batch imports (e.g., archiving logs, uploading large server backups, generating end-of-month reports).

Pattern	Primary Action	Biggest Advantage	Biggest Weakness	Classic Use Case
Cache-Aside	Read	Resilient to cache failure	App code is complex	General Web Apps
Read-Through	Read	Clean app code	Tightly coupled to cache	Read-heavy, simple apps
Write-Through	Write	Perfect data consistency	Slow write latency	Financial/Banking
Write-Behind	Write	Blazing fast writes	Risk of permanent data loss	View Counters / Analytics
Write-Around	Write	Prevents cache pollution	Read penalty after writing	Archival Data / Logs

5.1.3.4 Distributed Caching in Redis Clusters

The Breaking Point (Why we distribute)

Concept: A single cache server (like one Redis instance) is incredible. It can handle over 100,000 reads per second. But it has two fatal physical limits:

1. **The RAM Ceiling:** You can only buy so much RAM for one motherboard (usually capping out practically around a few hundred Gigabytes). What if Netflix needs 50 Terabytes of cache?

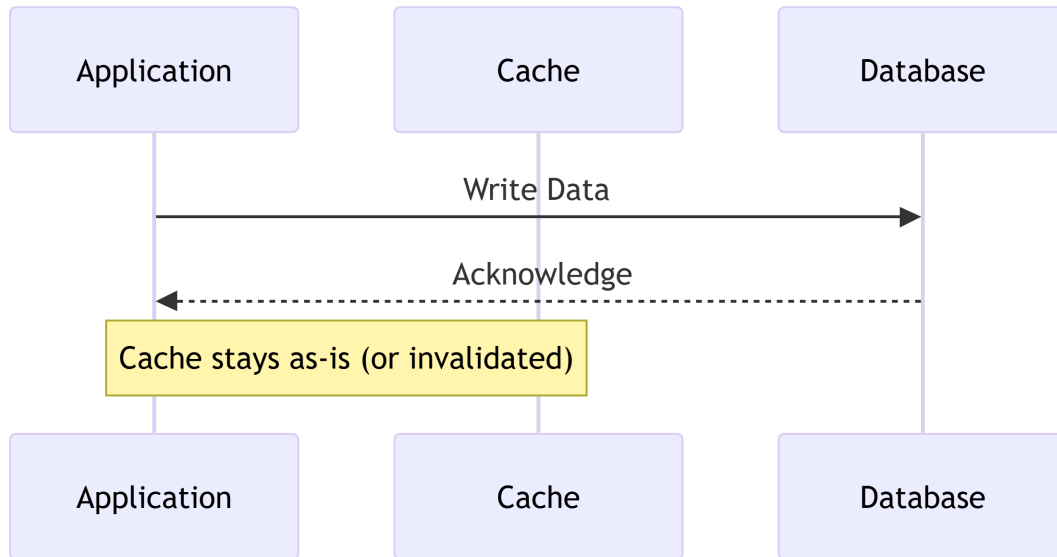


Figure 5.5: Write-Around Strategy

2. **The Single Point of Failure (SPOF):** If the motherboard burns out, your 100% cache hit rate instantly drops to 0%, shifting all 100,000 requests per second directly to your fragile relational database. The database will crash immediately.

The Solution: Distributed Caching. We take 10, 50, or 100 smaller, cheaper servers and stitch them together over the network to act as one giant, logical cache.

The Anecdote: The Librarian’s Desk * Single Cache: The librarian has a small desk. They can keep the 10 most popular books right on the desk for instant access. But when the desk gets full, they are stuck. If the desk collapses, the books are gone. *** Distributed Cache:** You hire 50 librarians, give them each a desk, and put them in a massive circle. Now you can store 500 books instantly. But now you have a new routing problem: *When a student asks for a specific book, which of the 50 librarians has it?*

How Redis Cluster Actually Works

While Memcached relies heavily on classic Consistent Hashing on the client side, **Redis Cluster** uses a slightly different, incredibly elegant approach called **Hash Slots**.

The Mechanics of Hash Slots:

1. Redis Cluster divides the entire universe of possible cached data into exactly **16,384 Hash Slots**.
2. If you have 3 Master servers in your cluster:
 - Node A holds slots 0 to 5,500.
 - Node B holds slots 5,501 to 11,000.
 - Node C holds slots 11,001 to 16,383.
3. When the application wants to read or write a key (e.g., `User_Profile_99`), Redis runs a lightweight algorithm: `CRC16("User_Profile_99") % 16384`.
4. If the math outputs 5,000, the system instantly knows to talk to Node A.

The Gossip Protocol:

How do the nodes know if Node B dies? They use a “Gossip Protocol.” Every node constantly pings a few random nodes every second, whispering, “*Hey, I’m alive, is Node B alive?*” If enough nodes agree that Node B isn’t answering, they automatically promote Node B’s replica to take over.

5.1.3.5 The Nightmare of Cache Invalidation

Concept: Now that our cache is distributed across 50 servers, how do we delete (invalidate) data when it changes in the main database?

The Famous Quote:

“There are only two hard things in Computer Science: cache invalidation and naming things.” —
Phil Karlton

The Anecdote: The Ghost PlayStation 6 Imagine an e-commerce giant on Black Friday. They have exactly *one* PlayStation 6 left in the database.

- The UI is incredibly fast because it is reading the inventory count from the **Distributed Cache**, which says: `PS6_Inventory: 1`.
- **User A** buys the console. The application writes the new inventory (0) to the main Database.
- **The Failure:** The application sends a network request to the Cache Cluster to **Invalidate** (delete) the `PS6_Inventory` key. But there is a microsecond network blip. The invalidation request drops.
- **The Ghost:** The database knows there are 0 consoles. But the cache still says there is 1. **User B** logs on, sees “In Stock!”, clicks buy, and their credit card is charged for an item that does not exist. Customer service chaos ensues.

The Core Lesson: In distributed systems, networks fail. If a cache invalidation fails, your application will confidently serve lies to your users.

5.1.3.6 Failsafes and The Thundering Herd

How do the big tech companies survive failed invalidations?

1. The Ultimate Failsafe: TTL (Time-To-Live)

You should *never* put data into a cache without a TTL. A TTL is a self-destruct timer. * Even if your application relies on explicit Cache Invalidation (deleting the key when a user updates their profile), you *also* put a 1-hour TTL on that key. * *Why?* Because if the explicit invalidation network packet drops (like the PlayStation example), the absolute worst-case scenario is that the cache serves stale data for 59 more minutes before the key automatically evaporates.

2. The Thundering Herd (Cache Stampede)

What happens when a massively popular key hits its TTL and evaporates?

- **The Setup:** A live cricket match score is cached. It has a TTL of 5 seconds. 50,000 users are refreshing the app every second.
- **The Disaster:** At exactly second 6, the cache key evaporates. At that exact millisecond, 50,000 simultaneous requests hit the cache. They *all* get a Cache Miss.
- **The Stampede:** Because of the Cache-Aside pattern, all 50,000 application threads say, *“Oh, I missed the cache! I better go ask the Database!”* 50,000 massive queries hit the PostgreSQL database at the exact same millisecond. The database instantly crashes.

The Solution: Distributed Mutex Locks

When the key expires, the first thread that gets a Cache Miss says, *“I will go to the database to get the new score.”* It immediately places a **Lock** on that cache key.

When the other 49,999 requests arrive a microsecond later, they see the Lock. Instead of rushing the database, they are forced to wait for 50 milliseconds. Once the first thread comes back and populates the cache, the lock is removed, and the 49,999 waiting threads safely read from the freshly updated cache.

Recap: * **Scaling Out:** We use Redis Clusters and Hash Slots to spread memory across dozens of physical machines. * **Invalidation Risk:** Deleting data over a network is prone to packet loss, leading to stale data

(The Ghost Inventory). * **Defense in Depth:** Always use a TTL as a backup to your explicit invalidation logic. * **The Stampede:** High-traffic cache misses will kill your DB. Use locking mechanisms to ensure only *one* thread does the heavy lifting to repopulate the cache.

5.2 Load Balancing

Goal: To understand how a Load Balancer (LB) acts as the gateway to a distributed system, the fundamental differences between routing at the Transport layer vs. the Application layer, and the mathematical algorithms used to distribute the load.

Need for a Load Balancer

In Module 1, we learned about Horizontal Scaling (adding more servers). But if you have 10 web servers, which IP address does the user's browser actually connect to? DNS can only do so much.

- **The Problem:** If `google.com` simply returned the IP address of Server #1, that server would melt under the global traffic, while Servers #2 through #10 sit idle at 0% CPU.
- **The Solution:** The Load Balancer. It is the single public-facing IP address. It sits in front of the server cluster, accepts all incoming traffic, and acts as a "Traffic Cop," deciding exactly which backend server should handle which request.
- **Key Benefits:**
 1. **Scalability:** Add or remove backend servers without the user ever knowing.
 2. **High Availability:** If Server #3 catches fire, the LB instantly stops sending traffic to it (via Health Checks).

5.2.1 Layer 4 Load Balancing

To understand how an LB makes routing decisions, we must reference the OSI (Open Systems Interconnection) model. Layer 4 is the **Transport Layer** (TCP/UDP).

How Layer 4 Works: At Layer 4, the Load Balancer has no idea what the actual data is. It does not look at the HTTP payload. It only looks at the envelope: * **Source IP & Port** (The user) * **Destination IP & Port** (The load balancer)

The LB simply grabs the TCP packet, changes the Destination IP to one of the backend servers via Network Address Translation (NAT), and blindly forwards it.

The Anecdote: The Mailroom Sorter Imagine a mailroom worker at a giant corporation. They look at a sealed envelope. It says "To: 123 Main St, Floor 4." The worker does not open the envelope. They do not care if it is a wedding invitation or a tax bill. They just toss it onto the Floor 4 delivery cart.

The Trade-offs: * **Pros:** Blazing fast. Because it doesn't spend CPU cycles decrypting SSL or reading HTTP headers, it can handle millions of concurrent connections with extremely low latency. * **Cons:** "Dumb" routing. It cannot route traffic based on what the user is actually asking for.

Applications

These are used when you need raw, blistering speed, ultra-low latency, and millions of concurrent connections, usually for non-HTTP traffic (like databases or multiplayer games).

- **AWS Network Load Balancer (NLB):** This is Amazon's managed Layer 4 service. It operates at the connection level (TCP/UDP) and is capable of handling tens of millions of requests per second while maintaining latencies in the single-digit milliseconds. It is heavily used for gaming servers and IoT telemetry data.
- **HAProxy (in TCP Mode):** HAProxy is a legendary, battle-tested open-source load balancer. When configured in TCP mode, it is the industry standard for database load balancing. For example, you

place it in front of a PostgreSQL cluster, and it blindly routes raw SQL TCP packets to the healthiest database node.

- **IPVS (IP Virtual Server):** This is a Layer 4 load balancer built directly into the Linux kernel itself. When massive companies like GitHub or Google need to build their own custom, bare-metal network balancers, they often use IPVS under the hood because it operates at the lowest, fastest level of the operating system.

5.2.2 Layer 7 Load Balancing

Layer 7 is the **Application Layer** (HTTP/HTTPS, WebSockets).

How Layer 7 Works: At Layer 7, the Load Balancer actually *stops* the connection, decrypts the SSL certificate (SSL Termination), opens the “envelope,” and reads the contents. It looks at: * The URL Path (e.g., /video vs /api) * HTTP Headers * Cookies (e.g., `SessionID=XYZ`)

The Anecdote: The Hotel Concierge Instead of a mailroom sorter, Layer 7 is a luxury hotel concierge. When you walk in, they stop you and ask, “*How can I help you?*” * If you say, “*I want to swim,*” they send you to the pool (Server A). * If you say, “*I want to eat,*” they send you to the restaurant (Server B). * They make a **smart routing decision** based on the *content* of your request.

The Microservice Superpower: Layer 7 is the foundation of modern Microservices. * Traffic to `www.amazon.com/checkout` goes to the specialized Payment Cluster. * Traffic to `www.amazon.com/images` goes to the lightweight CDN Cluster.

The Trade-offs: * **Pros:** Extremely smart routing. Allows for microservice architectures, caching at the LB level, and sticky sessions (routing the same user to the same server based on a cookie). * **Cons:** Slower. Decrypting SSL, reading HTTP payloads, and re-encrypting the data requires significantly more CPU power than Layer 4.

Applications

These are used when you need to inspect the traffic, route based on URLs, manage microservices, or terminate SSL certificates.

- **AWS Application Load Balancer (ALB):** The standard choice for cloud web applications. If a user goes to `example.com/api`, the ALB reads the /api path and routes it to your Node.js backend. If they go to `example.com/video`, it routes it to your media encoding servers.
- **Envoy Proxy:** This is the modern darling of the cloud-native world. Originally built by Lyft, Envoy is designed specifically for complex microservice architectures (like Kubernetes). It natively understands advanced Layer 7 protocols like HTTP/2 and gRPC, and it forms the backbone of “Service Meshes” like Istio.
- **NGINX:** Originally designed as a web server, NGINX is arguably the most famous open-source Layer 7 load balancer. It acts as a reverse proxy, sitting in front of your application, terminating the HTTPS connection, reading the headers, and smartly distributing the traffic.
- **Traefik:** A highly popular, modern Layer 7 edge router designed specifically for Docker and Kubernetes. It dynamically updates its routing rules in real-time by automatically looking at the tags on your deployed containers.

Feature	Layer 4 (Transport)	Layer 7 (Application)
Data Inspected	IP Address, TCP/UDP Port	URL, HTTP Headers, Cookies
Performance	Extremely Fast (Low CPU)	Slower (High CPU for SSL/Parsing)
Routing Intelligence	“Dumb” (Network level)	“Smart” (Microservices mapping)
Primary Use Case	Massive raw traffic (e.g., Gaming servers, DB load balancing)	Web Applications, APIs, Microservices

Summary

- **Layer 4** is your shield. It sits at the edge of your network, using dynamic connection counting to absorb the raw brunt of the internet.
- **Layer 7** is your brain. It sits safely inside your network, using dynamic response times to perfectly orchestrate traffic to your microservices.

5.2.3 Static Load Balancing Algorithms

Now that we know our Load Balancer acts as a Reverse Proxy, how does it decide which backend server gets the request? We start with the “blind” static algorithms.

5.2.3.1 Round Robin

- **The Logic:** The simplest algorithm in existence. It distributes requests sequentially. Server A, then Server B, then Server C, and back to Server A. Like dealing cards around a poker table.
- **The Pros:** Zero computational overhead. The load balancer doesn't need to think or calculate; it just passes the packet.
- **The Fatal Flaw:** It assumes every request takes the exact same amount of time to process.
 - *Example:* Request 1 is a simple text read (takes 10ms). Request 2 is a massive 4K video upload (takes 30 seconds). Round Robin blindly gives Request 1 to Server A, Request 2 to Server B, and keeps looping. Server B will eventually choke and crash, while Server A is sitting idle.

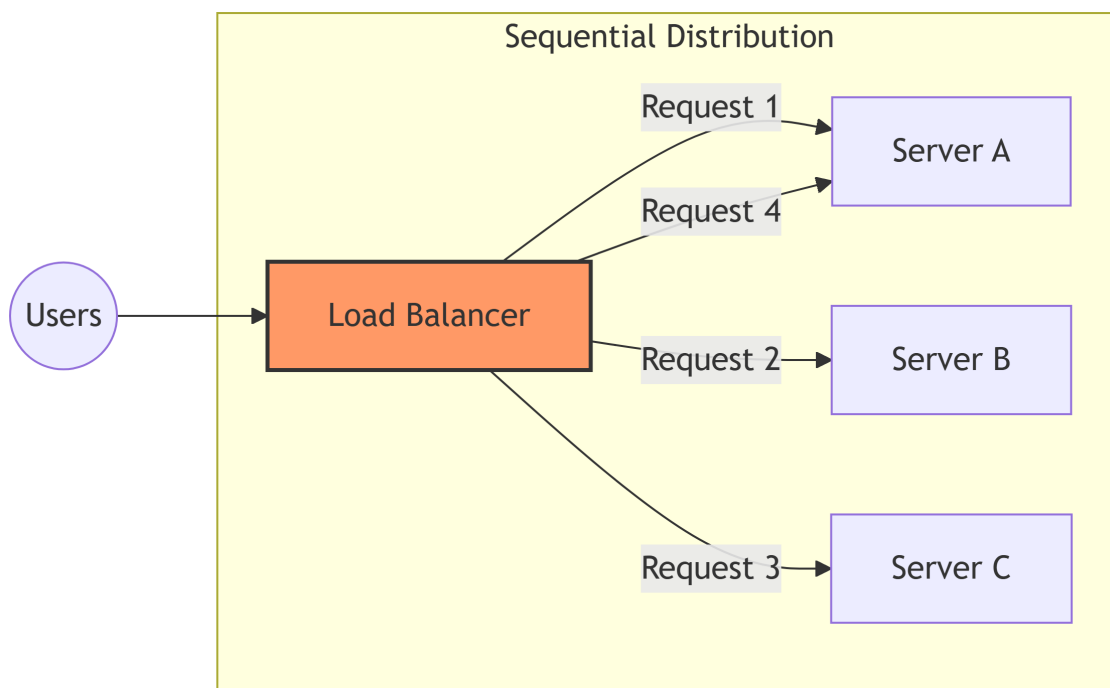


Figure 5.6: Round Robin Algorithm

5.2.3.2 Weighted Round Robin

Logic: In the real world, you rarely have identical servers. You might have one brand-new 64-core beast, and two older 16-core legacy machines. You assign a “Weight” to represent their capacity.

The Math: * Server A (Weight 4) * Server B (Weight 1) * Server C (Weight 1) * The routing sequence becomes: A, A, A, A, B, C. Server A receives 4 requests for every 1 request the others receive.

The Pros: Prevents you from melting older hardware while waiting for new hardware to be fully utilized.

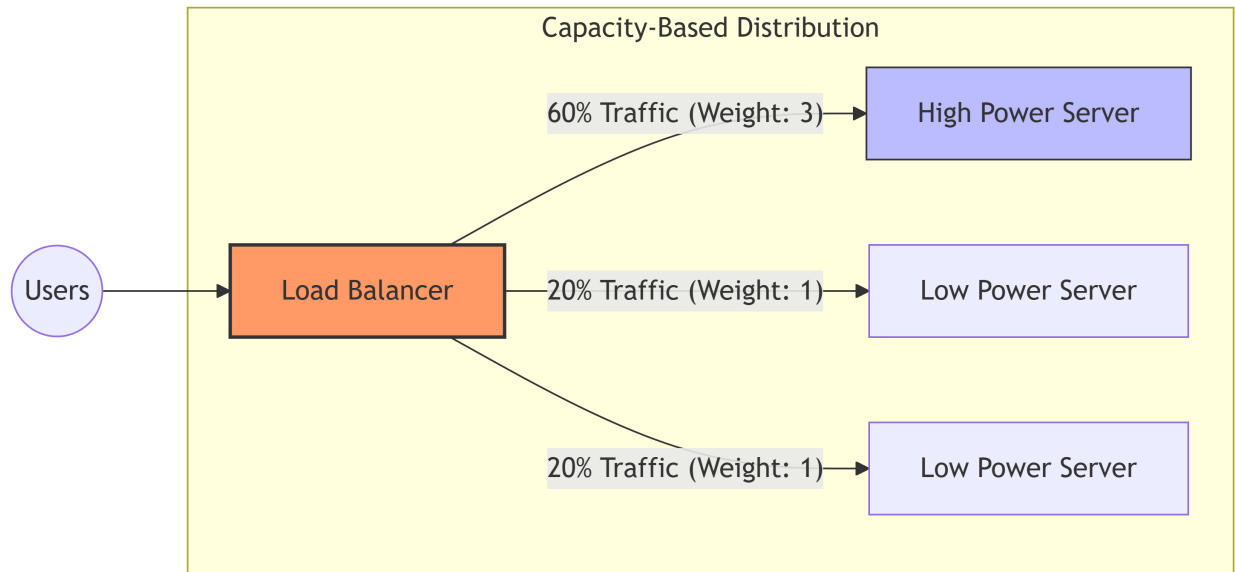


Figure 5.7: Weighted Round Robin Algorithm

5.2.4 Dynamic Load Balancing Algorithms

Concept: To fix the flaws of static routing, the Reverse Proxy must become “observant.” It must track the real-time health and load of the backend servers.

5.2.4.1 Least Connections

Logic: The load balancer keeps a live counter of open TCP connections for every server. When a new request arrives, it routes it to the server with the absolute lowest number of active connections.

The Anecdote (The Supermarket): When you walk up to the checkout registers, you don’t just blindly go to Register 1 because it’s “its turn” (Round Robin). You look to see who has the shortest line. If Register 1 has 5 people with full carts, and Register 3 has 1 person, you go to Register 3.

Best Use Case: Applications with **long-lived connections** (like Chat Applications, Multiplayer Games, or WebSockets) where a single connection might stay open for hours.

5.2.4.2 Weighted Least Connections

The Logic: The ultimate hybrid. It combines real-time monitoring with hardware capacity awareness.

The Math: When a request comes in, the load balancer quickly calculates a score for each server: $(\text{Active Connections}) / (\text{Weight})$. The server with the lowest score wins. * *Server A (Strong)*: 50 active connections, Weight = 5. Score = 10. * *Server B (Weak)*: 30 active connections, Weight = 2. Score = 15. * *Result*: Even though Server A has more absolute connections, it has a better ratio due to its hardware. The new request goes to Server A.

5.2.5 Load Balancing Strategy Comparison

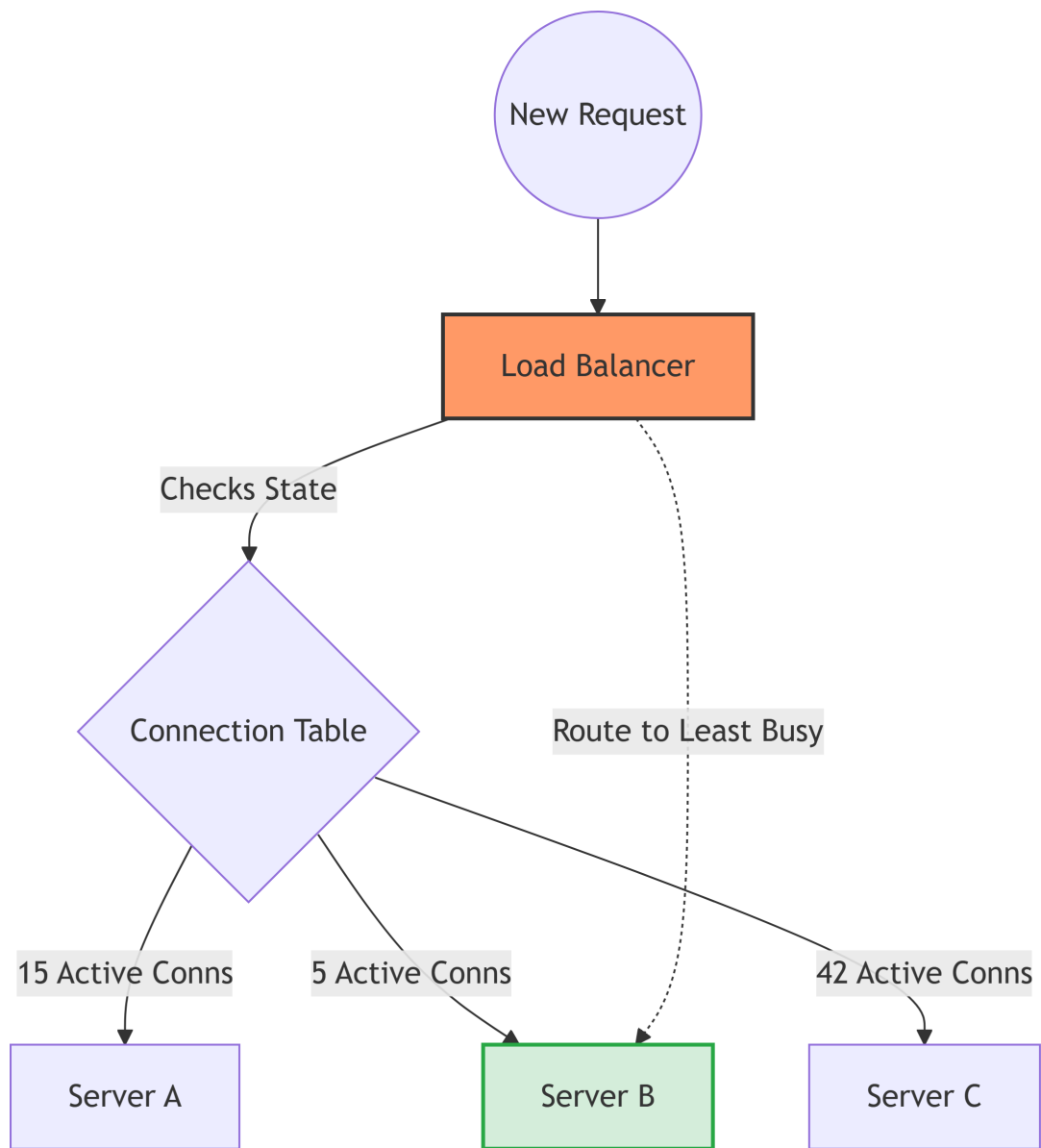


Figure 5.8: Least Connections Algorithm

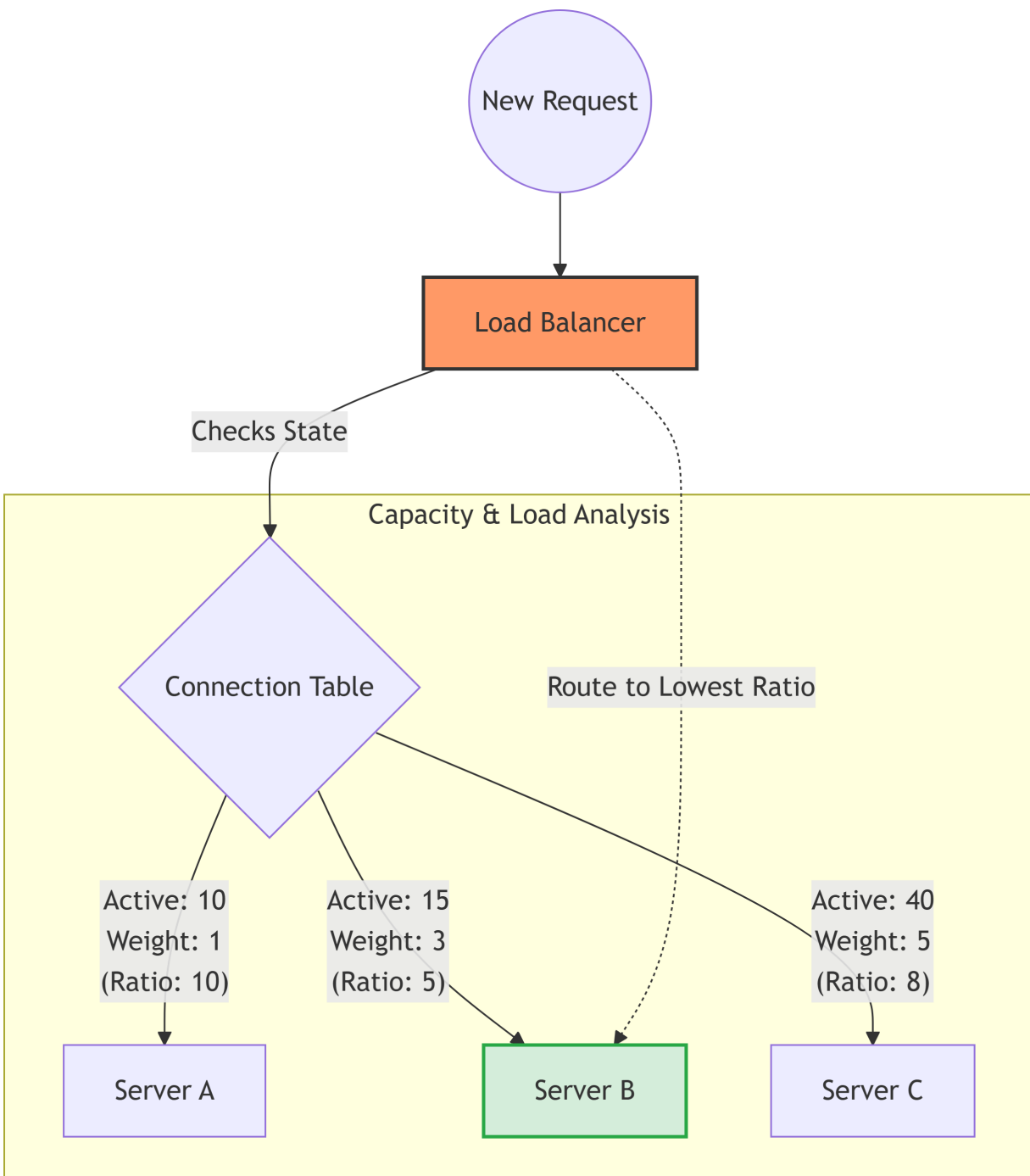


Figure 5.9: Weighted Least Connections Algorithm

Algorithm	Distribution Logic	Server Specs	Data State	Best For
Round Robin	Sequential/Static	Identical	Stateless	Simple web servers, microservices
Weighted RR	Proportion-based	Different	Stateless	Heterogeneous hardware clusters
Least Connections	Active Session Count	Identical	Stateful	SQL Databases, FTP, WebSocket
Weighted Least Conn.	Active Count / Weight	Different	Stateful	Enterprise apps with varying node power

5.2.5.1 Pros, Cons, and Use Cases

1. Round Robin

- **Pros:** Extremely low overhead; no processing required to choose a server; easy to implement and debug.
- **Cons:** Does not account for server health or current load; can easily overwhelm a slower server in a mixed cluster.
- **Ideal Use:** Quick-response, stateless services where each request takes roughly the same amount of CPU/Memory.

2. Weighted Round Robin

- **Pros:** Maximizes utilization of powerful hardware while still utilizing older/smaller nodes; better traffic management than standard RR.
- **Cons:** Weights are typically static; if a “powerful” server starts failing or slowing down, the LB will keep sending it the most traffic.
- **Ideal Use:** Cloud environments where you might have a mix of instance types (e.g., a few `c5.xlarge` and several `t3.medium` instances).

3. Least Connections

- **Pros:** Highly adaptive; naturally routes around servers that are struggling or taking longer to process requests.
- **Cons:** The Load Balancer must maintain a state table of all active connections, which increases LB memory/CPU usage.
- **Ideal Use:** Applications with long-lived sessions or “heavy” requests where some tasks take 10ms and others take 10 seconds.

4. Weighted Least Connections

- **Pros:** The most “intelligent” of the four; considers both the raw capacity of the server and its current real-time stress level.
- **Cons:** Most complex to configure; requires constant monitoring and high overhead on the load balancer.
- **Ideal Use:** Complex enterprise systems or database clusters where nodes have different RAM/CPU capacities and handle unpredictable, resource-intensive queries.

5.2.6 The Hybrid Reality

It is very common in massive enterprise systems to use **both** in a chained architecture.

A company might use an AWS Network Load Balancer (Layer 4) at the very edge of their network to absorb a massive DDoS attack and handle millions of raw TCP connections. That NLB then forwards the safe traffic to an internal cluster of NGINX servers (Layer 7) which decrypt the traffic and smartly route it to the correct microservices.

Layer 4/Layer 7 and **Static/Dynamic** are not competing categories. They are two completely different “dials” on the load balancer’s control panel.

- **Dial 1 (The Layer):** *What information can I see?* (Just IP/Ports, or full HTTP paths?)
- **Dial 2 (The Algorithm):** *What math do I use to pick the server?* (A fixed rule, or real-time server health?)

Because these dials are independent, you can mix and match them. However, in a massive enterprise system, they actually **stack on top of each other in a specific hierarchy**.

5.2.6.1 The Intersection (Mixing the Dials)

A load balancer at *either* layer can use *either* type of algorithm. Here is what that looks like in practice:

Configuration	What it looks like in the real world
Layer 4 + Static	A basic TCP router blindly forwarding raw packets using Round Robin . It’s incredibly fast but entirely oblivious to server health.
Layer 4 + Dynamic	A TCP router using Least Connections . It doesn’t know what the user is asking for, but it <i>does</i> know Server B currently has the fewest open TCP sockets, so it sends the raw packet there.
Layer 7 + Static	An NGINX server looking at the URL path (/api) and using Weighted Round Robin to blindly alternate between three backend API servers based on their CPU cores.
Layer 7 + Dynamic	An AWS Application Load Balancer looking at the URL (/video), decrypting the SSL, and using Least Response Time to send the request to whichever video server is currently replying the fastest.

5.2.7 The Enterprise Hierarchy (The “Stack”)

In a FAANG-scale system design interview, you don’t just choose one load balancer. You build a **funnel**. You put the “dumb but fast” balancers at the front to absorb the massive hit of the public internet, and you put the “smart but slow” balancers behind them to do the delicate microservice routing.

The three-tier hierarchy of a massive distributed system:

5.2.7.1 Tier 1: Global DNS Load Balancing (Static)

- **The Job:** Route the user to the closest data center in the world.
- **How it works:** When a user types `uber.com`, the DNS uses a static algorithm (like **Geolocation Routing**) to resolve the IP address. If the user is in Mumbai, DNS statically returns the IP address for the Mumbai data center. It does no real-time health checking of individual servers.

5.2.7.2 Tier 2: The Edge Layer 4 Load Balancer (Dynamic)

- **The Job:** Absorb raw network traffic and defend against DDoS attacks.
- **How it works:** The user’s connection hits the data center’s Layer 4 Network Load Balancer (NLB). The NLB handles raw TCP packets at millions of requests per second. It uses a dynamic algorithm (like **Least Connections**) to distribute this massive wave of raw traffic across a pool of internal Layer 7 proxies.

5.2.7.3 Tier 3: The Internal Layer 7 Load Balancer (Dynamic)

- **The Job:** Decrypt the traffic, read the request, and route to the correct microservice.
- **How it works:** The traffic arrives at the Layer 7 load balancer (e.g., NGINX or Envoy). This LB does the heavy CPU lifting. It decrypts the HTTPS, reads the HTTP headers, and sees the user is asking for `/payments`. It then uses a dynamic algorithm (like **Least Response Time**) to route that specific request to the healthiest server in the Payment Microservice cluster.

5.3 The Proxy – Forward vs. Reverse

A proxy is simply a middleman. It sits between two communicating parties and intercepts the traffic. The difference lies entirely in **who the proxy is trying to protect**.

5.3.1 The Forward Proxy (Protecting the Client)

The Architecture: Sits in front of the *users* (clients) and intercepts their outgoing requests to the internet. The internet only sees the proxy's IP address; it never sees the user's real IP.

The Anecdote (The Corporate Mailroom): Imagine working at a highly secure intelligence agency. If you want to send a letter, you can't put your personal home address as the return address. You give it to the mailroom. The mailroom repackages it with the "Agency Headquarters" return address and sends it. The recipient only knows it came from the Agency.

Use Cases: * **Corporate filtering:** Blocking employees from visiting social media. * **Anonymity:** VPNs are essentially encrypted forward proxies. * **Bypassing Geo-blocks:** A user in Mumbai connects to a Forward Proxy in New York to access US-only content.

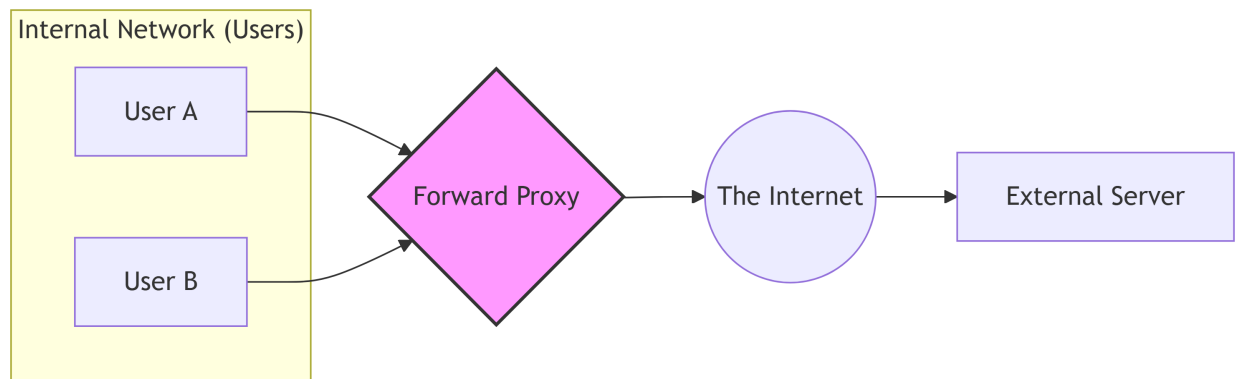


Figure 5.10: The Forward Proxy (Protecting the Client)

5.3.2 The Reverse Proxy (Protecting the Server)

The Architecture: Sits in front of your *web servers* and intercepts incoming requests from the internet. The user thinks they are talking directly to the server, but they are actually talking to the proxy.

The Anecdote (The Celebrity Spokesperson): A famous celebrity doesn't answer their own phone or talk directly to the paparazzi. They hire a Public Relations Spokesperson. The public asks the spokesperson a question, the spokesperson goes to the back room, asks the celebrity, gets the answer, and tells the public.

Use Cases: * **Security:** Hiding the internal IP addresses of your database and web servers from hackers. * **SSL Termination:** Decrypting HTTPS traffic at the edge so internal servers don't waste CPU doing it. * **Load Balancing:** This is the critical link! **A Load Balancer is simply a highly specialized Reverse Proxy.**

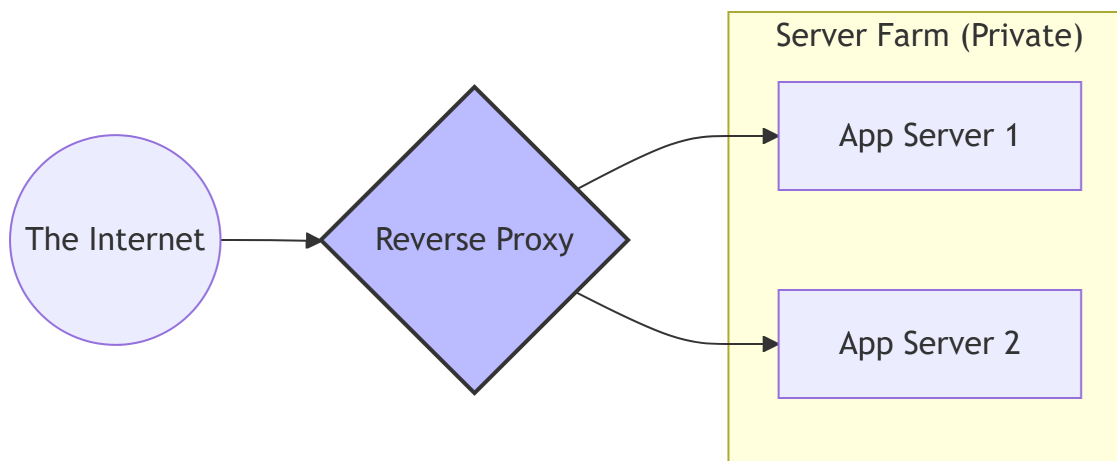


Figure 5.11: The Reverse Proxy (Protecting the Server)

5.3.3 Proxy Comparison: Forward vs. Reverse

When designing robust systems, choosing the right proxy depends entirely on your goal: protecting the **privacy of your users** (Forward) or the **stability of your infrastructure** (Reverse).

Feature	Forward Proxy	Reverse Proxy
Primary Beneficiary	The Client (User)	The Server (Backend)
Visibility	User is aware; Server sees the Proxy's IP	User is unaware; User sees the Proxy's IP
Key Use Cases	Content filtering, Anonymity, Bypassing geo-blocks	Load balancing, SSL termination, Caching, WAF
Traffic Direction	Outbound (Internal → Internet)	Inbound (Internet → Internal)
Security Focus	Protects client identity/privacy	Protects server from DDoS/Direct attacks
Common Tools	Squid , Apache (<code>mod_proxy</code>), Shadowsocks , Privoxy	Nginx , HAProxy , Traefik , Caddy , Envoy

5.3.3.1 Forward Proxy Tools

- **Squid:** The “old reliable” of forward proxies. Excellent for high-performance caching and deep ACL (Access Control List) filtering in corporate environments.
- **Privoxy:** A non-caching web proxy with advanced filtering capabilities for enhancing privacy and removing ads.
- **Shadowsocks:** Frequently used as a secure SOCKS5 proxy to bypass internet censorship.

5.3.3.2 Reverse Proxy Tools

- **Nginx:** Likely the most popular choice globally. It’s a web server that doubles as an incredibly efficient reverse proxy and load balancer.
- **HAProxy:** The gold standard for high-availability load balancing. It provides deep health checking and advanced routing logic.
- **Traefik / Envoy:** The “modern” choices. These are cloud-native proxies designed to integrate seamlessly with Docker and Kubernetes for automatic service discovery.

5.4 Throttling and Rate Limiting

While load balancing distributes the traffic, rate limiting acts as the ultimate shield against both malicious attacks (DDoS) and accidental self-inflicted crashes (runaway scripts).

Concept: In the industry, “Rate Limiting” and “Throttling” are often used interchangeably, but strictly speaking, they serve two slightly different architectural purposes.

5.4.1 Rate Limiting

The Goal: To protect the API from being overwhelmed by too many requests in a specific time window.

The Action: If a user exceeds their quota (e.g., 100 requests per minute), the server immediately drops the request and returns an **HTTP 429 (Too Many Requests)** error. * **The Anecdote (The Nightclub Bouncer):** The fire marshal says the club can only hold 100 people. Once 100 people are inside, the bouncer puts up their hand. Anyone else who tries to enter is immediately rejected and told to go away.

5.4.2 Throttling

The Goal: To control the *speed* or *flow* of data, ensuring fair usage across all users rather than just dropping connections instantly.

The Action: Instead of outright rejecting the request, the server artificially slows down the processing (putting the request in a queue) or limits the bandwidth (download speed).

The Anecdote (The Highway On-Ramp): During rush hour, a traffic light at the highway on-ramp turns green every 5 seconds. It doesn’t tell the cars to go home (Rate Limiting); it just forces them to space out and merge smoothly so the main highway doesn’t come to a standstill.

5.4.3 The Core Algorithms

Concept: How does the API Gateway actually keep count of millions of users in real-time? We use four primary algorithms, usually backed by a blazing-fast in-memory cache like Redis.

5.4.3.1 The Token Bucket

This is used by Amazon (AWS API Gateway) and Stripe.

- **The Logic:** Imagine a bucket that holds exactly 10 tokens. Every time a user makes a request, they take 1 token out of the bucket. If the bucket is empty, the request is dropped. Meanwhile, a background process drops a new token into the bucket every 1 second (the “refill rate”).
- **The Superpower:** It allows for **bursts** of traffic. A user can fire 10 requests at the exact same millisecond (emptying the bucket instantly), but then they must wait for it to refill.

5.4.3.2 2. The Leaky Bucket

Used heavily in network routing (like telecom companies).

The Logic: Imagine a bucket with a hole in the bottom. User requests are poured into the top of the bucket. The bucket leaks (processes requests) at a strict, constant rate—say, 5 requests per second. If the user pours requests into the top faster than they leak out the bottom, the bucket overflows, and the new requests are dropped.

The Superpower: It completely smooths out traffic. Even if the user sends a massive burst, the backend servers only ever see a steady, predictable drip of 5 requests per second.

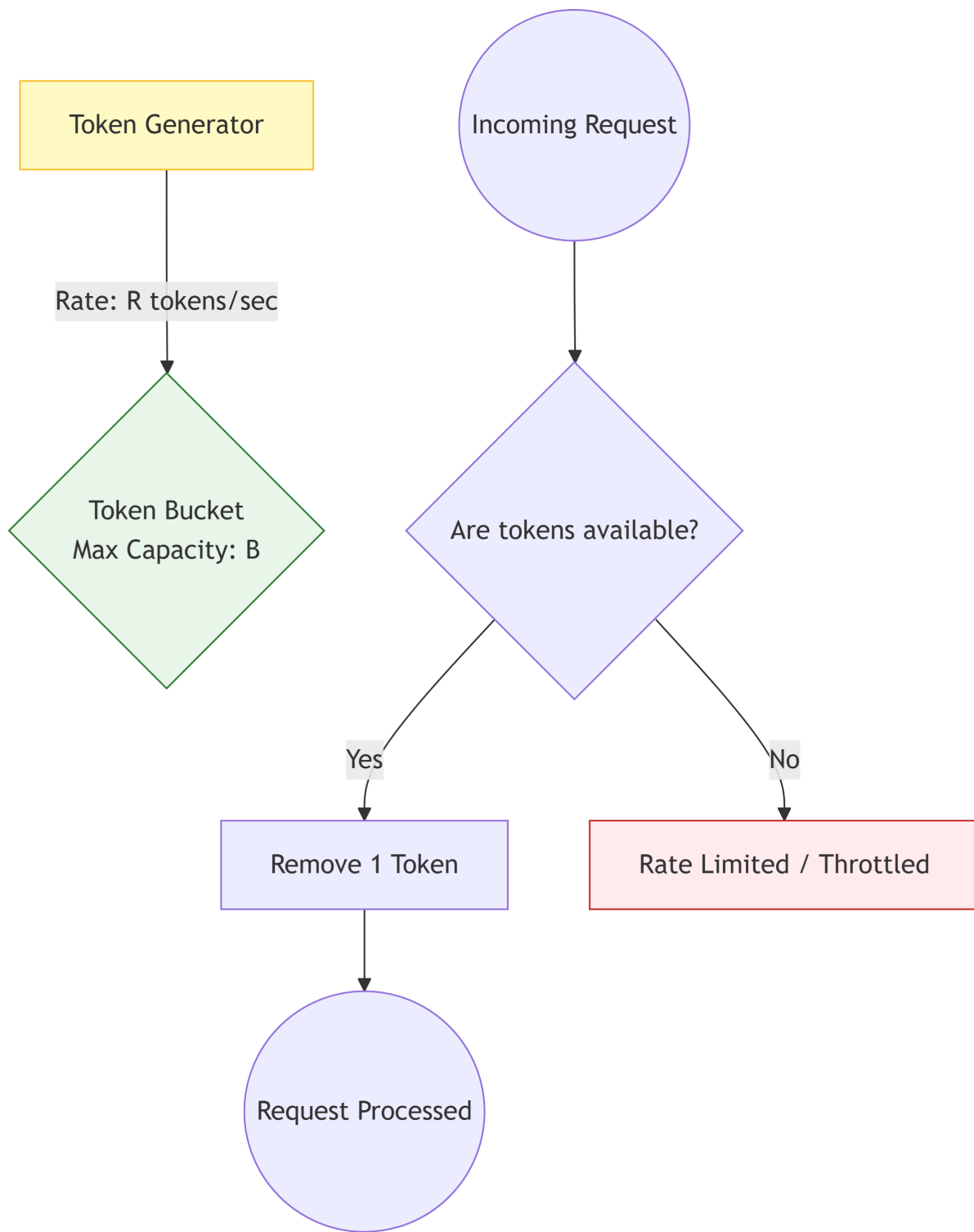


Figure 5.12: Token Bucket Algorithm

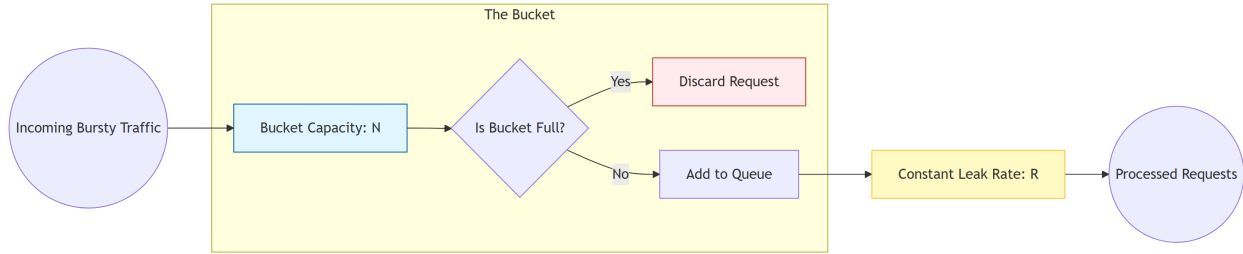


Figure 5.13: Leaky Bucket Algorithm

5.4.3.3 Fixed Window Counter

The Logic: The server divides time into strict blocks (e.g., 12:00 to 12:01, 12:01 to 12:02). Users get 100 requests per minute block.

The Fatal Flaw (The Edge Problem): A malicious user sends 100 requests at 12:00:59. The clock rolls over to 12:01:00, resetting their counter, and they immediately send another 100 requests. They just sent **200 requests in 2 seconds**, completely bypassing the intent of your 100/minute limit and potentially crashing the database.

5.4.3.4 Sliding Window

The Logic: To fix the Fixed Window edge problem, the server doesn't use rigid time blocks. Instead, when a request arrives, the server looks back exactly 60 seconds from *that exact millisecond*.

The Trade-off: It is perfectly accurate and prevents the 200-request burst. However, it requires storing a massive amount of timestamp data in Redis (a "log" of every single request time), making it much more memory-intensive than the Token Bucket.

5.4.4 Comparison of Rate Limiting Algorithms

In **Advanced System Design and SRE**, selecting a rate limiter isn't just about blocking traffic; it's about choosing a **traffic-shaping policy** that aligns with your infrastructure's capacity and your users' experience.

Algorithm	Logic	Burst Handling	Memory Cost	Accuracy	Best For
Token Bucket	Tokens added at rate R ; consumed per request.	Excellent (up to bucket size B)	Low (stores 2 values)	High	Modern public APIs (Stripe, AWS)
Leaky Bucket	Requests leak out at a constant, smooth rate.	None (strictly smoothed)	Low (Queue size)	High	Traffic shaping, Network egress
Fixed Window	Counter resets at discrete time intervals.	Poor (2x burst at boundaries)	Lowest (1 integer)	Low	Simple internal limits, Low-risk APIs
Sliding Window Log	Tracks exact timestamps of all recent requests.	Moderate (strictly rolling)	High (stores all timestamps)	Highest	High-security (Login, Billing)

Algorithm	Logic	Burst Handling	Memory Cost	Accuracy	Best For
Sliding Window Counter	Weighted average of current and previous windows.	Good (smooths boundaries)	Medium (2 counters)	High	High-scale distributed systems

5.4.4.1 Pros, Cons, and Applications

1. Token Bucket

- **Pros:** Very memory efficient; natively handles occasional bursts without failing legitimate users; easy to implement in distributed systems (Redis).
- **Cons:** “Burstiness” can potentially stress downstream services if B is set too high.
- **Application:** Standard choice for **REST APIs** and **SaaS platforms** where you want to allow a developer to perform a quick burst of actions (like a script startup) but limit their long-term average.

2. Leaky Bucket

- **Pros:** Guarantees a perfectly stable and predictable load on your backend; effectively “shapes” irregular traffic into a smooth stream.
- **Cons:** Can increase latency for users during bursts as requests are queued; lacks the flexibility to reward “quiet” periods with burst capacity.
- **Application:** Ideal for **asynchronous processing** or **database writers** where the downstream system has a very rigid maximum throughput it cannot exceed.

3. Fixed Window

- **Pros:** The easiest to understand and code; requires negligible memory per user.
- **Cons:** Vulnerable to “Boundary Spikes”—a user can send 100% of their limit at the very end of Window A and another 100% at the very start of Window B.
- **Application:** Suitable for **coarse-grained limits** like “1000 emails per day” or “5 password resets per hour” where the exact millisecond of the request doesn’t matter.

4. Sliding Window (Log & Counter) * **Pros:** Solves the boundary problem of Fixed Windows; ensures that at **any** given moment, the user hasn’t exceeded the limit over the last X seconds. * **Cons:** **Log** version is a memory hog ($O(N)$ where N is request count); **Counter** version is a mathematical approximation (though usually “good enough”). * **Application:** **Distributed Rate Limiters** in microservices. Many SREs prefer the **Sliding Window Counter** in Redis because it offers $O(1)$ memory like a Fixed Window but provides the fairness of a Sliding Window.

5.4.5 The User Experience & HTTP Headers

Concept: Good system design isn’t just about protecting the server; it is about communicating clearly with the client so their frontend application doesn’t crash.

When implementing Rate Limiting, you must use standard HTTP headers to tell the developer what is happening.

- **X-RateLimit-Limit:** “You are allowed 100 requests per hour.”
- **X-RateLimit-Remaining:** “You have 12 requests left.”
- **HTTP 429 Too Many Requests:** “You hit zero. Stop talking to me.”
- **Retry-After:** “Do not try again for 1,450 seconds.” (This prevents the client from aggressively polling the server to see if their timeout is over).

The Anecdote: Imagine going to an ATM. It doesn’t just eat your card when you hit your daily withdrawal limit. It explicitly prints a receipt saying: “*Your limit is \$500. You have \$0 remaining. Try again in 12 hours.*”

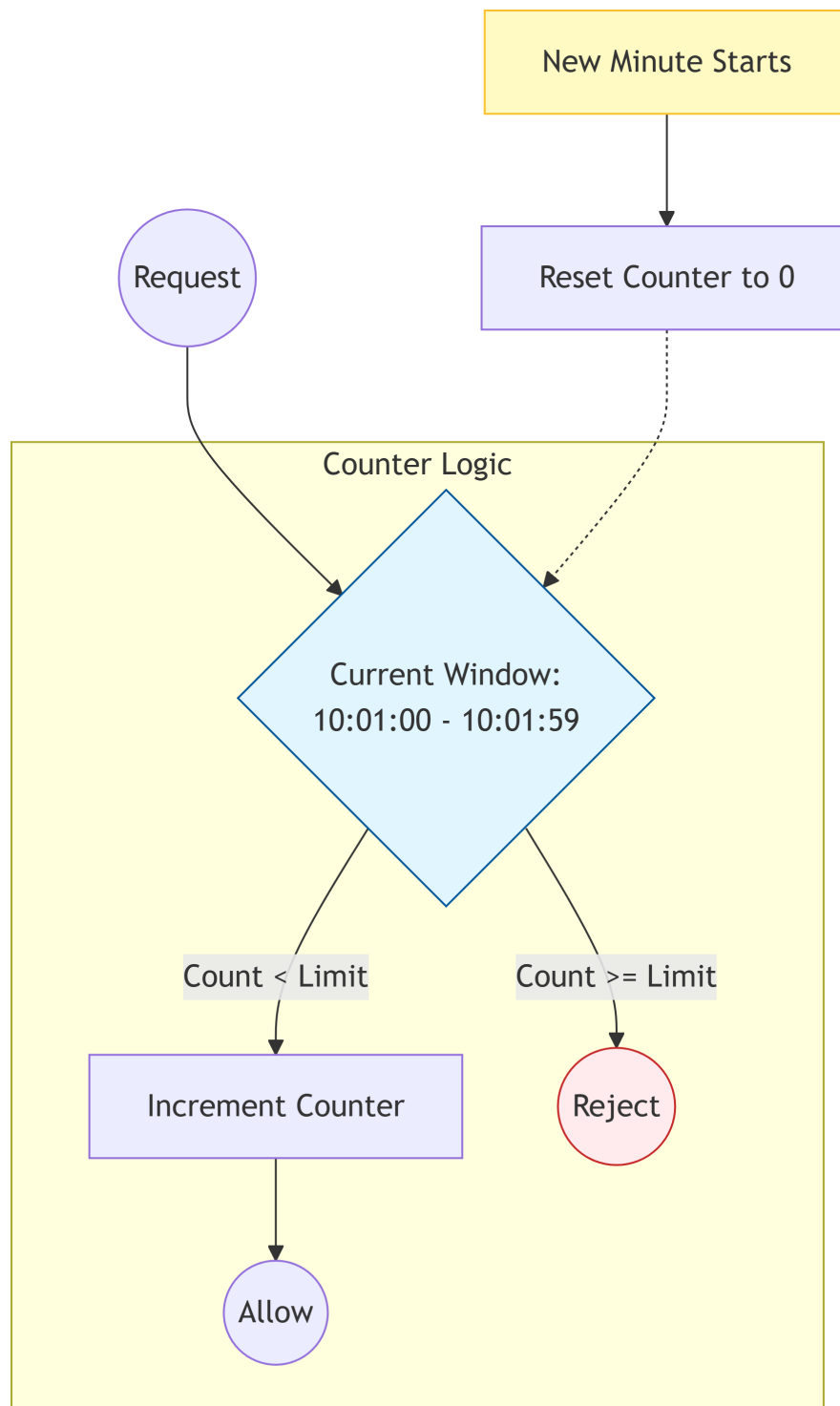


Figure 5.14: Fixed Window Counter

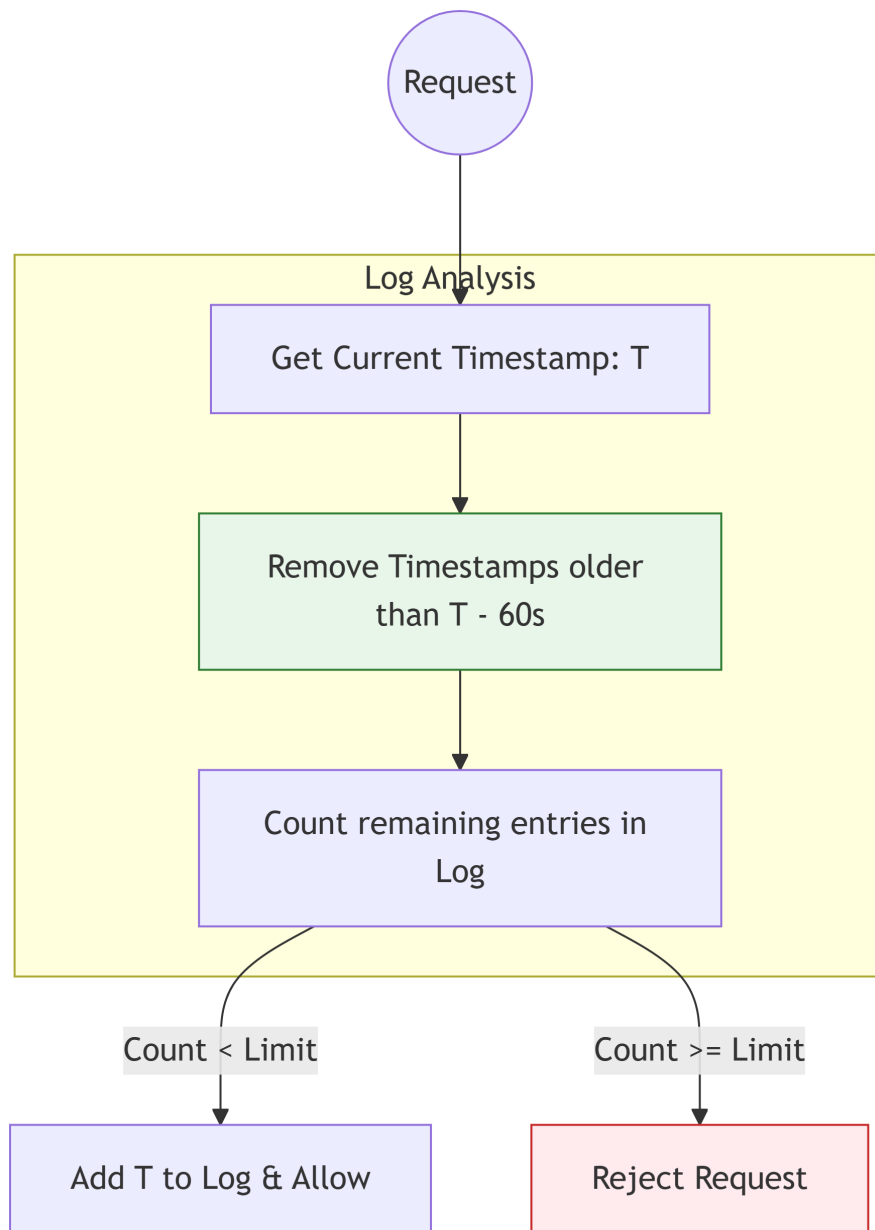


Figure 5.15: Sliding Window

Algorithm	Primary Trait	Best Used For
Token Bucket	Allows bursts	Public APIs (like Stripe or Twitter) where users occasionally send batches of data.
Leaky Bucket	Constant processing rate	Asynchronous queues or legacy databases that cannot handle any sudden spikes.
Fixed Window	Low memory, strict time blocks	Simple internal tools where malicious bursts aren't a concern.
Sliding Window	Perfect accuracy	High-security financial endpoints where strict limits are legally enforced.

5.4.6 Industry grade use case

The Reverse Proxy / API Gateway (The Industry Standard)

If your goal is to protect your servers from DDoS attacks, malicious scrapers, or generalized traffic spikes, you apply rate limiting at the edge (Nginx, HAProxy, AWS API Gateway, Cloudflare, or Kong).

How it works: The proxy tracks requests based on the user's IP address or a generic API key. If an IP hits the limit, the proxy instantly returns an HTTP 429 without ever bothering your FastAPI or Django servers.

The Massive Advantage (Saving the CPU): Your Python application is computationally expensive. It has to load the framework, parse JSON, and maybe boot up the ORM. If a hacker sends 10,000 requests a second, letting those requests reach your Python app—even just for Django to say “no”—will burn so much CPU that your server might crash anyway. The proxy handles this in C or Rust at lightning speed, acting as a blast shield.

Centralization: If you have 5 different microservices (User Profile, Payments, Video Streaming), you don't want to write rate-limiting code in 5 different codebases. You write it once at the gateway.

2. The Backend Application (FastAPI / Django)

You move rate limiting into your application code when the limit requires **business context** that the “dumb” reverse proxy doesn't have.

How it works: You use a library like `fastapi-limiter` or `django-ratelimit`, usually hooked up to a Redis cache. The request reaches your app, Python checks Redis, and Python decides whether to proceed or return a 429.

The Use Case (Tiered Billing): Imagine you run a SaaS company. * “Free Tier” users get 10 requests per minute. * “Pro Tier” users get 1,000 requests per minute. * “Enterprise” users have no limits. Your Nginx proxy generally doesn't know what database tier a user belongs to. Your Django app *does*. Therefore, Django must inspect the user's JWT token, look up their billing tier, and apply the specific rate limit.

Fine-Grained Actions: You might want global traffic to be unlimited, but specifically rate limit the “Reset Password” endpoint to 3 times per hour to prevent spam emails. This is much easier to write as a simple Python decorator (`@ratelimit(rate='3/h')`) directly above the function than trying to configure complex URL-matching rules in an Nginx config file.

The Hybrid Reality (Defense in Depth)

In a robust enterprise system, you don't choose one; you use both. They stack together to form “Defense in Depth.”

1. **The Global Shield (Proxy):** Nginx is configured with a broad, massive limit (e.g., “No single IP address can make more than 5,000 requests per minute”). This prevents a single botnet from taking down your infrastructure.

2. **The Business Bouncer (Python):** Inside the application, FastAPI checks the user’s subscription tier and enforces a tighter, business-specific limit (e.g., “This Free Tier user is limited to 50 API calls per minute”).

If you are just building a simple project or starting out, utilizing the **Proxy** is the safest, most performant way to keep your database from melting!

5.5 Performance Optimization

This chapter focuses on the architectural maneuvers required to reduce latency and increase throughput in high-scale systems. In System Design, performance optimization isn’t just about “faster code”—it’s about managing resources efficiently to avoid bottlenecks.

5.5.1 Connection Pooling

Opening a new database connection is an expensive operation involving a TCP handshake, TLS negotiation, and backend resource allocation.

The Problem: Connection Overhead In a naive setup, every incoming request opens and closes a connection. Under load, the database spends more CPU cycles managing connection overhead than executing queries.

The Solution: The Pool A **Connection Pool** maintains a “cache” of live, idle connections. When the application needs to talk to the DB, it “borrows” a connection from the pool and “returns” it when finished.

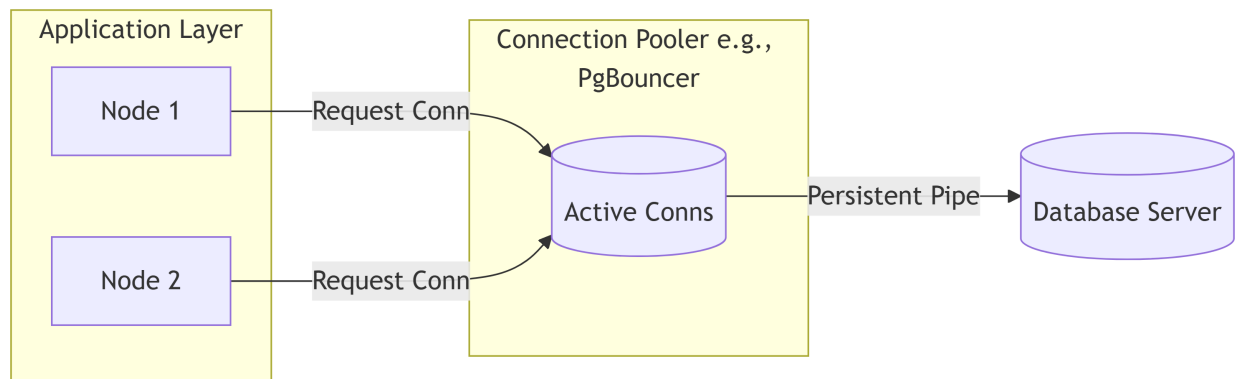


Figure 5.16: Connection Pooling

- **Key Benefit:** Reduces latency for individual requests and prevents the database from crashing due to “Too many connections.”
- **Tools:** **HikariCP** (Java), **PgBouncer** (PostgreSQL), **SQLAlchemy Pool** (Python).

5.5.2 Content Compression (Gzip/Brotli)

Reducing the payload size directly decreases the **Time to First Byte (TTFB)** and total download time, especially for mobile users on limited bandwidth.

Implementation

- **Gzip:** The industry standard. Uses the DEFLATE algorithm to compress text-based assets (HTML, CSS, JS, JSON).
- **Brotli:** A newer algorithm from Google that typically achieves **15–20% better compression** than Gzip for static assets.

Strategic Trade-off: CPU vs. Bandwidth

Compression is a trade-off. The server uses CPU cycles to compress, and the client uses CPU to decompress.
* **Best for:** Text (JSON/HTML) which is highly repetitive. * **Worst for:** Already compressed formats (JPEG, PNG, MP4). Compressing these again adds CPU overhead with zero size benefit.

5.5.3 Reducing I/O (Disk and Network)

I/O is almost always the slowest part of a system. > **The Golden Rule:** The fastest I/O is the one you don't do.

Strategies

- **Batching:** Instead of sending 100 small network requests, batch them into one. This reduces the “network trip” overhead.
- **Sequential vs. Random I/O:** Disk reads are much faster when data is stored sequentially (like in Kafka logs or LSM Trees) compared to jumping around (Random I/O).
- **Caching:** Use Redis or Memcached to keep frequently accessed data in RAM, avoiding the disk entirely.
- **Zero-Copy:** In high-performance systems (like Kafka), data is moved from the disk buffer directly to the network card buffer, bypassing the application memory entirely.

5.5.4 Database Query Optimization

A poorly indexed table can turn a 10ms query into a 10-second nightmare as the system grows.

The “Explain” Plan

Always use EXPLAIN ANALYZE to see if the database is doing a **Sequential Scan** (bad for large tables) or an **Index Scan** (good).

Key Techniques

1. **Indexing:** Create B-Tree indexes for equality/range queries and GIN indexes for full-text search.
2. **Avoid SELECT *:** Only fetch the columns you need. This reduces the amount of data moved from the DB disk to the App memory.
3. **N+1 Query Problem:** Use JOINS or Prefetch (in ORMs like Django) to get related data in a single query rather than firing new queries inside a loop.
4. **Denormalization:** In read-heavy systems, purposefully duplicating data to avoid expensive JOINS across multiple tables.

Summary Comparison Table

Technique	Primary Goal	Bottleneck Addressed
Connection Pooling	Reduce Handshake Overhead	Database CPU/Memory
Gzip/Brotli	Reduce Payload Size	Network Latency/Bandwidth
Reducing I/O	Minimize Data Movement	Disk/Network Latency
Query Optimization	Reduce Search Space	Database Execution Time

Chapter 6

Asynchronous Processing and Communication

In traditional web development using frameworks like Django or FastAPI, the default communication pattern is synchronous. A client sends an HTTP request, the server processes it, and the server returns a response. While effective for simple CRUD operations, this “Request-Reply” model creates severe bottlenecks in distributed systems. When microservices communicate synchronously, they form a “distributed monolith”—a chain of dependencies where the entire system is only as fast or as reliable as its slowest component.

To achieve true scalability and fault tolerance, modern systems decouple components temporally using asynchronous message passing.

6.1 Message Queues, Brokers, and Routing Models

At the heart of asynchronous design is the message broker—a centralized intermediary that accepts, stores, and routes messages between applications. This decoupling prevents sudden spikes in traffic from overwhelming backend databases.

There are two primary models for distributing these messages

6.1.1 Point-to-Point (Queue) Model

In this model, a message is sent to a specific queue and is consumed by exactly one worker. Once the task is processed, the message is permanently deleted. * **The Architecture:** Multiple producers can send tasks to the queue, and multiple consumers can listen to it. The broker distributes the tasks among the available workers in a round-robin fashion, ensuring no two workers process the same task. * **Real-World Application:** Consider a computationally expensive task like rendering a high-resolution, color-graded S-Log3 video timeline. If a user triggers this render from a web application, the main thread cannot wait 20 minutes for it to finish. Instead, the web app places a “render task” into a Point-to-Point queue and immediately returns a success response to the user. A background worker farm pulls tasks from the queue one by one, processing them at its own pace without crashing the frontend.

6.1.2 Publish-Subscribe (Pub/Sub) Model

In the Pub/Sub model, a message is broadcast to a “Topic” or “Exchange.” It is not meant for a single worker; instead, an exact copy of the message is delivered to every service that has subscribed to that topic. * **The Architecture:** The producer does not know who the consumers are. It simply announces that an event occurred. * **Real-World Application:** Imagine a large-scale Student Event Participation Tracking System managing data for over 1500 students and 300 faculty members. When a student registers for a national

hackathon, that event is published to a topic. Simultaneously, the `EmailService` reads the event and sends a confirmation, the `AnalyticsService` updates the department dashboard, and the `LedgerService` logs the activity. If the `EmailService` crashes, the other two continue operating flawlessly.

6.1.3 RabbitMQ

Introduction to RabbitMQ

RabbitMQ is one of the most widely deployed open-source message brokers in the world. Written in Erlang and built natively to support the **AMQP** (Advanced Message Queuing Protocol), it acts as a highly reliable intermediary that enables applications to communicate with each other asynchronously.

You can think of RabbitMQ as a digital post office for microservices. Instead of applications calling each other directly and waiting for a response (which causes bottlenecks), they hand their messages off to RabbitMQ, which guarantees that the data will be routed and delivered to the correct destination when the receiving service is ready.

Core Philosophy: “Smart Broker, Dumb Consumer”

RabbitMQ’s defining characteristic is its complex, built-in routing intelligence. The broker itself takes on the heavy lifting of figuring out where a message needs to go. The consumer services do not need to know anything about the broader system architecture; they simply connect to their designated queue, pull a task, and process it. Once a consumer acknowledges that a task is complete, RabbitMQ permanently deletes the message from the queue.

The 4 Pillars of RabbitMQ Architecture To understand how data flows through RabbitMQ, it is essential to understand its four primary components:

1. **Producer:** The application that generates and sends the message. *Crucially, a producer in RabbitMQ never sends a message directly to a queue.* It only sends messages to an Exchange.
2. **Exchange:** The routing mechanism (the sorting office). It receives messages from the producer and pushes them into zero, one, or multiple queues based on predefined rules.
3. **Bindings:** The specific rules or “links” connecting an Exchange to a Queue. Bindings dictate exactly *how* the Exchange should route the data (e.g., exact match, pattern match, or broadcast).
4. **Queue:** The buffer that stores the messages. Queues hold onto the data safely in memory or on disk until a worker is available to process it.
5. **Consumer:** The background worker or microservice that connects to the queue to retrieve and execute the messages.

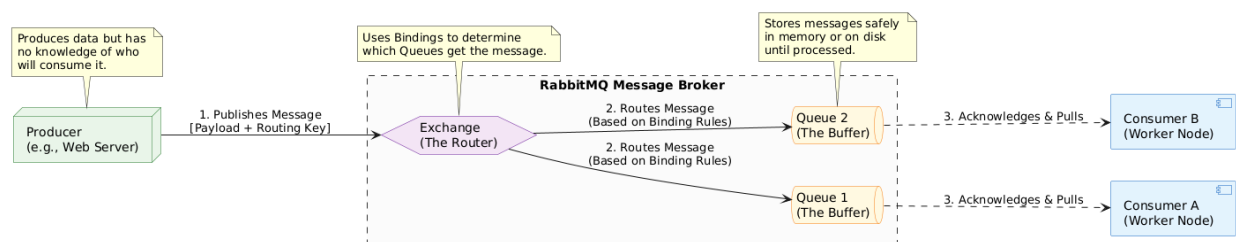


Figure 6.1: RabbitMQ Overview

6.1.3.1 Why Introduce RabbitMQ into a System?

- **Temporal Decoupling:** The producer and consumer do not need to be alive at the same time. If the backend database crashes, the web server can still accept user requests and store them safely in RabbitMQ until the database comes back online.

- **Load Leveling (Shock Absorption):** If a system experiences a sudden, massive spike in traffic, RabbitMQ absorbs the blow. The messages queue up, and the background workers process them at their maximum safe capacity without the server crashing.
- **Asynchronous Background Processing:** It keeps the user interface fast.

A Practical Scenario: Consider a high-traffic platform like a Student Event Participation Tracking System handling registrations for thousands of students and faculty members. When a student registers for a national hackathon, the web application shouldn't synchronously wait for the system to generate a PDF receipt, update the department ledger, and send a confirmation email.

Instead, the web application (the Producer) simply publishes an `EventRegistered` message to RabbitMQ and instantly tells the student, *"Registration Successful!"* Behind the scenes, RabbitMQ's Exchanges route that single event to multiple independent worker services (Consumers) that generate the PDF, update the database, and send the email at their own pace.

6.1.3.2 Exchange Types

1. Direct Exchange (Exact Match)

A Direct Exchange delivers messages to queues based on an exact match between the message's routing key and the queue's binding key. It is typically used for unicast routing of tasks.

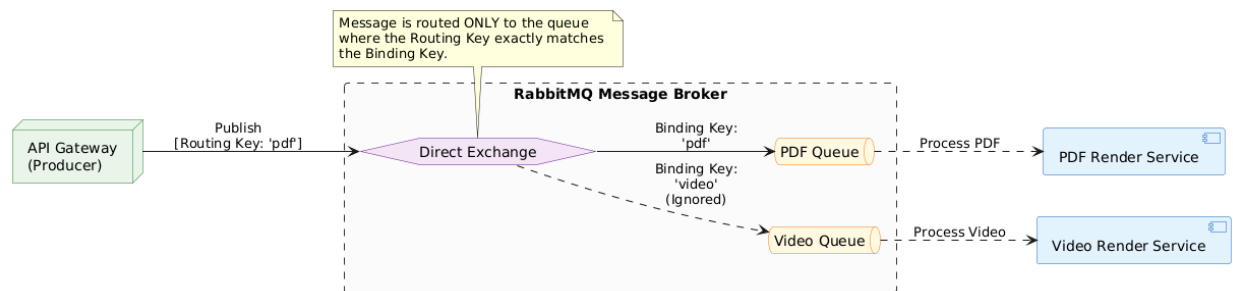


Figure 6.2: Direct Exchange (Exact Match)

2. Fanout Exchange (Broadcast)

A Fanout Exchange simply broadcasts all messages it receives to all queues that are bound to it, completely ignoring any routing keys.

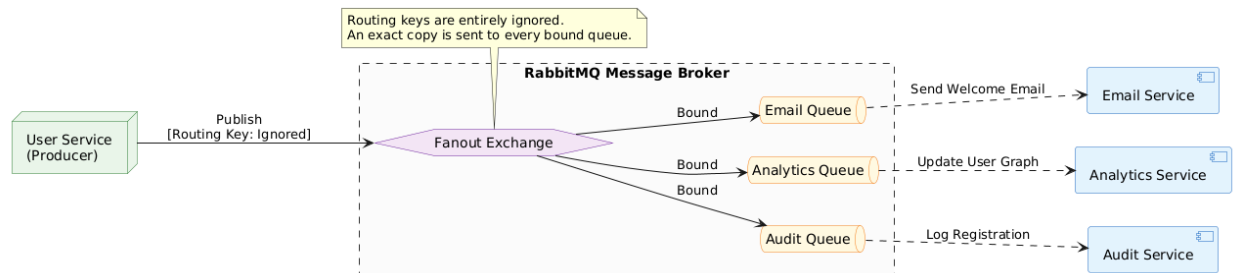


Figure 6.3: Fanout Exchange (Broadcast)

3. Topic Exchange (Pattern Matching)

A Topic Exchange routes messages to one or many queues based on wildcard matching. The routing key must be a list of words delimited by dots (e.g., `stock.usd.nyse`).

* matches exactly one word.
 # matches zero or more words.

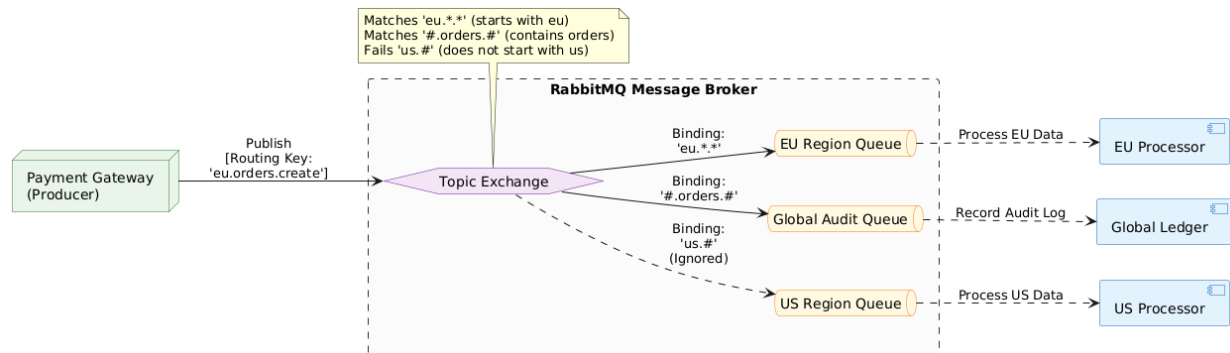


Figure 6.4: Topic Exchange (Pattern Matching)

4. Headers Exchange (Attribute Matching)

A Headers Exchange is designed for routing on multiple attributes that are more easily expressed as message headers than a routing key. It ignores the routing key and instead looks at standard headers (key-value pairs) inside the message payload.

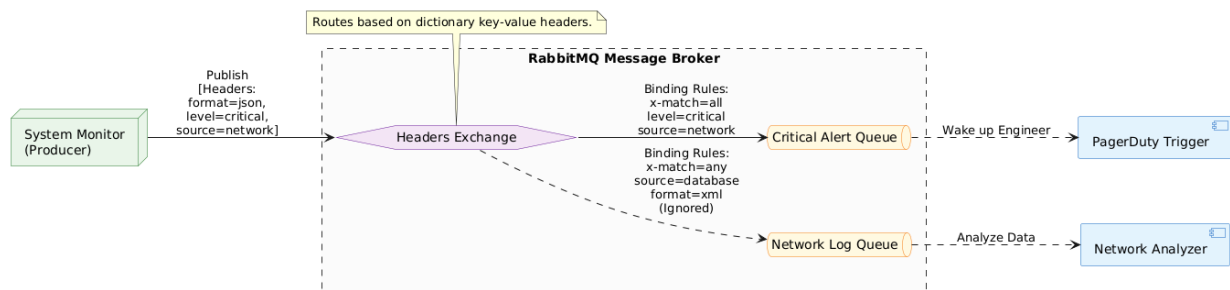


Figure 6.5: Header Exchange (Attribute Matching)

6.1.4 Apache Kafka

Apache Kafka is a distributed, high-throughput event streaming platform. Originally developed by LinkedIn to handle billions of activity tracking events per day, it is now an open-source Apache project and the industry standard for managing “Data in Motion.”

This diagram shows the high-level flow of data from Producers, into the distributed Kafka Cluster (organized by Topics), and out to Consumers. Notice that unlike RabbitMQ, the Consumers *pull* the data.

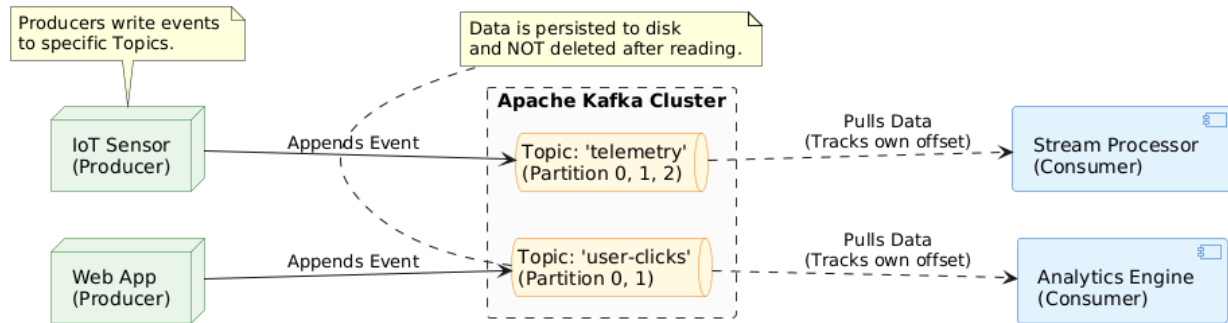


Figure 6.6: Apache Kafka Overview

While RabbitMQ operates like a digital post office (routing and deleting messages), Kafka operates like a **distributed, append-only ledger**. It does not delete messages once they are read. Instead, it permanently records a stream of historical events sequentially to a hard drive, allowing applications to replay the history of the system at any time.

Core Philosophy: “The Immutable Log” Kafka’s defining characteristic is its simplicity at the broker level. It adheres to the **“Dumb Broker, Smart Consumer”** philosophy.

Kafka does not track which consumer has read which message, nor does it push messages to workers. It simply writes incoming data to the end of a log. The consumers are responsible for actively pulling the data and keeping track of their own “bookmark” (called an offset) so they know where they left off. Because the log is immutable (unchangeable), multiple entirely different services can read the exact same stream of data at different speeds without interfering with each other.

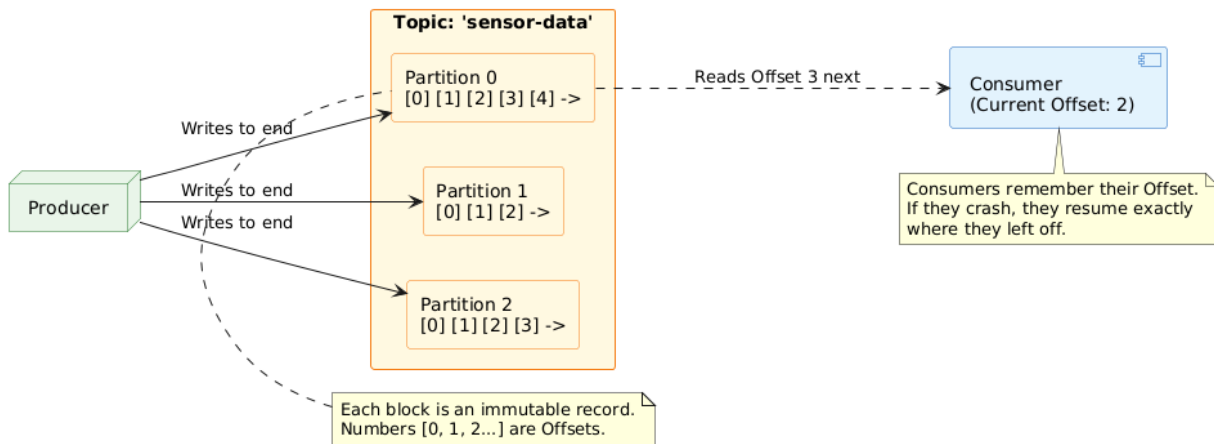


Figure 6.7: Topics, Partitions, and The Immutable Log

The 4 Pillars of Kafka Architecture

To understand how data flows through Kafka, you must understand its core abstractions:

1. **Producer:** The application or device that publishes streams of events to a specific topic.
2. **Topic & Partitions:** A Topic is a categorical name for a stream of records (e.g., `sensor-telemetry`). To scale across multiple servers, a Topic is divided into **Partitions**. Partitions are what allow Kafka to process massive volumes of data in parallel.

3. **Consumer & Consumer Groups:** The applications that read from topics. Consumers are organized into “Groups.” If you have three consumers in a group and a topic with three partitions, Kafka will perfectly load-balance the work, assigning one partition to each consumer.
4. **Broker & Cluster:** A single Kafka server is a Broker. A network of multiple brokers working together is a Cluster. Brokers replicate the data across each other so that if one server physically burns down, no data is lost.

Consumer Groups (Load Balancing vs. Broadcasting)

Kafka natively supports both Point-to-Point (Queueing) and Publish-Subscribe (Broadcasting) using Consumer Groups.

If consumers are in the same group, Kafka load-balances the partitions among them (Point-to-Point).

If consumers are in different groups, they each get a full copy of the data (Pub/Sub).

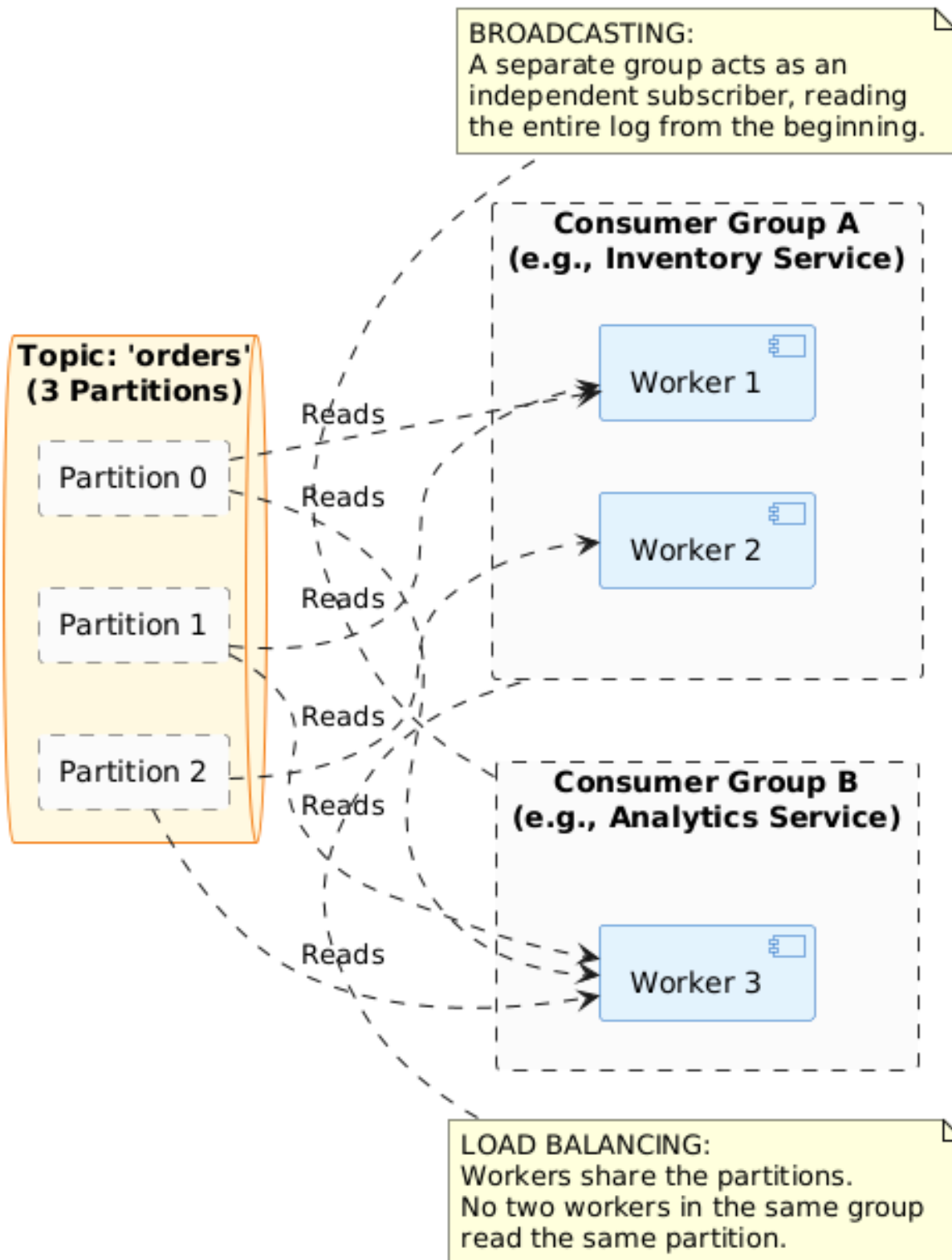


Figure 6.8: Topics, Partitions, and The Immutable Log

6.1.4.1 Why Introduce Kafka into a System?

- **Massive Throughput:** Kafka sequentially writes to the disk, which is incredibly fast. It can easily handle millions of messages per second.
- **Event Sourcing & Replayability:** Because messages are not deleted immediately, if a consumer crashes or a new service is added to the architecture months later, that new service can “rewind” the log and replay every event from the beginning of time to build its own database state.
- **Real-Time Stream Processing:** It serves as the foundation for complex pipelines where data is analyzed continuously before it ever hits a database.

A Practical Scenario: Consider a deployment of IoT edge devices running ML models to stream continuous time-series data. If these sensors are transmitting telemetry thousands of times per second, a traditional relational database would quickly collapse under the immense write-lock pressure.

In this architecture, Kafka acts as the ultimate industrial shock absorber. The edge devices (Producers) fire their data into a Kafka Topic. Kafka rapidly ingests that massive firehose, appending the data sequentially to disk. Downstream, your ML algorithms and time-series databases (Consumers) can pull the data from Kafka at a controlled, safe speed, ensuring no data packets are ever dropped, even during a massive traffic spike.

6.1.5 Kafka vs. RabbitMQ: Choosing the Right Broker

While both Apache Kafka and RabbitMQ handle asynchronous communication, their underlying architectures solve entirely different problems.

RabbitMQ (The Smart Broker)

RabbitMQ is a traditional message broker designed for task delegation. It uses complex, in-memory routing logic to push messages to consumers.

- **Philosophy:** “Smart Broker, Dumb Consumer.” The broker tracks exactly which messages have been delivered and deletes them as soon as the consumer acknowledges them.
- **Best for:** Complex routing, specific task delegation, and architectures where you want messages to disappear once handled.

Apache Kafka (The Immutable Log)

Kafka is not a queue; it is a distributed, append-only log. It writes every event sequentially to the hard drive, maintaining a permanent history of the system’s state.

- **Philosophy:** “Dumb Broker, Smart Consumer.” Kafka simply stores the data stream. Consumers must maintain their own “offsets” (bookmarks) to remember what they have read. Multiple different consumer groups can read the exact same log from the beginning without interfering with each other.
- **Best for:** High-throughput data ingestion, event sourcing, and real-time stream processing.

Feature / Aspect	Apache Kafka	RabbitMQ
Core Architecture	Distributed Append-Only Log. Data is written sequentially to disk. It acts as an immutable ledger of events.	Traditional Message Broker. Acts as a routing switchboard and temporary buffer for task delegation.
Design Philosophy	Dumb Broker, Smart Consumer. The broker simply stores the log. Consumers track their own read progress (offsets).	Smart Broker, Dumb Consumer. The broker calculates complex routing logic and tracks the exact delivery state of every message.

Feature / Aspect	Apache Kafka	RabbitMQ
Message Retention	Persistent. Messages are kept on disk for a configured duration (e.g., 7 days) or until a size limit is reached, regardless of whether they have been read.	Ephemeral. Messages are temporarily stored and permanently deleted the moment a consumer acknowledges (ACKs) successful processing.
Routing Flexibility	Basic (Pub/Sub). Producers publish to a Topic. Consumers subscribe to the Topic. Routing is strictly Topic-based.	Highly Advanced. Uses Exchanges (Direct, Fanout, Topic, Headers) to route messages to specific Queues based on intricate binding rules.
Throughput & Performance	Massive (Millions of msg/sec). Extremely high throughput due to sequential disk I/O and zero-copy data transfer.	High (Tens of thousands of msg/sec). Slower than Kafka due to the computational overhead of complex routing and per-message tracking.
Message Ordering	Guaranteed at the Partition level. Messages within a specific partition are strictly ordered, but order is not guaranteed across the entire topic.	Guaranteed at the Queue level. FIFO (First-In, First-Out) is maintained for messages passing through a single, specific queue.
Message Replayability	Yes. Because the log is immutable, new or recovering consumers can “rewind” their offset to replay historical events from the past.	No. Once a message is consumed and deleted, it is gone forever. Replaying history is not possible natively.
Scalability Mechanism	Partitioning. Topics are split into partitions distributed across a cluster of brokers, allowing massive parallel consumption.	Clustering & Federation. Nodes are clustered for high availability, and queues can be mirrored, but vertical scaling is often required for high loads.
Primary Protocols	Uses a custom, proprietary binary protocol over TCP.	Built natively for AMQP 0-9-1, but also supports MQTT and STOMP via plugins.
Payload Size	Optimized for small, high-frequency events (typically under 1MB).	Handles slightly larger payloads better, though standard best practice keeps messages relatively small.
Ideal Use Cases	Real-time stream processing, event sourcing, high-volume telemetry ingestion, log aggregation, and website activity tracking.	Task queues, microservice decoupling, point-to-point background jobs, and complex routing requirements.

6.2 Data Ingestion and Stream Processing

6.2.1 The Evolution of Data Processing: Batch vs. Stream

Historically, data processing was synonymous with **Batch Processing** (Data at Rest). Systems would collect data throughout the day, store it in a massive relational database or data warehouse, and run heavy ETL (Extract, Transform, Load) jobs overnight.

While batch processing is highly efficient for historical analytics, it fails in environments requiring immediate

action. Modern architectures have shifted toward **Stream Processing** (Data in Motion). In a streaming architecture, data is processed the millisecond it is generated, before it ever touches a permanent storage disk. This shift is what enables real-time fraud detection, dynamic algorithmic pricing, and instantaneous system alerts.

6.2.2 Data Ingestion: The Gateway to the Stream

Before data can be processed, it must be safely brought into the system. **Data Ingestion** is the architectural layer responsible for absorbing high-velocity, high-volume firehoses of data from external sources.

Consider a modern IIoT (Industrial Internet of Things) deployment: If multiple sensors detect rapid fluctuations, they might burst-transmit thousands of time-series telemetry pings per second.

If these edge devices attempted to write directly to a traditional PostgreSQL database, the sudden spike in concurrent connections and disk write-locks would instantly crash the database.

Instead, architectures utilize a high-throughput ingestion buffer—almost universally **Apache Kafka**. Kafka acts as an industrial shock absorber. The devices (Producers) fire their telemetry into a Kafka Topic. Kafka rapidly ingests this data, appending it sequentially to a highly optimized log on the disk. Downstream systems can then pull from this log at their own safe, controlled speeds without dropping a single packet.

6.3 Core Concepts of Stream Processing

Once data is safely ingested into a broker like Kafka, a stream processing engine takes over to analyze it on the fly. All stream processors rely on a few universal concepts:

1. Stateless vs. Stateful Processing

- **Stateless Processing:** Looking at a single event in isolation. For example, checking if a single temperature reading exceeds **100°C** and triggering an alert. This is simple and highly scalable.
- **Stateful Processing:** Processing an event based on the events that came before it. For example, triggering an alert only if the average temperature over the *last 5 minutes* exceeds **100°C**. The processor must temporarily memorize the “state” of the last 5 minutes.

2. Time Domains

Stream processors must reconcile two distinct types of time:

- **Event Time:** The exact timestamp when the event actually occurred out in the real world (e.g., when the hardware sensor recorded the anomaly).
- **Processing Time:** The timestamp when the event finally reached the stream processing server. Network latency can cause a massive disconnect between these two. Robust processors always calculate state based on *Event Time* to ensure accuracy.

3. Windowing Functions

To calculate aggregations (like averages or counts) on a never-ending stream of infinite data, the stream must be chopped into finite logical blocks called “Windows.”

- **Tumbling Windows:** Fixed-size, non-overlapping time blocks (e.g., exactly 00:00 to 00:05, then 00:05 to 00:10).
- **Sliding Windows:** Overlapping blocks of time that slide forward smoothly (e.g., an alert that triggers if the threshold is breached in *any* continuous 60-second period).
- **Session Windows:** Dynamic windows that group events based on activity bursts, separated by a defined period of inactivity (e.g., a user’s session on a website ends if they are idle for 10 minutes).

6.3.1 Kafka Streams: The Lightweight Native Engine

Kafka Streams is not a separate piece of infrastructure or a standalone cluster. It is simply a lightweight Java library that you embed directly inside your existing microservices.

It is designed to consume data from a Kafka topic, apply transformations, filtering, or aggregations, and immediately write the results back to a new Kafka topic.

Key Features:

- **Zero Cluster Overhead:** Because it is just a library, you deploy it exactly like you deploy any standard backend service (e.g., via Docker). You do not need to manage a separate “Kafka Streams Cluster.”
- **KStream vs. KTable Duality:** * A **KStream** represents an event stream (a history of every change).
 - A **KTable** represents a materialized view of the current state (like a traditional database table showing only the latest value for a specific key). Kafka Streams allows you to join streams and tables together seamlessly in memory.
- **Ideal Use Case:** Excellent for lightweight microservices already living deep inside the Kafka ecosystem that need to perform real-time ETL, data masking, or simple moving-average calculations.

Key Concepts:

1. **The Application Boundary:** The dashed box clearly shows that the Stream Processor and State Store live inside the microservice, not on the Kafka brokers.
2. **The Processor Topology:** Data flows sequentially from the Source Node (which deserializes the Kafka records), into the Processor (which applies your business logic), and out through the Sink Node (which serializes the data back to Kafka).
3. **Fault-Tolerant State:** If the processor does an aggregation (like “Count user clicks per minute”), it stores that running count locally in the State Store (typically RocksDB). To prevent data loss if the server crashes, Kafka Streams quietly streams every change in that local database to a hidden Changelog Topic back in the main Kafka cluster.

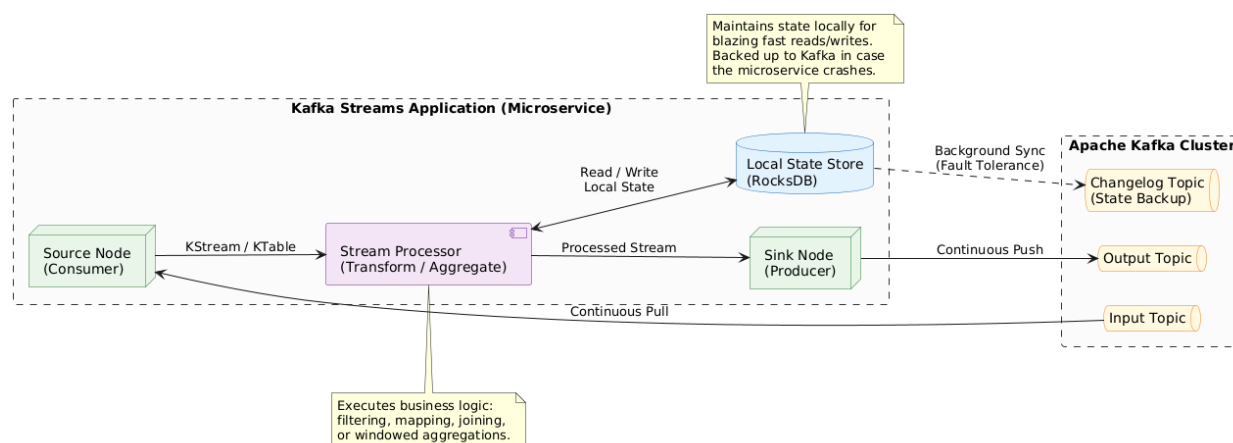


Figure 6.9: Kafka Streams Overview

6.3.2 Apache Flink: The Heavyweight Distributed Engine

While Kafka Streams is a library embedded in your application, **Apache Flink** is a massive, standalone, distributed computing cluster. It is the gold standard for highly complex, enterprise-grade event processing.

Key Features:

- **True Real-Time Streaming:** Unlike older systems (like Apache Spark Streaming) that simulated streaming by processing “micro-batches” every few seconds, Flink processes events individually the microsecond they arrive, offering exceptionally low latency.
- **JobManager & TaskManagers:** Flink uses a master-worker architecture. The **JobManager** orchestrates the execution graph, while numerous **TaskManagers** distribute the actual computational load across the cluster.
- **Exactly-Once Processing Guarantees:** Through a mechanism called distributed checkpointing, Flink frequently saves the entire state of the computation to durable storage (like AWS S3). If a node physically crashes, Flink can restart the job from the exact millisecond it failed, ensuring no data is processed twice and no data is lost.
- **Ideal Use Case:** Flink is required for heavy, stateful computations running over massive datasets—such as real-time financial fraud detection algorithms, dynamic ride-share pricing computations, or correlating millions of IoT telemetry streams across multiple geographic regions.

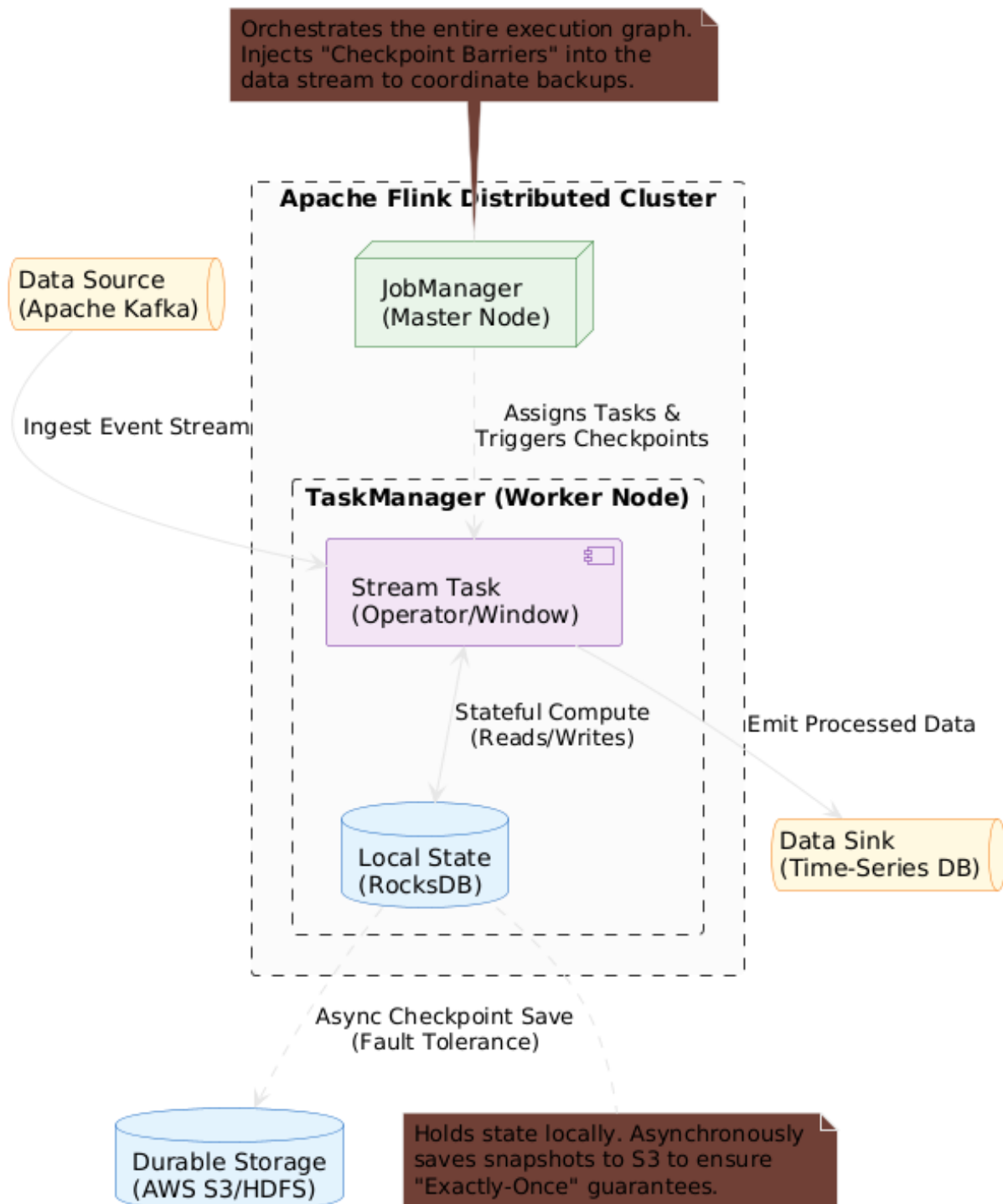


Figure 6.10: Apache Flink Overview

Key Concepts:

1. **Master-Worker Architecture:** Unlike Kafka Streams, Flink requires a dedicated cluster. The JobManager acts as the brain, scheduling tasks and monitoring health, while the TaskManagers are

the muscle, executing the actual stream processing.

2. **Decoupled Storage:** Flink pulls data from a source (like Kafka) but can push it anywhere. It is not locked into the Kafka ecosystem.
3. **The Checkpointing Mechanism:** This is Flink's superpower. The JobManager periodically tells the TaskManager to take a snapshot of its Local State. The TaskManager asynchronously writes this snapshot to a highly durable system like AWS S3 or HDFS (represented by the dashed arrow at the bottom). If the server physically burns down, a new TaskManager spins up, downloads the latest snapshot from S3, rewinds the Kafka offset to match, and resumes processing without ever dropping or duplicating an event.

Summary: Kafka Streams vs. Flink

- Choose **Kafka Streams** if you want to build simple, event-driven microservices without adding new infrastructure complexity to your stack.
- Choose **Apache Flink** if you need a dedicated, heavy-duty analytics engine capable of managing massive state, complex windowing, and strict exactly-once guarantees across a distributed cluster.

Feature / Aspect	Kafka Streams	Apache Flink
Architectural Model	Embedded Library. It is a standard Java library included directly inside your own application or microservice.	Standalone Distributed Cluster. It requires its own dedicated infrastructure (Master/Worker architecture with JobManagers and TaskManagers).
Deployment Complexity	Very Low. Deploys exactly like any standard backend application (e.g., a simple Docker container running your code).	High. Requires managing, scaling, and tuning a separate distributed compute cluster.
Ecosystem Lock-in	High. Strictly tied to Apache Kafka. It can only read from Kafka topics and write to Kafka topics.	Low. Highly agnostic. Can ingest from Kafka, RabbitMQ, Kinesis, or raw sockets, and sink to databases, S3, or message brokers.
State Management	RocksDB + Kafka. Keeps local state in RocksDB and backs it up to hidden "Changelog" topics inside the Kafka cluster.	RocksDB + Durable Storage. Keeps local state in RocksDB but backs it up asynchronously to durable storage like AWS S3 or HDFS via checkpoints.
Latency & Throughput	Milliseconds. Extremely fast, but network hops between the microservice and the Kafka broker for state backups can add slight latency.	Sub-milliseconds. True event-at-a-time processing. State is managed entirely within the Flink cluster for absolute minimum latency.
Exactly-Once Guarantees	Yes (Kafka-to-Kafka). Guarantees exactly-once processing, but <i>only</i> within the boundaries of the Kafka ecosystem.	Yes (End-to-End). Uses a sophisticated distributed checkpointing algorithm to guarantee exactly-once processing across external databases and sinks.
Language Support	Java & Scala. (Though there are community wrappers for other languages, native support is JVM-based).	Java, Scala, Python (PyFlink), & SQL. Flink SQL is extremely powerful for running real-time queries on live streams.

Feature / Aspect	Kafka Streams	Apache Flink
Ideal Use Case	Lightweight real-time ETL, building event-driven microservices, filtering/transforming data before it hits a database.	Heavyweight enterprise analytics, complex event processing (CEP), fraud detection, and multi-region telemetry aggregation.

6.3.3 REST (Representational State Transfer)

REST is the ubiquitous architectural style for web APIs. It relies on standard HTTP/1.1 methods (GET, POST, PUT, DELETE) and typically uses JSON for data exchange.

Pros: Highly readable, stateless, and universally supported by browsers and mobile clients.

Cons: JSON payloads are text-heavy and uncompressed. HTTP/1.1 requires opening new TCP connections for concurrent requests, creating overhead.

6.3.4 gRPC & Protocol Buffers

Developed by Google, gRPC is a high-performance Remote Procedure Call framework. Instead of JSON, it uses **Protocol Buffers (Protobuf)**—a strictly typed, binary serialization format.

Pros: Protobuf payloads are significantly smaller and faster to serialize/deserialize than JSON. gRPC runs on HTTP/2, allowing for multiplexing (sending multiple requests concurrently over a single TCP connection) and bi-directional streaming.

Cons: The binary data is not human-readable, making debugging harder without specific tools. It is generally not supported directly by web browsers without a translation proxy.

Use Case: gRPC is the gold standard for internal, server-to-server microservice communication where high throughput and low latency are critical.

6.3.5 gRPC vs. REST

Feature	REST (Representational State Transfer)	gRPC (gRPC Remote Procedure Calls)
Transport Protocol	HTTP/1.1 (Typically)	HTTP/2 (Strictly required)
Payload Format	JSON (Mostly), XML, HTML	Protocol Buffers (Protobuf)
Data Type	Text-based (Human-readable)	Binary (Machine-readable)
Payload Size	Larger (Uncompressed text, repeated keys)	Extremely Small (Compressed binary, no keys)
API Contract	Loose / Optional (e.g., OpenAPI/Swagger)	Strict / Mandatory (Enforced via <code>.proto</code> files)
Code Generation	Requires third-party tools (e.g., Swagger Codegen)	Native / Built-in (<code>protoc</code> generates client/server stubs)
Communication Model	Unary (Request → Response only)	Unary, Server Streaming, Client Streaming, Bidirectional Streaming
Browser Support	Universal (Browsers natively speak HTTP/1.1 and JSON)	Poor (Requires a proxy like gRPC-Web to bridge to HTTP/1.1 or HTTP/2 frames browsers can read)

Feature	REST (Representational State Transfer)	gRPC (gRPC Remote Procedure Calls)
Latency & Speed	Slower (Due to text parsing and Head-of-Line blocking)	Blazing Fast (Due to binary deserialization and HTTP/2 multiplexing)
Caching	Excellent (Native support using HTTP GET verbs)	Poor (Uses HTTP POST for everything, bypassing standard caching)
Best Used For	Public-facing APIs, external integrations, direct frontend-to-backend web traffic	Internal microservice-to-microservice communication, IoT, polyglot environments

6.4 Real-Time Communication Patterns

When the server needs to push updates to the client (such as live chat, stock tickers, or a live map of moving assets), traditional Request-Reply HTTP falls short. Engineers must choose from several real-time patterns:

1. **Short Polling:** The client repeatedly asks the server, “Are there updates?” every few seconds.
 - *Drawback:* Highly inefficient. 99% of requests return empty, wasting server CPU and network bandwidth.
2. **Long Polling:** The client asks the server for updates. If there are none, the server *holds the connection open* until new data arrives, sends it, and then closes the connection. The client immediately reconnects.
 - *Drawback:* Better than short polling, but constantly opening and closing connections still incurs TLS handshake overhead.
3. **Server-Sent Events (SSE):** A unidirectional protocol over standard HTTP where the client opens a single, long-lived connection, and the server continuously streams text data down to the client.
 - *Use Case:* Excellent for live news feeds, social media timelines, or one-way notification systems.
4. **WebSockets:** A distinct protocol (ws://) that establishes a persistent, full-duplex, bi-directional connection over a single TCP socket.
 - *Use Case:* Essential for applications requiring low-latency, two-way communication, such as multiplayer gaming, collaborative document editing, or live chat platforms.

6.4.1 Comparison of Real-Time Communication Patterns

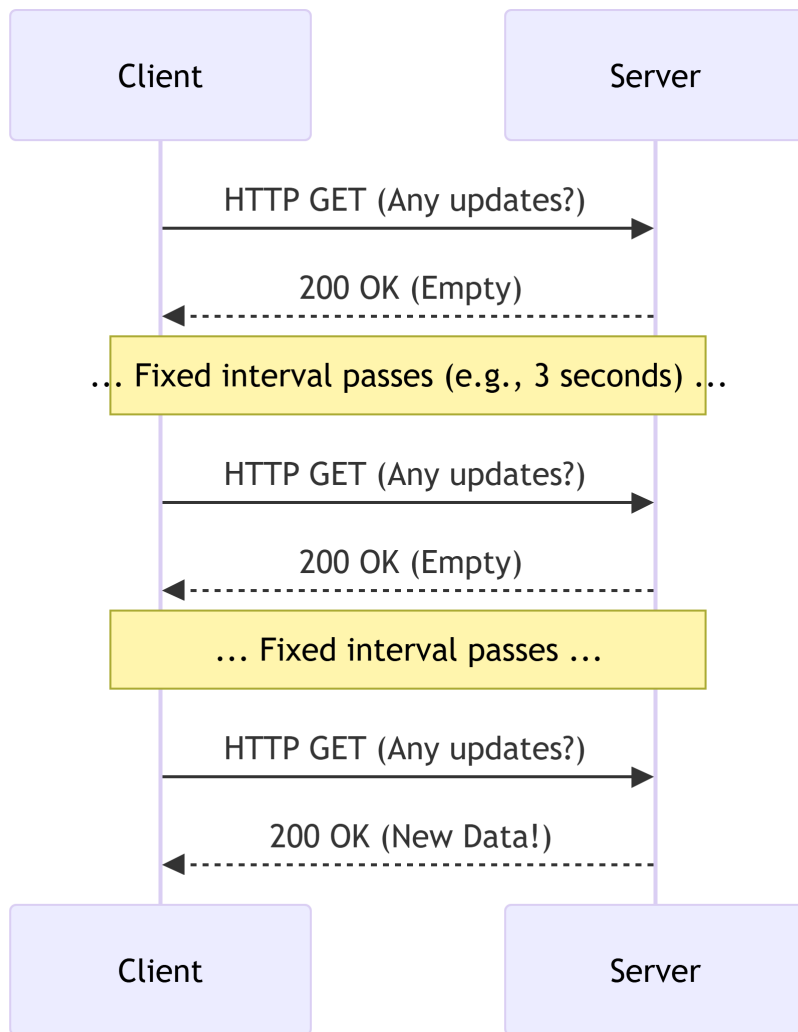


Figure 6.11: Short Polling

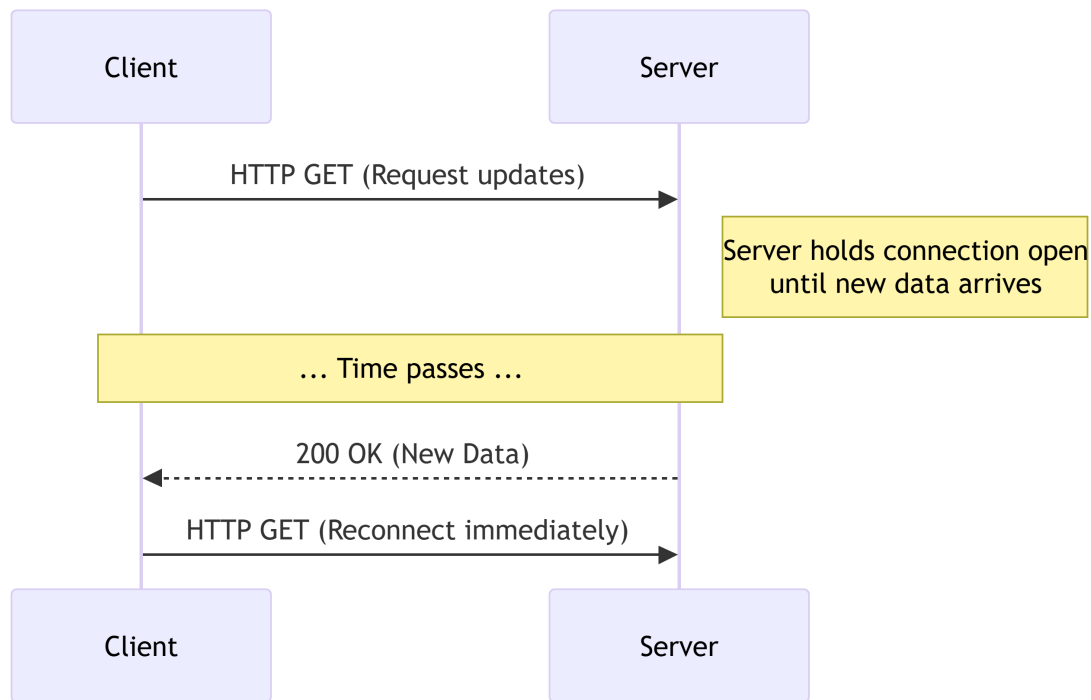


Figure 6.12: Long Polling

Communication Pattern	Directionality & Protocol	Network Overhead & Latency	Detailed Real-World Use Case
Short Polling	Unidirectional Pull Standard HTTP/1.1	High Overhead Constant TLS handshakes, high CPU usage for empty responses.	Smart Mirror Weather Dashboard: The client fetches the current weather every 15 minutes. The data changes slowly, and real-time millisecond accuracy is irrelevant. Opening a persistent WebSocket just to check the temperature every few minutes would be a massive waste of server RAM, making basic polling the most efficient choice.

Communication Pattern	Directionality & Protocol	Network Overhead & Latency	Detailed Real-World Use Case
Long Polling	Unidirectional Push Standard HTTP/1.1	Moderate Overhead Eliminates empty responses, but still requires a new TLS handshake after every payload.	Legacy Web Email Clients: A browser tab waits for a new email. If no email arrives, the server holds the connection open. When an email drops, the server sends it and closes the connection. The browser immediately opens a new one. It prevents the server from being spammed with “Any new mail?” requests, while working perfectly over strict corporate firewalls that might block WebSocket traffic.
Server-Sent Events (SSE)	Unidirectional Push Persistent HTTP/1.1 or HTTP/2	Low Overhead A single TLS handshake. The connection stays open indefinitely for text streaming.	Live Stock Market Ticker / Sports Scores: A web application displaying the live price of a stock. The client never needs to send complex data <i>back</i> to the server; it only needs to listen. SSE is chosen over WebSockets here because it runs over standard HTTP, has built-in automatic reconnection logic, and requires significantly less backend infrastructure to scale.

Communication Pattern	Directionality & Protocol	Network Overhead & Latency	Detailed Real-World Use Case
WebSockets	Bidirectional (Full-Duplex) WS/WSS Protocol	Very Low Overhead Once upgraded, data frames are sent back and forth with almost zero header overhead.	Collaborative Design Tools (e.g., Figma): Multiple users are moving their cursors and editing a canvas simultaneously. The client must constantly stream X/Y mouse coordinates <i>to</i> the server, while the server simultaneously streams the movements of other users <i>back</i> to the client. This requires a persistent, two-way open pipe where neither party has to wait their turn to speak.

When designing a system, you can use this simple decision tree to pick the right tool:

1. **Does the client need to send a massive amount of high-frequency data *to* the server?**
 - If **Yes** → Use **WebSockets**.
2. **If No, does the server need to push continuous, live updates *to* the client?**
 - If **Yes** → Use **Server-Sent Events (SSE)**.
3. **If No, but updates happen randomly and must be delivered quickly when they do?**
 - Use **Long Polling** (or Webhooks, if server-to-server).
4. **If updates are rare, predictable, or non-urgent?**
 - Use standard **Short Polling**.

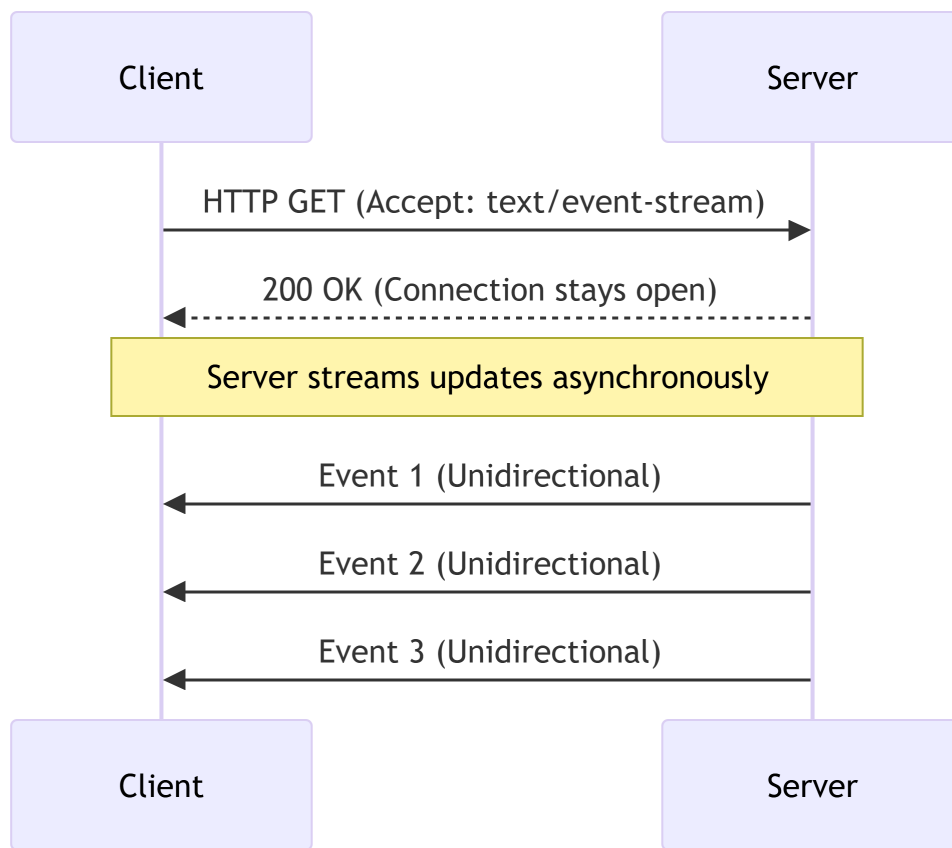


Figure 6.13: Server-Sent Events

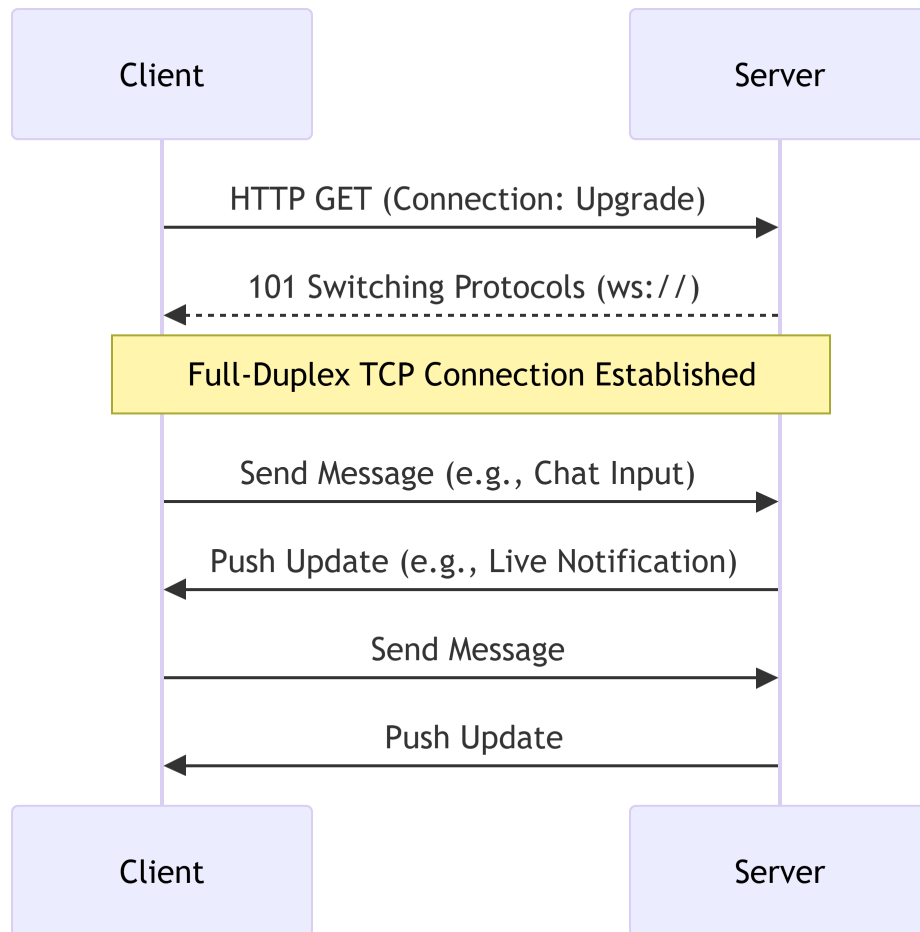


Figure 6.14: Websockets

Chapter 7

Security, Monitoring, and Reliability

Deploying a complex distributed system without monitoring is like flying a commercial airliner blindfolded. If a server crashes, a database slows down, or a microservice starts throwing errors, you have no way of knowing until your users start complaining.

To achieve **Observability**—the ability to understand the internal state of a system entirely from its external outputs—architects rely on the three pillars of telemetry: **Metrics**, **Logs**, and **Traces**.

7.1 Observability Architecture

Understanding the Observability Flow

This diagram illustrates how modern distributed systems handle telemetry using two distinct, parallel pipelines: the **Logging Stack** (for deep context) and the **Metrics Stack** (for real-time health).

The architecture highlights a fundamental difference in how data is collected: Logs are **pushed**, while Metrics are **pulled**.

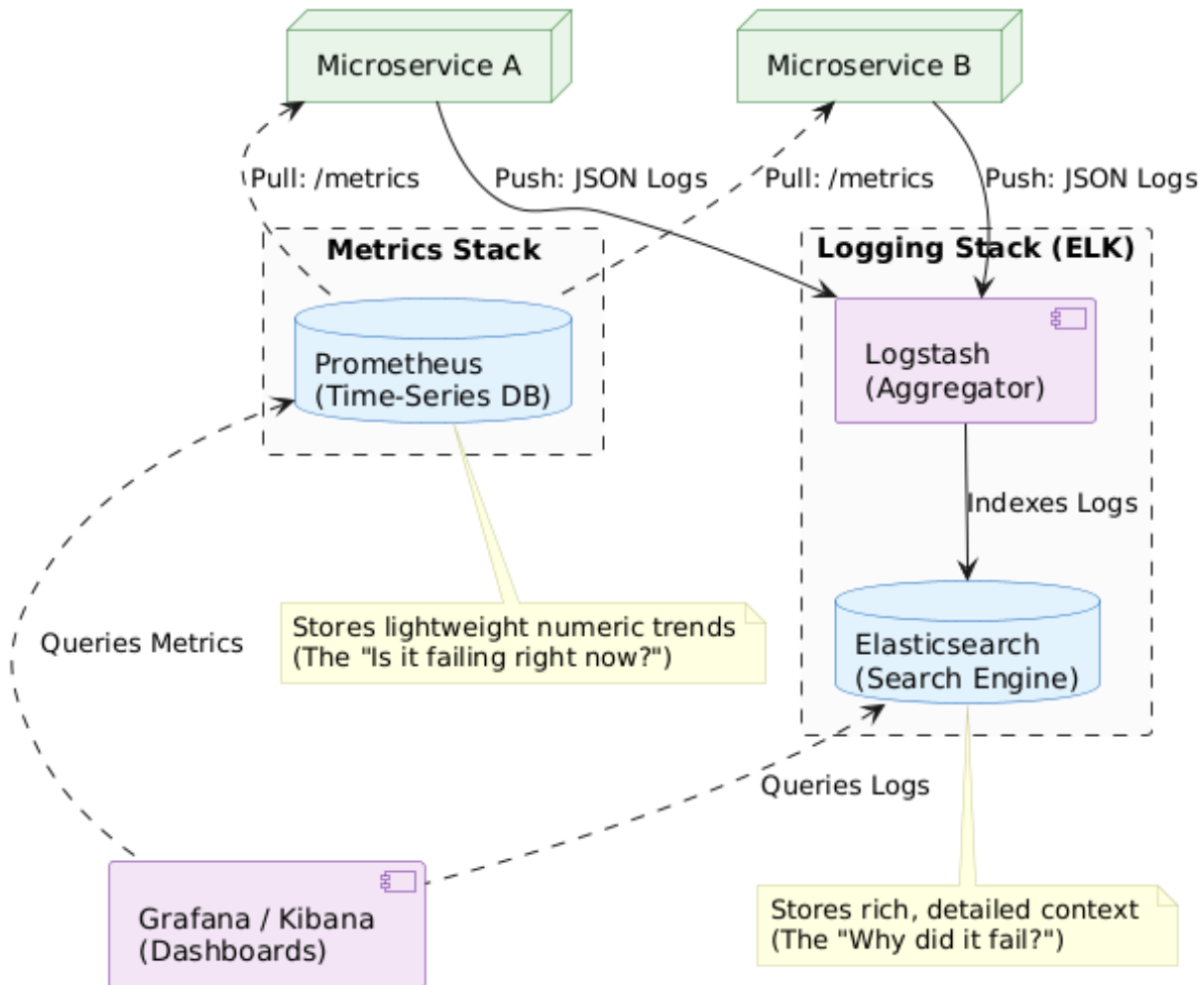


Figure 7.1: Observability Architecture Overview

1. The Data Generators (Top Layer) At the top of the architecture are the microservices (Microservice A and Microservice B). These are the active applications serving user traffic. As they execute code, they simultaneously generate two types of telemetry data: detailed event logs and lightweight health numbers.

2. The Logging Pipeline (The Push Model)

- **Logstash (The Aggregator):** When an event occurs (e.g., an error or a user login), the microservices actively *push* a structured JSON log to Logstash. Logstash acts as a buffer and processing pipeline, ensuring the logs are correctly formatted before passing them downstream.
- **Elasticsearch (The Search Engine):** Logstash pushes the formatted logs into Elasticsearch. Elasticsearch is a heavy-duty database optimized for indexing text. It stores the rich, detailed context needed to answer the question, “*Why did the system fail?*”

3. The Metrics Pipeline (The Pull Model)

- **Prometheus (The Time-Series Database):** Unlike the logging stack, Prometheus does not wait for data to be sent to it. Instead, the microservices expose a lightweight HTTP endpoint (usually `/metrics`). Prometheus operates on a timer, actively reaching out to *pull* (scrape) the current numeric values (like CPU usage, error counts, or active memory) from those endpoints. It stores these numbers as time-series data to answer the question, “*Is the system failing right now?*”

4. The Visualization Layer (Bottom Layer) At the bottom sits the presentation layer (typically **Grafana** for metrics and **Kibana** for logs). While the underlying databases (Elasticsearch and Prometheus) are completely separate, the dashboard layer unifies them for the engineering team. An engineer can look at Grafana to see a sudden spike in a Prometheus metric (like HTTP 500 errors), and immediately pivot to Kibana to query the Elasticsearch logs for the exact JSON stack traces that caused the spike.

7.2 Telemetry

Telemetry (from the Greek *tele* meaning “remote”, and *metron* meaning “measure”) is the automated process of collecting data from remote systems and transmitting it to centralized equipment for monitoring and analysis.

In software architecture, telemetry dictates *how* data moves from your microservices to your monitoring dashboards. There are two primary telemetry models:

- **Push Model:** The application actively sends its data to a centralized collector (e.g., a microservice pushes a log entry to Logstash every time an error occurs).
- **Pull Model:** The application exposes a localized endpoint (like `/metrics`), and a centralized monitoring server periodically connects to the application to scrape and download the data (e.g., Prometheus scraping a server every 10 seconds).

7.3 The Pillars of Observability and MELT

Historically, the software industry defined observability using the “**Three Pillars**”: Metrics, Logs, and Traces.

However, as distributed architectures (like microservices and serverless functions) grew exponentially more complex, a fourth data type became necessary to bridge the gap between high-level system health and low-level code execution.

This evolution led to the modern standard of telemetry known as **MELT**: **M**etrics, **E**vents, **L**ogs, and **T**rades.

True observability is not just about collecting these four data types; it is about correlating them so that an engineer can seamlessly navigate from a high-level spike on a dashboard down to the exact line of failing code.

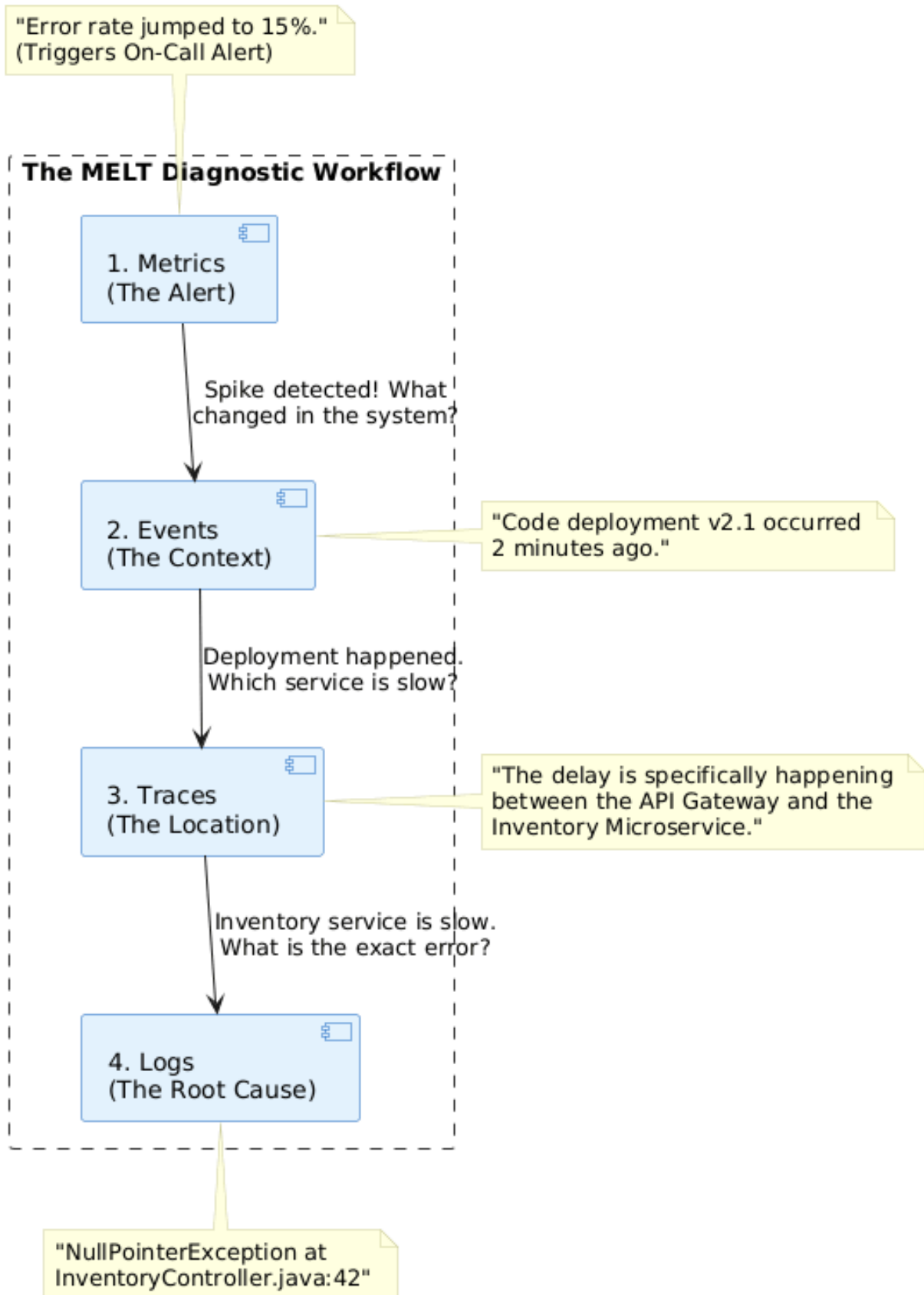


Figure 7.2: MELT Workflow

7.3.1 Metrics

Metrics are aggregated, numeric measurements of data collected over intervals of time (time-series data). They do not contain details about specific users or transactions. Instead, they provide a macro-level view of system health.

- **Characteristics:** Extremely lightweight, cheap to store, and highly compressible.
- **Examples:** CPU usage is at 92%, memory consumption is 4GB, or HTTP 500 errors jumped to 45 per minute.
- **Purpose: Alerting.** Metrics are the heart rate monitor of your system. You use metrics to trigger automated alerts (e.g., paging an engineer at 2:00 AM because the error rate crossed a critical threshold).

Metrics tell you *how the system is performing overall*. Metrics are numeric representations of data measured over intervals of time (Time-Series Data). They are highly compressed and cheap to store.

Types of Metrics

- **Counters:** Numbers that only ever go up (e.g., Total HTTP requests received, Total errors thrown).
- **Gauges:** Numbers that can go up and down (e.g., Current CPU usage, Current active memory, Current queue size).
- **Histograms:** Distributions of values over time, used to calculate averages and percentiles (e.g., 95th percentile of HTTP response times).

The Metrics Ecosystem (Prometheus & Grafana)

- **Prometheus:** A powerful time-series database specifically built for metrics. It operates on a **Pull Model**. Your microservices expose a `/metrics` HTTP endpoint, and Prometheus continuously scrapes those endpoints to record the current state of the counters and gauges.
- **Grafana:** The visual dashboard that connects to Prometheus. It takes the raw time-series numbers and turns them into the beautiful, real-time line charts, dials, and heatmaps that engineers monitor on large screens in operations centers.

7.3.2 Events

An event is a discrete, high-level record of a significant action that occurred at a specific moment in time. While a metric tells you *how many* times something happened in the last minute, an event tells you *that a specific thing just happened*.

- **Characteristics:** More detailed than a metric, but less granular than a log. They often carry key-value pairs (attributes) for filtering.
- **Examples:** “User 891 upgraded to the Pro Tier”, “Deployment v2.4.1 completed successfully”, or “Kubernetes Pod 3 restarted.”
- **Purpose: Context.** Events help correlate system anomalies with real-world actions. If a metric shows a sudden spike in CPU usage, an event showing that a “New version deployment” occurred at the exact same second provides instant context.

7.3.3 Logs

Logs are discrete, immutable records of specific events that happened at a specific point in time within the system. They provide the highly detailed context needed to debug *why* a failure occurred.

- **Characteristics:** High volume, expensive to store, and rich in deep technical context.
- **Examples:**

```
{"level": "error", "message": "Failed to connect to PostgreSQL on port 5432: Connection timed out", "module": "auth_service"}
```

- **Purpose: Debugging.** Once a metric alerts you to a problem, and an event gives you the context, you dive into the logs to read the exact stack traces and variable states to fix the broken code.

7.3.3.1 Structured vs. Unstructured Logging

- **Unstructured Logging:** The traditional method of writing raw text strings.
 - *Example:* [2026-04-26 19:36:40] ERROR: Database connection failed for user 891.
 - *Problem:* Humans can read this easily, but searching through millions of lines of unstructured text using code is incredibly slow and inefficient.
- **Structured Logging:** Writing logs in a machine-readable format, almost universally **JSON**.
 - *Example:* {"timestamp": "2026-04-26T19:36:40Z", "level": "error", "event": "db_timeout", "user_id": 891}
 - *Advantage:* A centralized logging engine can instantly index, filter, and query JSON keys across billions of records.

7.3.3.2 Log Levels

To prevent storage systems from being overwhelmed, developers categorize logs by severity:

- **DEBUG:** Deep, granular details useful only during development (e.g., variable states).
- **INFO:** Normal system operations (e.g., “Server started on port 8080”, “User logged in”).
- **WARN:** Something unexpected happened, but the system recovered (e.g., “API rate limit nearing capacity”).
- **ERROR:** A localized failure occurred, but the broader application is still running (e.g., “Failed to process payment”).
- **FATAL:** A catastrophic failure requiring immediate shutdown (e.g., “Cannot connect to core database”).

7.3.3.3 Centralized Logging (The ELK Stack)

In a microservices architecture, you cannot SSH into 50 different servers to read 50 different text files. Logs must be centralized. The industry standard is the **ELK Stack**:

1. **Elasticsearch:** The distributed search and analytics engine that stores the structured JSON logs.
2. **Logstash:** The data processing pipeline that ingests logs from all microservices, formats them, and pushes them into Elasticsearch.
3. **Kibana:** The web-based UI used by engineers to search the logs and build visual dashboards.

7.3.4 Traces

In a monolithic application, logs are usually enough. But in a microservices architecture, a single user request might travel through an API Gateway, a billing service, an inventory service, and a database before returning a response. If the request fails or takes 5 seconds to load, logs alone cannot easily tell you *which* service caused the bottleneck.

- **Characteristics:** Traces track the entire end-to-end journey of a single request across distributed boundaries. A Trace is made up of multiple **Spans** (one span for each service the request touches).
- **Examples:** User clicks “Checkout” → API Gateway (10ms) → Auth Service (15ms) → Inventory Service (4000ms - *Bottleneck found!*) → Database (5ms).

- **Purpose: Localization.** Traces isolate exactly where latency or errors are originating within a massive web of microservices.

7.4 Distributed Tracing and Jaeger

In a traditional monolithic application, debugging a slow user request is straightforward: a single server executes the code sequentially, and all the logs are written to a single file.

However, in a distributed microservices architecture, a single user action (e.g., a student submitting a registration for a hackathon) might pass through an API Gateway, an Authentication Service, an Event Tracking Service, and a Database.

If that registration takes 8 seconds to process, where is the bottleneck?

- Is the Auth Service struggling to validate the token?
- Is the Event Service waiting on a slow database query?
- Is there network latency between the Gateway and the backend?

Logs and metrics cannot easily answer this because they lack **correlation**. You might see a log showing the Auth Service took 10ms, and another showing the Database took 7000ms, but you cannot easily prove they belong to the *same* user request.

This is the problem **Distributed Tracing** solves. It allows you to track the complete, end-to-end journey of a single request as it crosses multiple network and service boundaries.

7.4.1 The Anatomy of a Trace

To understand tracing, you must understand its three core vocabulary terms:

1. The Trace

A **Trace** represents the complete execution path of a single transaction or request through your entire system. It is a tree-like structure made up of individual steps.

2. The Span

A **Span** is a single unit of work within a Trace. Every time a request crosses a boundary (e.g., calling a new microservice, executing a database query, or hitting a cache), a new Span is created.

A Span contains:

- An **Operation Name** (e.g., HTTP GET /register).
- A **Start Time** and **End Time** (which calculates the exact duration of that specific step).
- **Tags / Attributes** (e.g., `http.status_code=200`, `user_id=105`).

3. Context Propagation (The Secret Sauce)

How do 5 different microservices know they are working on the same user request? **Context Propagation**.

When the API Gateway receives a brand new request, it generates a unique, globally identifiable **Trace ID**. As the Gateway makes HTTP or gRPC calls to downstream services, it injects this Trace ID directly into the network headers (often as `X-B3-TraceId` or the modern W3C `traceparent` header).

Every downstream service reads that header, attaches its own work to that Trace ID, and passes it along to the next service.

7.4.2 Visualizing the Trace Context

The following is an example illustrating how a single Trace ID propagates through an Event Registration system, creating a Parent-Child hierarchy of Spans.

Understanding the Trace Context Propagation Flow

This sequence diagram illustrates the precise mechanics of **Distributed Context Propagation**. It visualizes exactly how a single user request—in this case, an event registration—is tracked across multiple independent network boundaries without losing its identity.

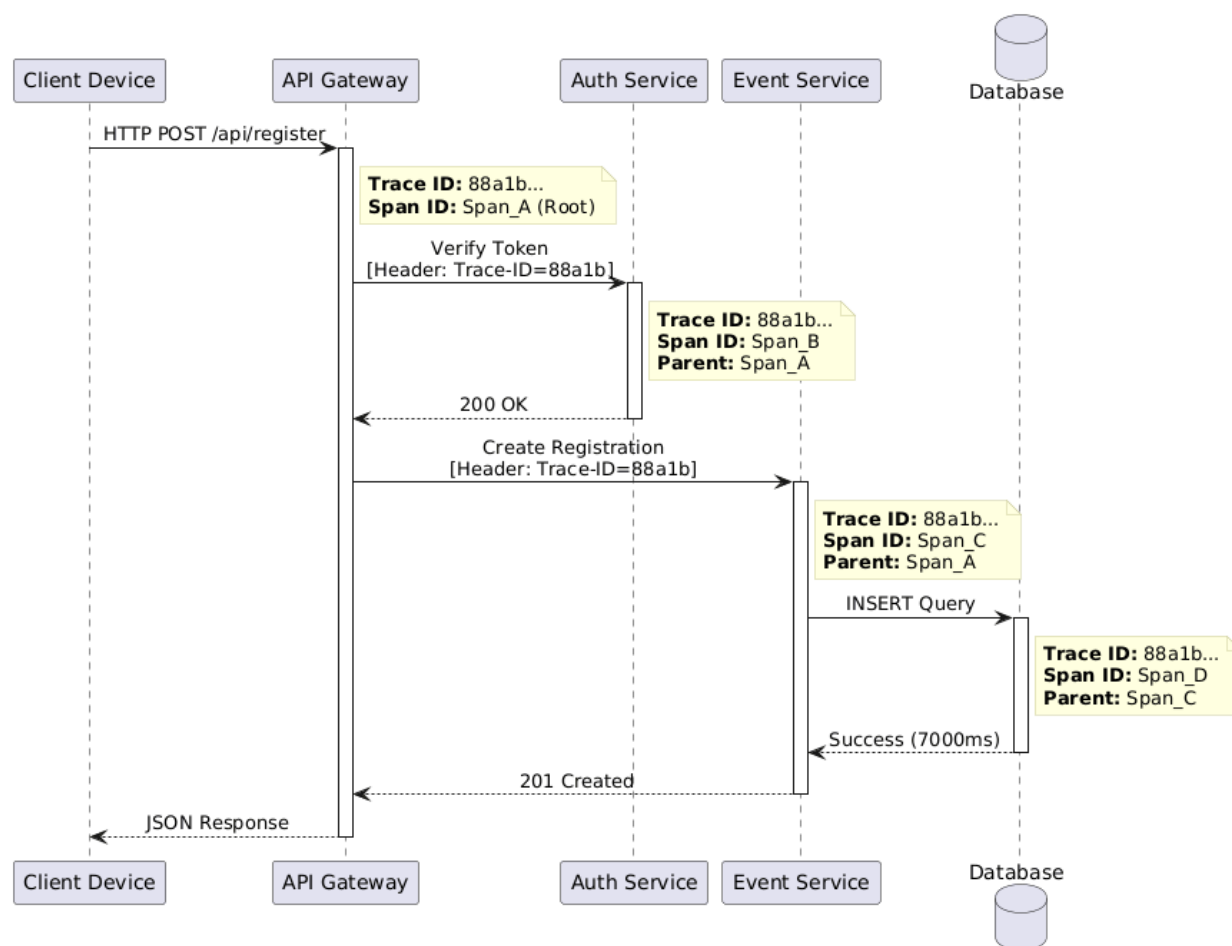


Figure 7.3: Trace Context Propagation Flow

1. The Ingestion and Trace Initiation

- **Action:** The Client Device initiates the transaction by sending an HTTP POST /api/register request to the API Gateway.
- **Tracing Logic:** Because this is the very first time the request is entering the system, the Gateway acts as the initiator. It generates a globally unique **Trace ID** (88a1b...). It also generates the very first span, **Span_A**, which is designated as the “Root Span.” For the rest of this transaction’s life, that Trace ID will never change.

2. The Authentication Step

- **Action:** Before processing the registration, the Gateway must ensure the user is authorized. It makes a synchronous network call to the **Auth Service**.
- **Tracing Logic (The Secret Sauce):** The Gateway injects the Trace ID directly into the HTTP headers of the outbound request ([Header: Trace-ID=88a1b]). The **Auth Service** reads this header, realizing it is part of an ongoing transaction. It generates its own span (**Span_B**) and explicitly declares **Span_A** as its parent. After a quick validation, it returns a 200 OK to the Gateway.

3. Executing the Business Logic

- **Action:** With authentication complete, the Gateway forwards the actual registration payload to the **Event Service**.
- **Tracing Logic:** Once again, the Gateway passes the Trace ID in the headers. The **Event Service** receives it, creates **Span_C**, and also links it back to the Gateway by declaring **Span_A** as its parent.

4. The Database Bottleneck

- **Action:** To persist the registration, the **Event Service** must execute an INSERT query against the **Database**.
- **Tracing Logic:** The context propagates one layer deeper. The Database operation generates **Span_D**. However, because the Database was called by the Event Service (not the Gateway), its parent is marked as **Span_C**.
- **The Critical Insight:** Notice the (7000ms) delay on the Database return trip. This signifies a massive performance bottleneck.

5. The Return Journey

- **Action:** The Database finally finishes its query, allowing the **Event Service** to return a 201 Created status to the Gateway, which in turn delivers the final JSON response back to the **Client Device**.
- **Tracing Logic:** As each service finishes its work, its respective Span is closed (given an “End Time”) and asynchronously fired off to the tracing backend (like Jaeger) for storage.

How This Looks in a Tracing Dashboard

When an engineer searches for Trace ID 88a1b... in a tool like Jaeger, the backend reconstructs this parent-child hierarchy into a visual “Flame Graph.”

By looking at the tree structure: * **Span_A** (Root) * **Span_B** (Auth) * **Span_C** (Event) * **Span_D** (Database)

The engineer would instantly see that while the total request took over 7 seconds, the API Gateway, Auth Service, and Event Service executed instantly. The visual timeline would clearly highlight that **Span_D** (the Database INSERT query) consumed 7000ms, isolating the exact root cause of the system latency without the engineer needing to manually stitch together separate log files.

7.4.3 Introduction to Jaeger

Jaeger (pronounced YAY-ger, German for “Hunter”) is an open-source distributed tracing platform originally created by Uber Technologies and now maintained by the Cloud Native Computing Foundation (CNCF).

Jaeger provides the backend infrastructure to collect, store, and visualize the millions of spans generated by your microservices.

*(Note: While Jaeger provides the storage and UI, modern applications usually rely on **OpenTelemetry (OTel)** SDKs to actually generate the spans inside the code.)*

7.4.3.1 Jaeger Backend Architecture

To prevent tracing from slowing down your actual application, Jaeger uses a decoupled, asynchronous architecture.

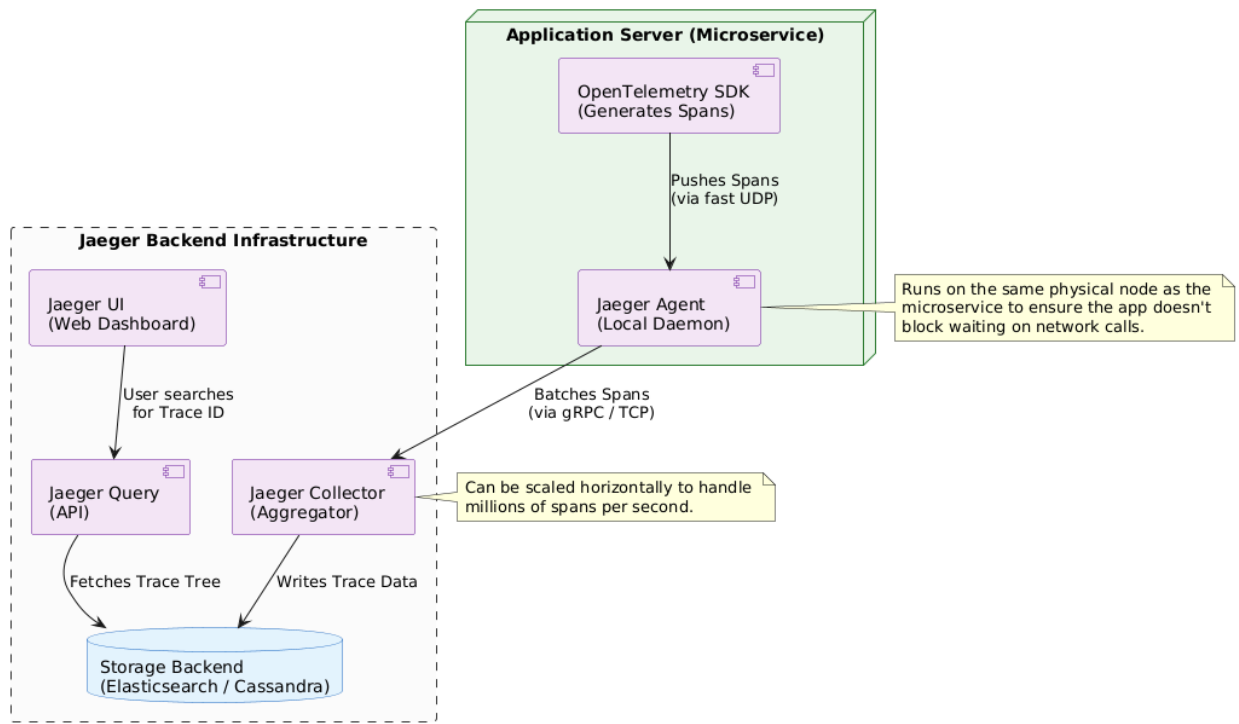


Figure 7.4: Jaeger Backend Architecture

1. **Jaeger Client (OpenTelemetry SDK):** Embedded in your microservice. It creates the spans, but instead of writing them directly to a database, it fires them over a fast UDP connection to a local Agent.
2. **Jaeger Agent:** A lightweight daemon that runs on the same server/node as your application. It listens for UDP packets, batches them up, and securely forwards them to the central Collector.
3. **Jaeger Collector:** The central processing engine. It receives batches of spans from thousands of Agents, validates them, and writes them to a database.
4. **Storage:** The database. Because traces generate a massive amount of data, Jaeger is typically backed by highly scalable databases like **Elasticsearch** or **Cassandra**.
5. **Jaeger UI / Query:** The beautiful web dashboard where engineers search for specific Trace IDs and visualize the latency bottlenecks.

Chapter 8

Case Studies

Note on Estimations and Approximations

Certain numerical examples (like Twitter’s DAU, QPS, storage, and fan-out calculations) and case study details (e.g., Discord’s sharding, Netflix’s Cassandra configs) are simplified approximations designed for pedagogical clarity. They illustrate core system design concepts rather than exact historical production metrics, which vary over time and aren’t always publicly documented. Always cross-reference with primary sources for real-world implementations.

8.1 The Twitter “Fail Whale”

From Monolithic Fragility to Distributed Resilience

In the late 2000s, Twitter was synonymous with the “Fail Whale”—an illustration of a whale being lifted by birds that appeared whenever the service crashed. For an SRE or System Designer, the Fail Whale wasn’t just a cute graphic; it was a symptom of a **write-heavy monolith** reaching its breaking point.

The Initial Architecture: The Ruby on Rails Monolith

Originally, Twitter was built as a traditional **CRUD** (Create, Read, Update, Delete) application using Ruby on Rails and a single, massive MySQL database.

- **The Workflow:** When a user posted a tweet, it was written to the MySQL database. When followers viewed their timeline, the system performed a massive “JOIN” query across the database to find all tweets from people they followed.
- **The Bottleneck:** In a fan-out scenario (e.g., a celebrity with millions of followers), the database had to perform millions of operations simultaneously to refresh timelines.

The Scaling Pivot: Fan-out and Message Queues

To kill the Fail Whale, Twitter moved away from “pulling” data on demand to “pushing” data into caches.

The “Push” Model (Fan-out):

Instead of querying the database when you log in, Twitter started doing the work at **write-time**.

1. **User A** posts a tweet.
2. The **Fan-out Service** identifies all followers of User A.
3. The tweet is injected into the **Timeline Cache** (using Redis) for every single follower.
4. **Result:** When a follower logs in, their timeline is already pre-computed. Reading is now $O(1)$.

SRE Lessons: The “Celebrity” Edge Case

The push model worked for everyone except celebrities. If an account with 100 million followers tweets, pushing that tweet to 100 million caches causes a massive “thundering herd” problem and latency spikes.

The Hybrid Solution:

- **For regular users:** Use the **Push** model (Fan-out to caches).
- **For celebrities:** Revert to a **Pull** model. The tweet is only stored in one place, and followers’ systems “pull” it and merge it into their timeline on the fly.

The Twitter case study teaches three core principles of **Architecting Scalability**:

1. **Decouple Writes from Reads:** Moving the heavy lifting to the background (Fan-out) saved the system.
2. **Know Your Data Access Patterns:** Twitter is a 100:1 read-to-write ratio system; optimizing for reads was the correct trade-off.
3. **State Management is Hard:** Moving from a single MySQL instance to a distributed Redis mesh required solving complex consistency issues (the “Else-Consistency” part of PACELC).

8.2 Netflix: Global State Persistence with Apache Cassandra

Netflix operates at a scale where a single “Source of Truth” database is impossible due to speed-of-light latency. They utilize **Apache Cassandra** to manage user state (like movie bookmarks) across multiple global regions.

The Problem: If a user in Mumbai pauses a show and immediately tries to resume it on a different device while traveling, the “resume point” must be available globally without causing a 500ms round-trip delay to a central US server.

The Architecture: Netflix uses a **Masterless (Peer-to-Peer)** architecture. Every node in the Cassandra cluster can accept both reads and writes.

The Mechanism (Eventual Consistency):

Local Write: When the user hits “pause” in Mumbai, the local AWS Mumbai node accepts the write instantly.

Asynchronous Replication: The Mumbai node then sends this update to clusters in London and Virginia in the background.

NFR Justification (PACELC): This is a **PA/EL** system. It prioritizes **Availability** during a partition and **Low Latency** during normal operations. The trade-off is that a user *might* see a stale resume point if they switch devices faster than the replication time.

8.3 Amazon’s \$100 Million Shopping Cart (Dynamo)

This case study is the primary real-world example of prioritizing **Availability** over **Consistency**.

The Problem: Traditional RDBMS uses strict locking to ensure consistency. During a network partition (P), if the database cannot confirm a write across all replicas, it “locks” and rejects the write to prevent data corruption. For Amazon, a locked database meant a customer couldn’t add items to a cart, leading to massive revenue loss.

The Solution (Dynamo): Amazon built **Dynamo**, a distributed key-value store that is **always writeable**.

Conflict Resolution: * If a network partition occurs, two different nodes might accept two different versions of the same shopping cart.

Instead of the system choosing one, it preserves **both**.

The “conflict” is resolved at checkout (on the read path) by merging the two versions of the cart.

SRE Principle: This is an example of **Eventual Consistency** where the system remains available (A) at the cost of temporary inconsistency (C).

8.4 Facebook: Haystack (Object Storage Efficiency)

Facebook’s photo storage is a classic study in overcoming the limitations of standard filesystems.

The Problem: Standard filesystems (like ext4) store metadata in inodes. To retrieve a single photo, the OS has to perform multiple disk seeks: one to find the directory, one to find the inode, and finally one to read the actual file. At Facebook’s scale, these “metadata seeks” became a massive bottleneck.

The Solution (Haystack): Facebook moved from file-level storage to a specialized **Object Storage** system.

Mechanism: Haystack packs thousands of small images into a few large “Volume” files (approx. 100 GB each).

It maintains a small, in-memory index that maps a `Photo_ID` to an offset within a large volume file.

Efficiency Gain: By keeping the index in RAM, the system reduces the disk activity to a **single seek**—directly fetching the image data from the large file.

8.5 Discord: Scaling the Message Store

Discord’s migration from Cassandra to ScyllaDB highlights the **Hot Partition** problem in sharding design.

The Problem: Discord originally sharded their message database by `Channel_ID`. While this worked for small groups, “Whale” servers (like the Fortnite or Minecraft official servers) had millions of users. All messages in those channels hit a single database partition on a single physical node, causing it to “melt down” while other nodes stayed idle.

The Solution: They migrated to **ScyllaDB** (a C++ rewrite of Cassandra) and redesigned their routing.

The Mechanism (Consistent Hashing & Virtual Nodes): They refined their **Sharding Key** to distribute load more evenly.

They implemented a “Data Service” layer written in Rust to coalesce multiple identical requests into a single database query, protecting the database from “thundering herd” spikes.

8.6 Dropbox’s “Magic Pocket”

Custom Storage Infrastructure at Exabyte Scale

In 2016, Dropbox completed a two-year project to migrate over 500 PB of user data from Amazon S3 to their own custom storage system, known as **Magic Pocket**. This was driven by the need to optimize for specific performance characteristics and reduce the massive costs associated with petabyte-scale cloud storage.

1. The Problem: The Financial and Technical “Ceiling” of S3

While AWS S3 provided excellent durability, it presented several challenges for Dropbox at their scale:

Cost Sustainability: At exabyte scale, the overhead of public cloud margins became financially unsustainable compared to the cost of raw hardware.

One-Size-Fits-All Latency: S3 is designed for a wide variety of use cases, whereas Dropbox had a very specific workload: **Write Once, Read Many (WORM)**.

Control over Hardware: Dropbox needed to optimize for **Shingled Magnetic Recording (SMR)** drives to increase density, which was not a choice available on standard cloud instances.

2. The Solution: Shingled Magnetic Recording (SMR)

To achieve the necessary storage density, Dropbox utilized SMR technology.

The Mechanism: Traditional hard drives write data in parallel tracks. SMR “overlaps” these tracks like shingles on a roof.

The Trade-off: While SMR increases density by up to 25%, it makes random writes extremely slow because writing to one track may “overwrite” the overlapping track, requiring a sequential rewrite of a block.

The Fit: Because Dropbox users typically upload a file once and then read it many times, the “slow write” penalty of SMR was an acceptable trade-off for the immense gain in storage density and cost efficiency.

Magic Pocket Architecture

The system was designed using a “Shared Nothing” architecture to ensure maximum parallelism.

Cells: The top-level unit of isolation. Each cell is a self-contained storage cluster capable of holding up to 100 PB.

Storage Nodes: Custom-built servers (the “Magic Pockets”) packed with SMR drives.

Metadata Layer: A separate, high-performance layer used to track where each block of data is stored across the cells, utilizing an in-memory index similar to **Facebook’s Haystack** to reduce disk seeks.

SRE and Reliability: Erasure Coding

To ensure “Five Nines” of durability without the 3x cost of standard replication, Magic Pocket uses **Erasure Coding**.

Mechanism: Data is split into k fragments and m parity fragments. The system can reconstruct the original data even if m disks or nodes fail.

Efficiency: This provides high durability with much lower storage overhead (e.g., 1.5x) compared to traditional 3x mirroring.

The “Magic Pocket” project teaches three core principles of **Architecting Scalability**:

1. **Vertical Integration:** When scale is massive, building your own hardware stack can offer optimizations that “Generic Cloud” cannot.
2. **Workload Matching:** Dropbox optimized for **WORM** workloads, allowing them to use high-density, slow-write SMR drives that would be unusable for a high-transaction SQL database.
3. **Cost as a Constraint:** In System Design, cost is a primary Non-Functional Requirement (NFR) that can eventually dictate the entire technical architecture.

Chapter 9

Labs

9.1 Experiment 1

Aim:

Back-of-the-Envelope Calculator: Create a spreadsheet or simple program to quickly calculate storage (TB/year) and QPS for a given user base (e.g., 100M Daily Active Users).

9.2 Experiment 2

Aim:

SQL vs. NoSQL Schema Modeling: Design the schema for a simple E-commerce cart: one using a normalized SQL model and one using a de-normalized Document/NoSQL model.

9.3 Experiment 3

Aim:

Consistent Hashing Simulation: Implement a small simulation of Consistent Hashing (using a language of choice, e.g., Python) to observe key redistribution when 1 node is added and 1 node is removed from an initial 3-node cluster.

9.4 Experiment 4

Aim:

Data Sharding Design: Propose 3 different sharding keys (eg., user_id, geolocation, date) for a highly-used logging service and analyze the pros/cons of each (e.g., data skew, query access pattern).

References

1. **Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019).** *Database System Concepts* (7th ed.). McGraw-Hill Education. (Reference for Parallel Databases, Intra/Inter Query Evaluation, and RAID architectures).
2. **Elmasri, R., & Navathe, S. B. (2015).** *Fundamentals of Database Systems* (7th ed.). Pearson. (Reference for Disk Storage, Query Processing, and Distributed Databases).
3. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications*. O'Reilly Media. (Reference for Consistent Hashing, Partitioning vs. Sharding, Replication Algorithms, PACELC, and the CAP theorem).
4. **Abadi, D. (2012).** *Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story*. IEEE Computer. (Primary source for the PACELC theorem).
5. **DeCandia, G., et al. (2007).** *Dynamo: Amazon's Highly Available Key-value Store*. In Proceedings of SOSR. (Primary source for Eventual Consistency, Quorum math, and the Amazon Shopping Cart case study).
6. **Beaver, D., et al. (2010).** *Finding a Needle in Haystack: Facebook's Photo Storage*. In Proceedings of OSDI. (Primary source for Object Storage vs. POSIX Filesystem limitations at scale).